

Table of Contents

Helper functions.....	2
Problem 1.....	16
1a.....	16
1b.....	16
1c.....	16
1d.....	16
1e.....	17
1f.....	17
1g.....	18
1h.....	19
1i.....	19
1j.....	20
1k.....	21
Problem 2.....	22
2a.....	22
2b.....	22
2c.....	22
2d.....	22
2e.....	22
2f.....	23
2g.....	23
Problem 3.....	25
3a.....	25
3b.....	25
3c.....	27
Problem 4.....	29
4a.....	29
4b.....	35
4c.....	37
4d.....	41
4e.....	50
4f.....	61
4g.....	63
4h.....	67

Helper functions

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% Helper functions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% plot_pole_zero
%
% Plot the pole-zero diagram, given numerator & denominator co-efficients
% of the transfer function.
% Parameters:
%   b - Co-efficients of the numerator (decreasing powers of  $z^{-1}$ )
%   a - Co-efficients of the denominator (decreasing powers of  $z^{-1}$ )
%   main_title - String for figure title
function plot_pole_zero(b, a, main_title)
    figure;
    % Plot the unit circle
    plot(exp(1j * 2 * pi * (0 : 0.01 : 1)), "LineWidth", 2);
    hold on;
    [hz, hp, ht] = zplane(b, a);
    % Set the zeros & poles to be bolder and in red
    set(findobj(hz, "Type", "line"), "LineWidth", 2, "Color", "r");
    set(findobj(hp, "Type", "line"), "LineWidth", 2, "Color", "r");
    % Make the default zplane circle, axis, etc. faint
    set(findobj(ht, "Type", "line"), "LineWidth", 0.01);
    hold off;
    if (main_title ~= "")
        title(sprintf("Poles and zeros; %s", main_title));
    else
        title("Poles and zeros");
    end
    grid on; grid minor; axis("square");
    axis([-1.2 1.2 -1.2 1.2]);
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% plot_freq_resp_from_imp_resp
%
% Plot the frequency response of a filter, given the impulse response and
% the sampling frequency.
% Parameters:
%   h - Impulse response (scaled to unity gain at DC)
%   NFFT - Number of points in FFT
%   fs - Sampling frequency (in Hz)
```

```

function plot_freq_resp_from_imp_resp(h, NFFT, fs)
    f_h = fftshift(20 * log10(abs(fft(h, NFFT))));
    f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs;

    figure;
    plot(f, f_h, "LineWidth", 2);
    grid on; grid minor; axis on;
    ylim([-100 10]);
    title("Frequency response");
    xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% plot_filt_resp_from_transfer_fn
%
% Plot the frequency response of a filter, given numerator & denominator
% co-efficients of the transfer function.
% Parameters:
%   b - Co-efficients of the numerator (decreasing powers of z^-1)
%   a - Co-efficients of the denominator (decreasing powers of z^-1)
%   fs - Sampling frequency (in Hz)
%   main_title - String for figure title
%   normalized_freq - Optional parameter to indicate whether frequency
%                     response is to be plotted with normalized frequency
%                     on the x-axis (default: false)
%   unity_gain - Optional parameter to indicate whether unity gain
%                magnitude response is to be plotted or not (default: false)
%   imp_resp - Optional parameter to indicate whether impulse response
%              is to be plotted or not (default: false)
%   pos_spectrum - Optional parameter to indicate whether frequency response
%                  is to be plotted only for positive spectrum (default:
false)
%   imp_resp_len - Optional parameter for length of unit impulse sequence
to be
%                  used for filter/sosfilt (default: 251)
%   spectral_mask - Optional parameter to indicate whether spectral mask is
to
%                  be shown on the frequency response (default: false)
%   Wp - Vector for passband edge(s), normalized to Nyquist frequency
%   Ws - Vector for stopband edge(s), normalized to Nyquist frequency
%   A_dB - Desired minimum stopband attenuation (in dB), for plotting
function plot_filt_resp_from_transfer_fn(...)
    b, a, fs, main_title, normalized_freq, unity_gain, imp_resp, ...
    pos_spectrum, imp_resp_len, spectral_mask, Wp, Ws, A_dB)
arguments
    b
    a
    fs
    main_title

```

```

normalized_freq = false
unity_gain = false
imp_resp = false
pos_spectrum = false
imp_resp_len = 251
spectral_mask = false
Wp = []
Ws = []
A_dB = 0
end
NFFT = 8192;

% Get the impulse response
% Check if we've an SOS or not
if (size(b, 1) > 1)
    h = sosfilt(b, [1 zeros(1, imp_resp_len - 1)]);
else
    h = filter(b, a, [1 zeros(1, imp_resp_len - 1)]);
    % Scale it to unity gain magnitude
    scl = 1 / sum(abs(h));
    h = h .* scl;
end

if (normalized_freq)
    f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT);
else
    f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs;
end

f_h = fft(h, NFFT);

figure("Position", [0 0 900 950]);

% Plot at most 251 samples to show details; especially in the case of IIR
h = h(1 : min(251, length(h)));
if (unity_gain) && (imp_resp)
    subplot(3, 1, 1);
elseif (~unity_gain) && (imp_resp)
    subplot(2, 1, 1);
end
if (imp_resp)
    plot(h, "LineWidth", 1.5);
    grid on; grid minor; axis on;
    axis([-2, length(h) + 2, 1.1 * min(h), 1.1 * max(h)]);
    title(sprintf("Impulse response; %s", main_title));
end

% Normally, we just plot the log magnitude gain, but plot the unity gain
% magnitude as well if unity_gain is set.
if (unity_gain) && (imp_resp)

```

```

    subplot(3, 1, 2);
elseif (unity_gain) && (~imp_resp)
    subplot(2, 1, 1);
end
if (unity_gain)
    plot(f, fftshift(abs(f_h)), "LineWidth", 1.5);
    grid on; grid minor; axis on;
    axis([-0.5, 0.5, min(abs(f_h)) - 0.2, max(abs(f_h)) + 0.2]);
    title(sprintf("Frequency response; %s", main_title));
    if (normalized_freq)
        xlabel("Normalized frequency (f/fs)");
    else
        xlabel("Frequency (Hz)");
    end
    ylabel("Magnitude");
end

if (unity_gain) && (imp_resp)
    subplot(3, 1, 3);
elseif (unity_gain) || (imp_resp)
    subplot(2, 1, 2);
end

plot(f, fftshift(20 * log10(abs(f_h))), "LineWidth", 1.5);
if ((spectral_mask) && size(Wp, 2) == 2 && size(Ws, 2) == 2)
    % Spectral mask for band pass filter
    hold on;
    Wp = Wp * fs/2;
    Ws = Ws * fs/2;
    fc = (Wp(1) + Wp(2)) / 2;
    plot([fc fc], [-100 10], ":r", "LineWidth", 1.5);
    %if (pos_spectrum)
    % plot([Wp(1) Wp(1) Wp(2) Wp(2)], [-100 0 0 -100], ":r", "LineWidth",
1.5);
    % plot([0 Ws(1) Ws(1)], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    % plot([Ws(2) Ws(2) fs/2], [0 -A_dB -A_dB], ":r", "LineWidth", 1.5);
    %else
    plot([Wp(1) Wp(1) Wp(2) Wp(2)], [-100 0 0 -100], ":r", "LineWidth",
1.5);
    plot([0 Ws(1) Ws(1)], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([Ws(2) Ws(2) fs/2], [0 -A_dB -A_dB], ":r", "LineWidth", 1.5);
    %end
    hold off;
end
grid on; grid minor; axis on;
ylim([-100 10]);
if (pos_spectrum)
    if (normalized_freq)
        xlim([0, 0.5]);
    else

```

```

        xlim([0 fs/2]);
    end
end
if (pos_spectrum)
    title(...
        sprintf("Frequency response (positive spectrum only); %s",
main_title));
else
    title(sprintf("Frequency response; %s", main_title));
end
if (normalized_freq)
    xlabel("Normalized frequency (f/fs)");
else
    xlabel("Frequency (Hz)");
end
ylabel("Log magnitude (dB)");
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%
% design_plot_fir_iir_bandpass_filt_resp
%
% Given below filter specifications, designs both FIR & IIR bandpass
filters and
% plots their frequency response side side-by-side.
%
% Parameters:
%   fc - Center frequency (in Hz)
%   fs - Sampling frequency (in Hz)
%   Rp - Desired maximum passband (in-band) ripple (in dB)
%   A_dB - Desired minimum stopband attenuation (in dB)
%   main_title - String for figure title
%   FPASS_HALFBW - Half of passband width (in Hz); passband being centered
%                 at fc
%   FSTOP_WIDTH_FIR - Width of transition band for FIR filter (in Hz)
%   FSTOP_WIDTH_IIR - Width of transition band for IIR filter (in Hz)
function design_plot_fir_iir_bandpass_filt_resp(...
    fc, fs, Rp, A_dB, main_title, FPASS_HALFBW, FSTOP_WIDTH_FIR, ...
    FSTOP_WIDTH_IIR)
NFFT = 8192;
f = [0, ...
    (fc - (FPASS_HALFBW + FSTOP_WIDTH_FIR)) / (fs/2), ...
    (fc - FPASS_HALFBW) / (fs/2), ...
    (fc + FPASS_HALFBW) / (fs/2), ...
    (fc + (FPASS_HALFBW + FSTOP_WIDTH_FIR)) / (fs/2), 1];

Ds = 10^(-A_dB / 20);    % Stopband deviation (linear)
Dp = (10^(Rp / 20)) - 1; % Passband deviation (linear)

dev = [Ds, Dp, Ds];

```

```

[NFILTER, fo, ao, w] = firpmord(f(2:5) * fs/2, [0 1 0], dev, fs);

% NOTE: "myfrf" does not work with band pass filter?
%b = firpm(NFILTER, f, {"myfrf", a});

% The filter order returned by firpmord does not meet the spec w.r.t.
passband
% ripple and stopband attenuation. Hence, increasing the filter order.
NFILTER = NFILTER + 21;

% Using Remez/Parks-McClellan algorithm to design the FIR bandpass
filters.
b = firpm(NFILTER, fo, ao, w);

% Scaling for plotting impulse response
h = b;
scl = max(h);
h_scaled = h / scl;

f_h = fftshift(20 * log10(abs(fft(h, NFFT))));

figure("Position", [0 0 900 950]);
sgtitle(...
{...
    sprintf(...
        "%s Bandpass Filter, f_s = %d kHz, f_{center} = %d Hz", ...
        main_title, fs/1000, fc), ...
    "f_{pass} BW = 20 Hz"});

subplot(3, 2, 1);
plot(h_scaled, "LineWidth", 1.5);
grid on; grid minor;
axis([-2, length(h) + 2, min(h_scaled) - 0.2, max(h_scaled) + 0.2]);
% firpm returns (n + 1) coefficients
title(sprintf("Impulse response of FIR filter (order = %d)", NFILTER +
1));
xlabel("Time index"); ylabel("Amplitude");

f = f * fs/2;
subplot(3, 2, 3);
hold on;
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, f_h, "LineWidth", 1.5);
% Spectral mask for band pass filter
plot([fc fc], [-100 10], ":r", "LineWidth", 1.5);
plot([f(3) f(3) f(4) f(4)], [-100 0 0 -100], ":r", "LineWidth", 1.5);
plot([f(1) f(2) f(2)], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
plot([f(5) f(5) f(6)], [0 -A_dB -A_dB], ":r", "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([0.9 * f(2), 1.1 * f(5), -100, 10]);

```

```

title(...
{...
    sprintf(...
        "Frequency response of FIR filter"), ...
    sprintf(...
        "f_{transition} BW = %d Hz; limited to spectrum of interest", ...
        FSTOP_WIDTH_FIR));
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

% Checking passband ripple in range (start_idx:end_idx) corresponding to
the
% positive frequency passband frequency response
start_idx = NFFT/2 + int32(((f(3) / (fs/2)) * NFFT/2));
end_idx = NFFT/2 + int32(((f(4) / (fs/2)) * NFFT/2));
max_dev = max(f_h(start_idx:end_idx));

subplot(3, 2, 5);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, f_h, "LineWidth", 1.5);
hold on;
y_lim = 2 * max_dev; % y-axis limit, scaled according to the peak
deviation
plot(...
    [f(3) f(3) f(4) f(4)], [-y_lim -max_dev -max_dev -y_lim], ":r", ...
    "LineWidth", 1.5);
plot(...
    [f(3) f(3) f(4) f(4)], [y_lim max_dev max_dev y_lim], ":r", ...
    "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([f(3) - 5, f(4) + 5, -y_lim, y_lim]);
title("Zoom to passband ripple of FIR filter");
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

% Passband edge frequencies (normalized to Nyquist frequency)
Wp = [fc - FPASS_HALFBW, ...
    fc + FPASS_HALFBW] / (fs/2);

% Stopband edge frequencies (normalized to Nyquist frequency)
Ws = [fc - (FPASS_HALFBW + FSTOP_WIDTH_IIR), ...
    fc + (FPASS_HALFBW + FSTOP_WIDTH_IIR)] / (fs/2);

NFILTER = cheb2ord(Wp, Ws, Rp, A_dB);
[B3_lo, A3_lo] = cheby2(NFILTER, A_dB, Ws);

% Convert to Second-Order Systems to account for numerical instabilities
sos = tf2sos(B3_lo, A3_lo);

NIMPSEQ = 1251; % Length of impulse series

% Get the impulse response

```

```

h = sosfilt(sos, [1 zeros(1, NIMPSEQ-1)]);

% Scaling for plotting impulse response (plot only 251 to show details)
scl = max(h(1:251));
h_scaled = h(1:251) / scl;

f_h = fftshift(20 * log10(abs(fft(h, NFFT))));

subplot(3, 2, 2);
plot(h_scaled, "LineWidth", 1.5);
grid on; grid minor;
axis([-2, length(h_scaled) + 2, min(h_scaled) - 0.2, max(h_scaled) + 0.2]);
% Actual filter order is double that returned by cheb2ord for bandpass
title(sprintf("Impulse response of IIR filter (order = %d)", 2 *
NFILTER));
xlabel("Time index"); ylabel("Amplitude");

subplot(3, 2, 4);
Wp = Wp * fs/2;
Ws = Ws * fs/2;
hold on;
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, f_h, "LineWidth", 1.5);
% Spectral mask for band pass filter
plot([fc fc], [-100 10], ":r", "LineWidth", 1.5);
plot([Wp(1) Wp(1) Wp(2) Wp(2)], [-100 0 0 -100], ":r", "LineWidth", 1.5);
plot([0 Ws(1) Ws(1)], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
plot([Ws(2) Ws(2) fs/2], [0 -A_dB -A_dB], ":r", "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([0.9 * Ws(1), 1.1 * Ws(2), -100, 10]);
title(...
{...
    sprintf(...
        "Frequency response of IIR filter; f_{transition} BW = %d Hz", ...
        FSTOP_WIDTH_IIR), ...
    "Limited to spectrum of interest"});
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

% Checking passband ripple in range (start_idx:end_idx) corresponding to
the
% positive frequency passband frequency response
start_idx = NFFT/2 + int32(((Wp(1) / (fs/2)) * NFFT/2));
end_idx = NFFT/2 + int32(((Wp(2) / (fs/2)) * NFFT/2));
max_dev = max(abs(f_h(start_idx:end_idx)));

subplot(3, 2, 6);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, f_h, "LineWidth", 1.5);
hold on;

```

```

y_lim = 2 * max_dev; % y-axis limit, scaled according to the peak
deviation
plot(...)
    [Wp(1) Wp(1) Wp(2) Wp(2)], [-y_lim -max_dev -max_dev -y_lim], ":r", ...
    "LineWidth", 1.5);
plot(...)
    [Wp(1) Wp(1) Wp(2) Wp(2)], [y_lim max_dev max_dev y_lim], ":r", ...
    "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([Wp(1) - 5, Wp(2) + 5, -y_lim, y_lim]);
title("Zoom to passband ripple of IIR filter");
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% plot_fir_iir_filt_resp
%
% Plot the frequency response of FIR & IIR filters for the same filter
% specification, given the impulse responses and other characteristics.
%
% Parameters:
%   h_fir - FIR filter impulse response
%   h_iir - IIR filter impulse response
%   n_fir - FIR filter order
%   n_iir - IIR filter order
%   f1 - Passband edge (in Hz)
%   f2 - Stopband edge (in Hz)
%   fs - Sampling frequency (in Hz)
%   A_dB - Desired minimum stopband attenuation (in dB)
%   filter_type - Optional parameter to indicate whether this is a low or a
%   high
%                   pass filter (default: "Low", values: "Low", "High")
function plot_fir_iir_filt_resp(...)
    h_fir, h_iir, n_fir, n_iir, f1, f2, fs, A_dB, filter_type, ...
    main_title, lim_spectrum, IIRLEN)
arguments
    h_fir
    h_iir
    n_fir
    n_iir
    f1
    f2
    fs
    A_dB
    filter_type = "Low"
    main_title = ""
    lim_spectrum = false
    IIRLEN = length(h_iir)

```

```

end
NFFT = 8192;

% Scaling for plotting impulse response
scl = max(h_fir);
h_fir_scaled = h_fir / scl;
scl = max(h_iir);
if IIRLEN < length(h_iir)
    h_iir_scaled = h_iir(1 : IIRLEN) / scl;
else
    h_iir_scaled = h_iir / scl;
end

figure("Position", [0 0 900 950]);
if main_title ~= ""
    sgtitle(main_title)
else
    sgtitle(...
        sprintf(...
            "%s Pass Filter, f_s = %d kHz, f_{pass} = %.5g kHz, " + ...
            "f_{stop} = %.5g kHz", ...
            filter_type, fs/1000, f1/1000, f2/1000));
end

subplot(3, 2, 1);
plot((0 : length(h_fir_scaled) - 1), h_fir_scaled, "LineWidth", 1.5);
grid on; grid minor;
axis(...
    [-2, length(h_fir_scaled) + 2, ...
     min(h_fir_scaled) - 0.2, max(h_fir_scaled) + 0.2]);
title(sprintf("Impulse response of FIR filter (order = %d)", n_fir));
xlabel("Time index"); ylabel("Amplitude");

subplot(3, 2, 2);
plot((0 : length(h_iir_scaled) - 1), h_iir_scaled, "LineWidth", 1.5);
grid on; grid minor;
axis(...
    [-2, length(h_iir_scaled) + 2, ...
     min(h_iir_scaled) - 0.2, max(h_iir_scaled) + 0.2]);
title(sprintf("Impulse response of IIR filter (order = %d)", n_iir));
xlabel("Time index"); ylabel("Amplitude");

f_h_fir = fftshift(20 * log10(abs(fft(h_fir, NFFT))));
f_h_iir = fftshift(20 * log10(abs(fft(h_iir, NFFT))));

f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs;
subplot(3, 2, 3);
plot(f, f_h_fir, "LineWidth", 1.5);
hold on;
if (filter_type == "Low")

```

```

    % Spectral mask for low pass filter
    plot([-fs/2 -f2 -f2], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([fs/2 f2 f2], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([-f1 -f1 f1 f1], [-100 0 0 -100], ":r", "LineWidth", 1.5);
elseif (filter_type == "High")
    % Spectral mask for high pass filter
    plot([-f2 -f2 f2 f2], [0 -A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([-fs/2 -f1 -f1], [0 0 -100], ":r", "LineWidth", 1.5);
    plot([f1 f1 fs/2], [-100 0 0], ":r", "LineWidth", 1.5);
end
hold off;
grid on; grid minor;
if (lim_spectrum)
    axis([0.9 * f2, 1.1 * f1, -100, 10]);
else
    axis([-fs/2 fs/2 -100 10]);
end
if (lim_spectrum)
    title(...
        {"Frequency response of FIR filter", "Limited to spectrum of
interest"});
else
    title(...
        "Frequency response of FIR filter");
end
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

subplot(3, 2, 4);
plot(f, f_h_iir, "LineWidth", 1.5);
hold on;
if (filter_type == "Low")
    % Spectral mask for low pass filter
    plot([-fs/2 -f2 -f2], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([fs/2 f2 f2], [-A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([-f1 -f1 f1 f1], [-100 0 0 -100], ":r", "LineWidth", 1.5);
elseif (filter_type == "High")
    % Spectral mask for high pass filter
    plot([-f2 -f2 f2 f2], [0 -A_dB -A_dB 0], ":r", "LineWidth", 1.5);
    plot([-fs/2 -f1 -f1], [0 0 -100], ":r", "LineWidth", 1.5);
    plot([f1 f1 fs/2], [-100 0 0], ":r", "LineWidth", 1.5);
end
hold off;
grid on; grid minor;
if (lim_spectrum)
    axis([0.9 * f2, 1.1 * f1, -100, 10]);
else
    axis([-fs/2 fs/2 -100 10]);
end
if (lim_spectrum)
    title(...)

```

```

        {"Frequency response of IIR filter", "Limited to spectrum of
interest"}});
    else
        title(...
            "Frequency response of IIR filter");
    end
    xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

    % Checking passband ripple in range (start_idx:end_idx) corresponding to
the
    % positive frequency passband frequency response
    if (filter_type == "Low")
        start_idx = NFFT / 2;
        end_idx = int32(start_idx + ((f1 / fs) * NFFT));
        max_dev = max(f_h_fir(start_idx:end_idx));
    elseif (filter_type == "High")
        end_idx = NFFT;
        start_idx = end_idx - int32(((fs/2 - f1) / (fs)) * NFFT);
        max_dev = max(f_h_fir(start_idx:end_idx));
    end

    subplot(3, 2, 5);
    plot(f, f_h_fir, "LineWidth", 1.5);
    hold on;
    y_lim = 2 * max_dev; % y-axis limit, scaled according to the peak
deviation
    % Show the max ripple in passband
    if (filter_type == "Low")
        plot(...
            [-f1 -f1 f1 f1], [-y_lim -max_dev -max_dev -y_lim], ":r",
"LineWidth", 1.5);
        plot([-f1 -f1 f1 f1], [y_lim max_dev max_dev y_lim], ":r", "LineWidth",
1.5);
    elseif (filter_type == "High")
        plot(...
            [f1 f1 fs/2 fs/2], [y_lim max_dev max_dev y_lim], ":r", "LineWidth",
1.5);
        plot(...
            [f1 f1 fs/2 fs/2], [-y_lim -max_dev -max_dev -y_lim], ":r", ...
            "LineWidth", 1.5);
    end
    hold off;
    grid on; grid minor;
    if (lim_spectrum)
        axis([f1 - 50, fs/2, -y_lim, y_lim]);
        title(...
            {"Zoom to passband ripple of FIR filter", ...
            "Limited to spectrum of interest"});
    else
        if (filter_type == "Low")

```

```

    f0 = (f1 + f2) / 2;
    axis([-f0 f0 -y_lim y_lim]);
elseif (filter_type == "High")
    f0 = (fs - (f1 + f2)) / 2;
    axis([f0 fs/2 -y_lim y_lim]);
end
title("Zoom to passband ripple of FIR filter");
end
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

% Checking passband ripple in range (start_idx:end_idx) corresponding to
the
% positive frequency passband frequency response
if (filter_type == "Low")
    start_idx = NFFT / 2;
    end_idx = int32(start_idx + ((f1 / fs) * NFFT));
    max_dev = max(abs(f_h_iir(start_idx:end_idx)));
elseif (filter_type == "High")
    end_idx = NFFT;
    start_idx = end_idx - int32(((fs/2 - f1) / (fs)) * NFFT);
    max_dev = max(abs(f_h_iir(start_idx:end_idx)));
end

subplot(3, 2, 6);
plot(f, f_h_iir, "LineWidth", 1.5);
hold on;
y_lim = 2 * max_dev; % y-axis limit, scaled according to the peak
deviation
% Show the max ripple in passband
if (filter_type == "Low")
    plot(...
        [-f1 -f1 f1 f1], [-y_lim -max_dev -max_dev -y_lim], ":r",
"LineWidth", 1.5);
    plot([-f1 -f1 f1 f1], [y_lim max_dev max_dev y_lim], ":r", "LineWidth",
1.5);
elseif (filter_type == "High")
    plot(...
        [f1 f1 fs/2 fs/2], [y_lim max_dev max_dev y_lim], ":r", "LineWidth",
1.5);
    plot(...
        [f1 f1 fs/2 fs/2], [-y_lim -max_dev -max_dev -y_lim], ":r", ...
"LineWidth", 1.5);
end
hold off;
grid on; grid minor;
if (lim_spectrum)
    axis([f1 - 50, fs/2, -y_lim, y_lim]);
    title(...
        {"Zoom to passband ripple of IIR filter", ...
"Limited to spectrum of interest"});
end

```

```

else
    if (filter_type == "Low")
        f0 = (f1 + f2) / 2;
        axis([-f0 f0 -y_lim y_lim]);
    elseif (filter_type == "High")
        f0 = (fs - (f1 + f2)) / 2;
        axis([f0 fs/2 -y_lim y_lim]);
    end
    title("Zoom to passband ripple of IIR filter");
end
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");
end

```

Problem 1

1a

Difference equation for the given system is:

$$y[n] = h[0]x[n] + h[1]x[n-1] + h[2]x[n-2] + h[3]x[n-3] + h[4]x[n-4]$$

$$y[n] = \sum_{k=0}^4 x[n-k]h[k]$$

1b

Impulse response:

$$x[n] = \delta[n]$$

$$y[n] = h[0]\delta[n] + h[1]\delta[n-1] + h[2]\delta[n-2] + h[3]\delta[n-3] + h[4]\delta[n-4]$$

$$y[n] = \sum_{k=0}^4 \delta[n-k]h[k]$$

1c

The difference equation is basically a convolution of the input sequence and $h[n]$:

$$y[n] = \sum_{k=0}^4 x[n-k]h[k]$$

$$y[n] = x[n] * h[n]$$

Convolution in the time-domain translates to multiplication in the z-domain:

$$Y(z) = H(z)X(z)$$

1d

Closed form z-transform for

$$h_1[n] = \{1, 1, 1, 1, 1\}$$

$$H_1(z) = \sum_{k=-\infty}^{\infty} h_1[k]z^{-k} = \sum_{k=0}^4 z^{-k} = 1 + z^{-1} + z^{-2} + z^{-3} + z^{-4}$$

$$H_1(z) = \frac{1 - z^{-5}}{1 - z^{-1}} = \frac{z^5 - 1}{z^4(z - 1)}; ROC: z \neq 0$$

or

$$H_1(z) = \frac{1}{z^4} \cdot \frac{z^{-2.5}(z^{2.5} - z^{-2.5})}{z^{-0.5}(z^{0.5} - z^{-0.5})} = \frac{1}{z^6} \cdot \frac{z^{2.5} - z^{-2.5}}{z^{0.5} - z^{-0.5}}$$

1e

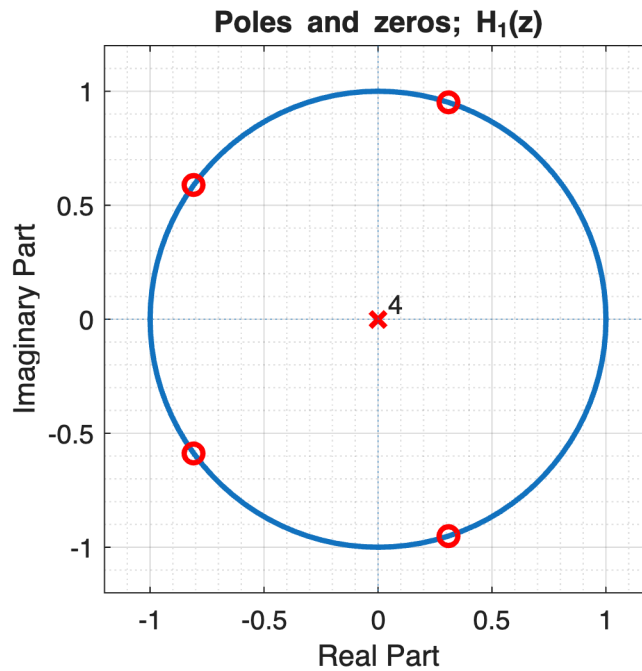
From inspecting the z-transform:

5 poles: 4 poles at $z = 0$ & 1 pole at $z = 1$

5 zeros: 5 zeros equally spaced on the unit circle (including one at $z = 1$) since the roots are solved by the equation, $z^5 = 1^5 \cdot e^{j2\pi k}$ or $z = e^{j2\pi k/5}; k = 0, 1, 2, 3, 4$

Matlab code for the pole-zero diagram:

```
plot_pole_zero([1 1 1 1 1], 1, "H_1(z)");
```



1f

Closed form frequency response:

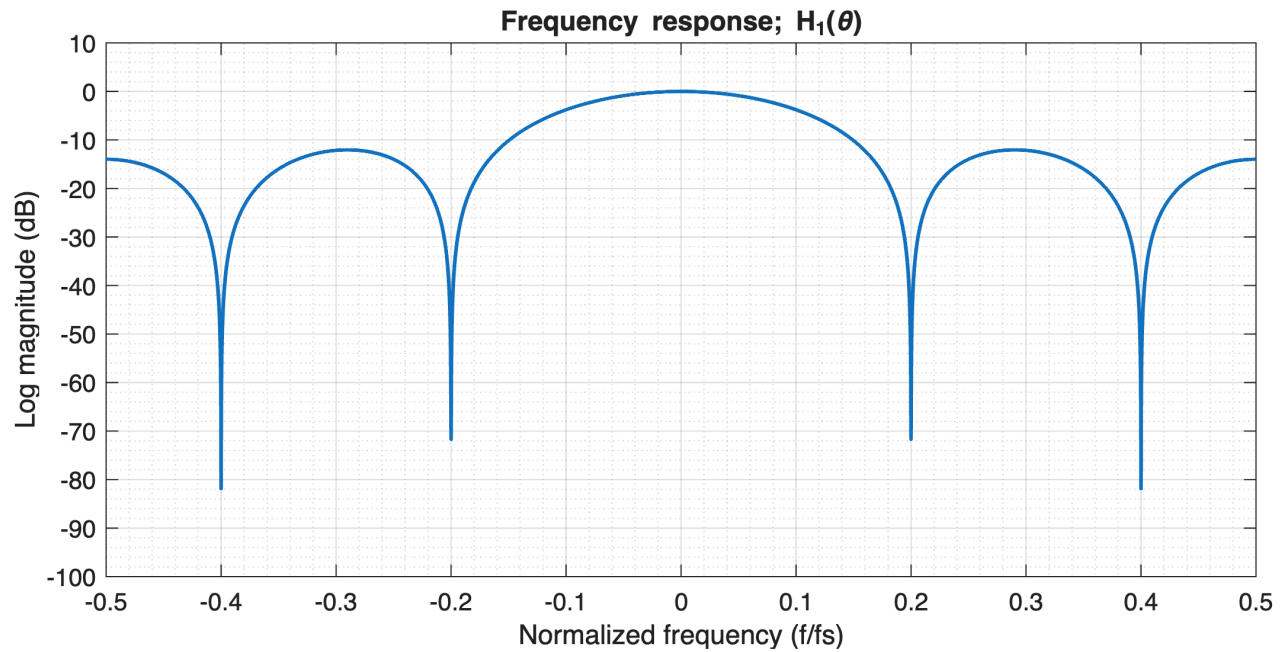
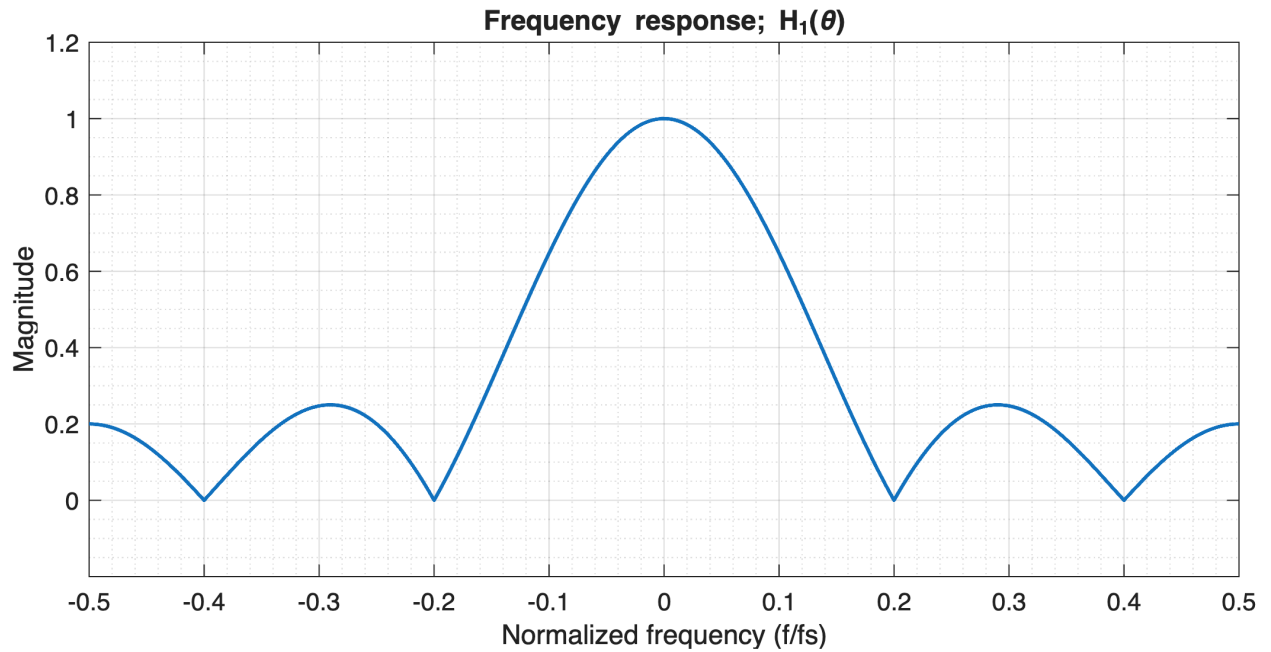
$$H_1(z) = \frac{1}{z^6} \cdot \frac{z^{2.5} - z^{-2.5}}{z^{0.5} - z^{-0.5}}$$

$$H_1(\theta) = H_1(z)|_{z=e^{j\theta}} = e^{-j6\theta} \frac{e^{j2.5\theta} - e^{-j2.5\theta}}{e^{j0.5\theta} - e^{-j0.5\theta}}$$

$$H_1(\theta) = e^{-j6\theta} \frac{\sin(2.5\theta)}{\sin(0.5\theta)}$$

1g

```
% Plot frequency response  
plot_filt_resp_from_transfer_fn(...  
    [1 1 1 1 1], 1, 8192, "H_1(\theta)", true, true);
```



1h

From inspection, given filter's impulse response is a heterodyne of the previous filter.

That is, $h_2[n] = h_1[n]\cos(\pi(-M : M))$; $M = (\text{Length of filter} - 1) / 2$

$$h_2[n] = \{1, -1, 1, -1, 1\}$$

This can also be written as $h_2[n] = h_1[n] (-1)^{-n}$

$$H_2(z) = \sum_{k=-\infty}^{\infty} h_2[k]z^{-k} = 1 - z^{-1} + z^{-2} - z^{-3} + z^{-4}$$

$$H_2(z) = \frac{z^4 - z^3 + z^2 - z + 1}{z^4}; \text{ROC} : z \neq 0$$

Since $h_2[n] = h_1[n](-1)^{-n}$

$$H_2(z) = \sum_{k=-\infty}^{\infty} h_2[k]z^{-k} = \sum_{k=0}^4 h_1[k](-1)^{-k}z^{-k} = \sum_{k=0}^4 h_1[k](-z)^{-k}$$

$$H_2(z) = H_1(-z)$$

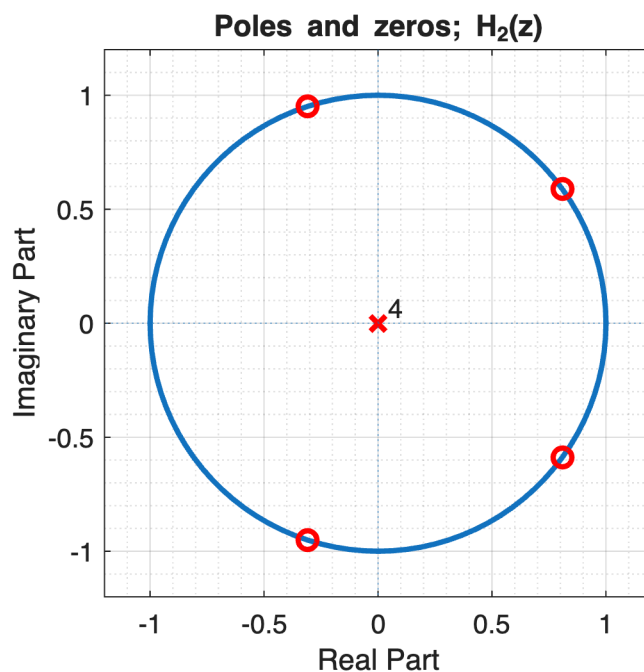
$$H_2(z) = \frac{(-z)^5 - 1}{(-z)^4(-z - 1)} = \frac{z^5 + 1}{z^4(z + 1)}$$

or

$$H_2(z) = \frac{1}{z^4} \cdot \frac{z^{2.5}(z^{2.5} + z^{-2.5})}{z^{0.5}(z^{0.5} + z^{-0.5})} = z^2 \cdot \frac{z^{2.5} + z^{-2.5}}{z^{0.5} + z^{-0.5}}$$

1i

```
plot_pole_zero([1 -1 1 -1 1], 1, "H_2(z)");
```



1j

Closed form frequency response

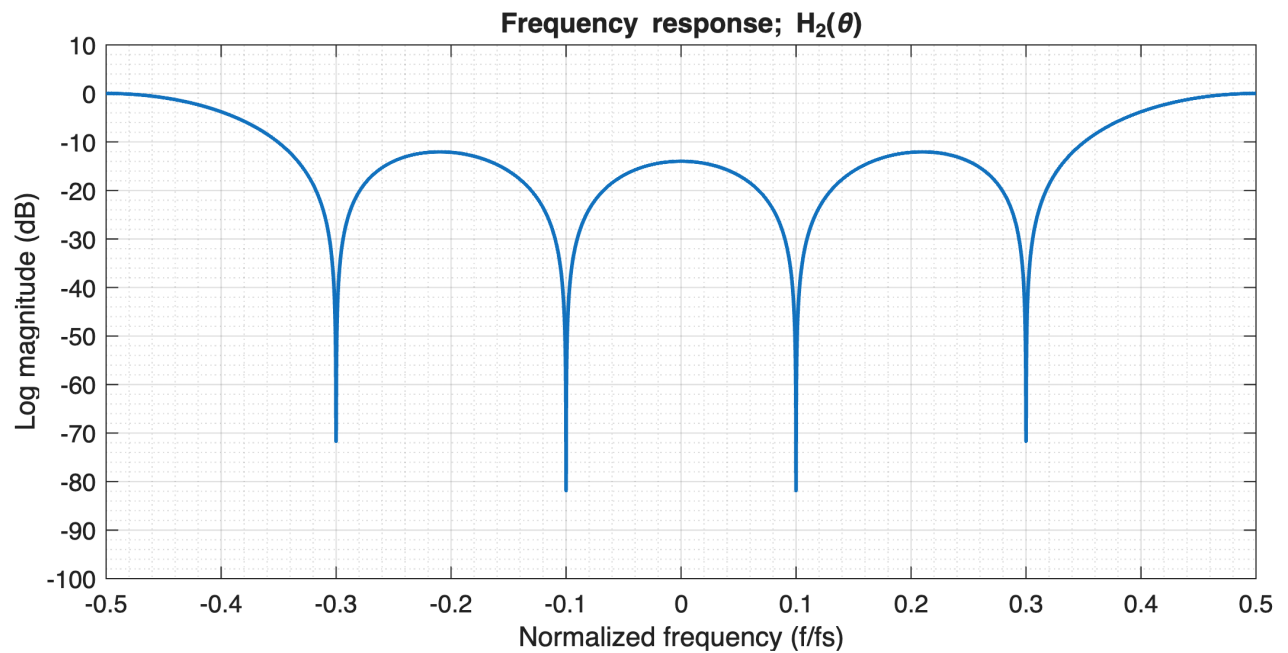
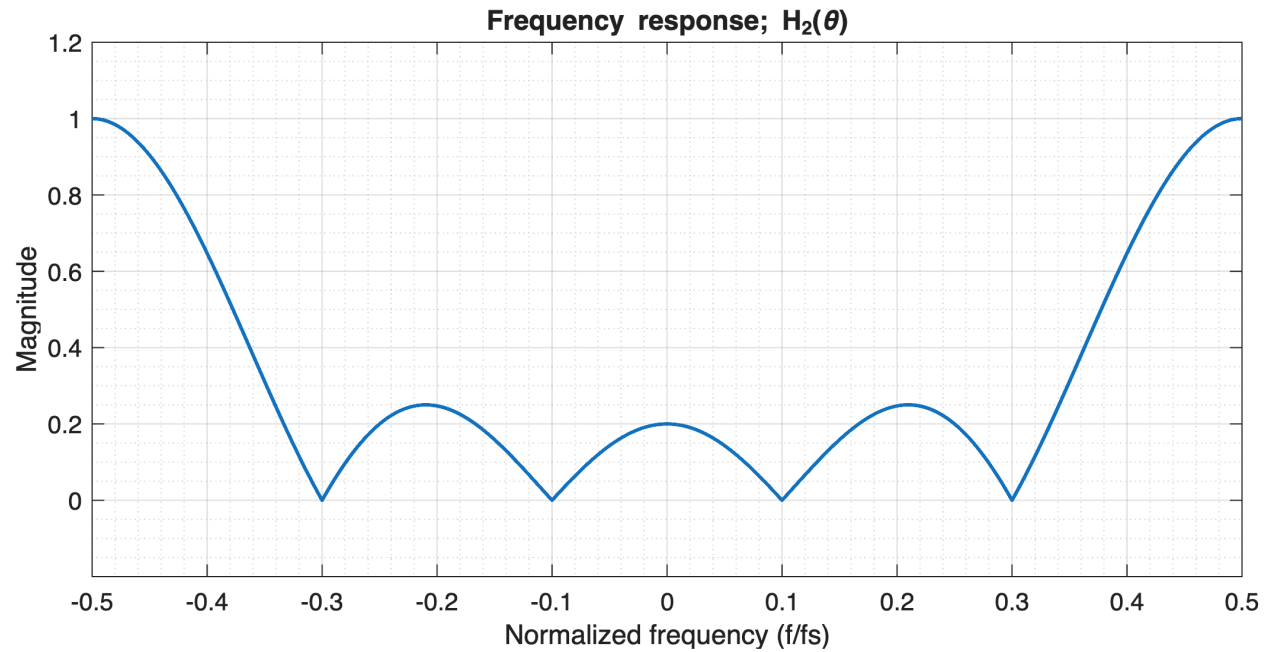
$$H_2(z) = z^2 \cdot \frac{z^{2.5} + z^{-2.5}}{z^{0.5} + z^{-0.5}}$$

$$H_2(\theta) = H_2(z)|_{z=e^{j\theta}} = e^{j2\theta} \frac{e^{j2.5\theta} + e^{-j2.5\theta}}{e^{j0.5\theta} + e^{-j0.5\theta}}$$

$$H_2(\theta) = e^{j2\theta} \frac{\cos(2.5\theta)}{\cos(0.5\theta)}$$

1k

```
% Plot frequency responses  
plot_filt_resp_from_transfer_fn(...  
    [1 -1 1 -1 1], 1, 8192, "H_2(\theta)", true, true);
```



Problem 2

2a

$$w_1[n] = -a_{1,1}w_1[n-1] + x[n]$$

$$\frac{W_1(z)}{X(z)} = \frac{1}{1 + a_{1,1}z^{-1}} = \frac{z}{z + a_{1,1}}$$

2b

$$s_1[n] = w_1[n] + b_{1,1}w_1[n-1]$$

$$\frac{S_1(z)}{W_1(z)} = 1 + b_{1,1}z^{-1} = \frac{z + b_{1,1}}{z}$$

2c

$$\frac{S_1(z)}{X(z)} = \frac{W_1(z)}{X(z)} \cdot \frac{S_1(z)}{W_1(z)} = \frac{1 + b_{1,1}z^{-1}}{1 + a_{1,1}z^{-1}} = \frac{z + b_{1,1}}{z + a_{1,1}}$$

$$y_1[n] = b_{1,0}s_1[n]$$

$$\frac{Y_1(z)}{X(z)} = \frac{S_1(z)}{X(z)} \cdot \frac{Y_1(z)}{S_1(z)} = b_{1,0} \frac{1 + b_{1,1}z^{-1}}{1 + a_{1,1}z^{-1}} = b_{1,0} \frac{z + b_{1,1}}{z + a_{1,1}}$$

2d

$$w_2[n] = -a_{2,1}w_2[n-1] - a_{2,2}w_2[n-2] + y_1[n]$$

$$\frac{W_2(z)}{Y_1(z)} = \frac{1}{1 + a_{2,1}z^{-1} + a_{2,2}z^{-2}} = \frac{z^2}{z^2 + a_{2,1}z + a_{2,2}}$$

2e

$$s_2[n] = w_2[n] + b_{2,1}w_2[n-1] + b_{2,2}w_2[n-2]$$

$$\frac{S_2(z)}{W_2(z)} = 1 + b_{2,1}z^{-1} + b_{2,2}z^{-2} = \frac{z^2 + b_{2,1}z + b_{2,2}}{z^2}$$

2f

$$\frac{S_2(z)}{Y_1(z)} = \frac{W_2(z)}{Y_1(z)} \cdot \frac{S_2(z)}{W_2(z)} = \frac{1 + b_{2,1}z^{-1} + b_{2,2}z^{-2}}{1 + a_{2,1}z^{-1} + a_{2,2}z^{-2}} = \frac{z^2 + b_{2,1}z + b_{2,2}}{z^2 + a_{2,1}z + a_{2,2}}$$

$$y_2[n] = b_{2,0}s_2[n]$$

$$\frac{Y_2(z)}{Y_1(z)} = \frac{S_2(z)}{Y_1(z)} \cdot \frac{Y_2(z)}{S_2(z)} = b_{2,0} \frac{1 + b_{2,1}z^{-1} + b_{2,2}z^{-2}}{1 + a_{2,1}z^{-1} + a_{2,2}z^{-2}} = b_{2,0} \frac{z^2 + b_{2,1}z + b_{2,2}}{z^2 + a_{2,1}z + a_{2,2}}$$

2g

$$H_1(z) = b_{1,0} \frac{z + 1}{z - 0.5606}$$

$$H_2(z) = b_{2,0} \frac{z^2 + 1.4920z + 1}{z^2 - 1.1587z + 0.6184}$$

DC gain will be $H_1(1)$ and $H_2(1)$ respectively. ($z = e^{j\theta}$ and $\theta = 0$ for DC)

$$H_1(1) = 1$$

$$b_{1,0} = \frac{1 - 0.5606}{1 + 1} = 0.2197$$

$$H_2(1) = 1$$

$$b_{2,0} = \frac{1 - 1.1587 + 0.6184}{1 + 1.4920 + 1} = 0.1316$$

```
a11 = -0.5606; b11 = +1.0000; b10 = 0.2197;
a21 = -1.1587; a22 = +0.618;
b21 = +1.4920; b22 = +1.0000; b20 = 0.1316;
```

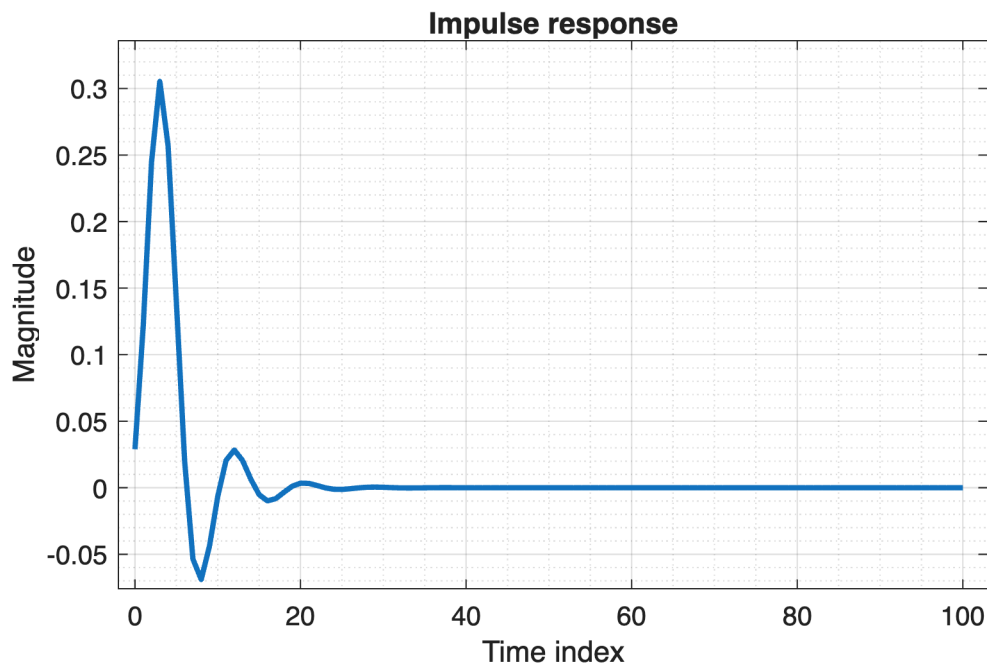
```
w1 = 0; % registers for filter 1
w2 = [0 0]; % registers for filter 2
x = [1 zeros(1, 100)]; % input time series
y1 = zeros(1, 101); % output time series, filter 1
y2 = zeros(1, 101); % output time series, filter 2

for n = 1:100 % for loop
    sm1 = x(n) - a11 * w1; % input sum, filter 1
    s1 = sm1 + b11 * w1; % output sum, filter 1
    y1(n) = b10 * s1; % scale filter output, filter 1
    w1 = sm1; % update register, filter 1

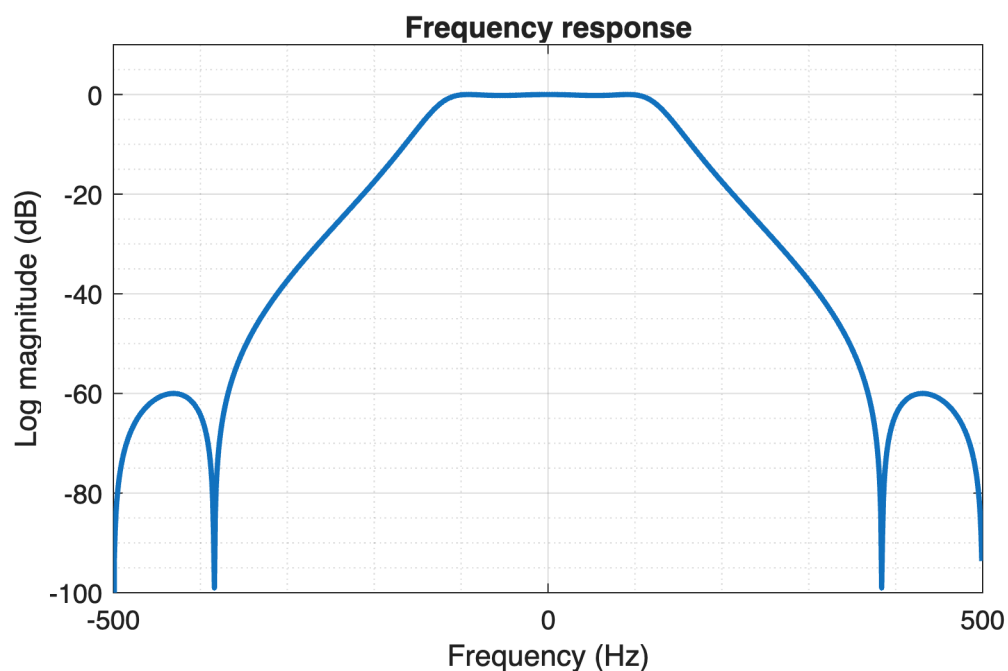
    sm2 = y1(n) - a21 * w2(1) - a22 * w2(2); % input sum, filter 2
    s2 = sm2 + b21 * w2(1) + b22 * w2(2); % output sum, filter 2
    y2(n) = b20 * s2; % scale filter output, filter 2
    w2 = [sm2 w2(1)]; % update registers, filter 2
```

```
end
```

```
figure;  
plot((0 : length(y2) - 1), y2, "LineWidth", 2);  
grid on; grid minor; axis on;  
axis([-2, length(y2) + 2, 1.1 * min(y2), 1.1 * max(y2)]);  
title("Impulse response");  
xlabel("Time index"); ylabel("Magnitude");
```



```
plot_freq_resp_from_imp_resp(y2, 1024, 1000);
```



Problem 3

3a

$$H_1(z) = b_{1,0} \frac{z + 1}{z - 0.4734}$$

$$H_2(z) = b_{2,0} \frac{z^2 + 0.5470z + 1}{z^2 - 1.0625z + 0.3498}$$

$$H_3(z) = b_{3,0} \frac{z^2 - 0.3959z + 1}{z^2 - 1.3811z + 0.7130}$$

DC gain will be $H_1(1)$, $H_2(1)$ and $H_3(1)$ respectively.

$$H_1(1) = 1$$

$$b_{1,0} = \frac{1 - 0.4734}{1 + 1} = 0.2633$$

$$H_2(1) = 1$$

$$b_{2,0} = \frac{1 - 1.0625 + 0.3498}{1 + 0.5470 + 1} = 0.1128$$

$$H_3(1) = 1$$

$$b_{3,0} = \frac{1 - 1.3811 + 0.7130}{1 - 0.3959 + 1} = 0.2069$$

3b

```
a11 = -0.4734; b11 = +1.0000; b10 = 0.2633;
a21 = -1.0625; a22 = +0.3498;
b21 = +0.5470; b22 = +1.0000; b20 = 0.1128;
a31 = -1.3811; a33 = +0.7130;
b31 = -0.3959; b33 = +1.0000; b30 = 0.2069;

w1 = 0; % registers for filter 1
w2 = [0 0]; % registers for filter 2
w3 = [0 0]; % registers for filter 3
x = [1 zeros(1, 100)]; % input time series
y1 = zeros(1, 101); % output time series, filter 1
y2 = zeros(1, 101); % output time series, filter 2
y3 = zeros(1, 101); % output time series, filter 3

for n = 1:100 % for loop
    sm1 = x(n) - a11 * w1; % input sum, filter 1
    s1 = sm1 + b11 * w1; % output sum, filter 1
    y1(n) = b10 * s1; % scale filter output, filter 1
    w1 = sm1; % update register, filter 1
```

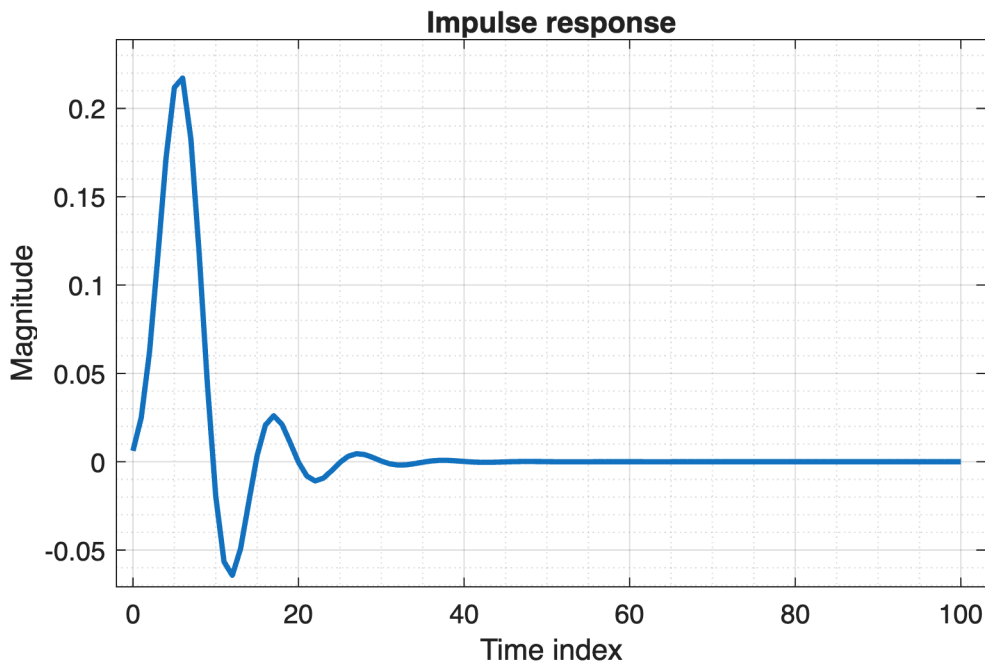
```

sm2 = y1(n) - a21 * w2(1) - a22 * w2(2); % input sum, filter 2
s2 = sm2 + b21 * w2(1) + b22 * w2(2); % output sum, filter 2
y2(n) = b20 * s2; % scale filter output, filter 2
w2 = [sm2 w2(1)]; % update registers, filter 2
sm2 = y1(n) - a21 * w2(1) - a22 * w2(2); % input sum, filter 2

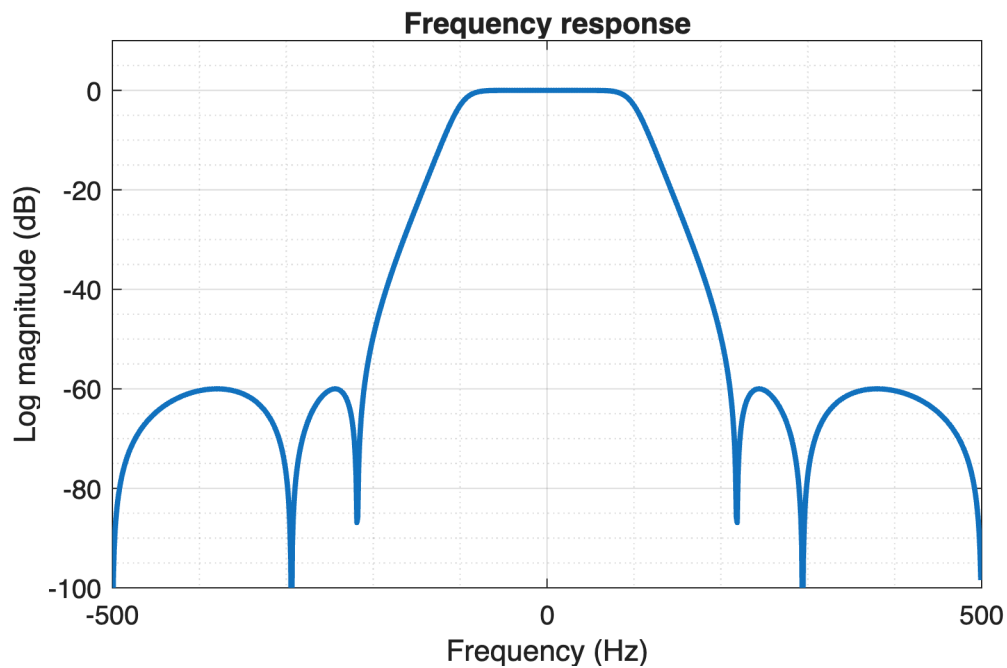
sm3 = y2(n) - a31 * w3(1) - a33 * w3(2); % input sum, filter 3
s3 = sm3 + b31 * w3(1) + b33 * w3(2); % output sum, filter 3
y3(n) = b30 * s3; % scale filter output, filter 3
w3 = [sm3 w3(1)]; % update registers, filter 3
sm3 = y2(n) - a31 * w3(1) - a33 * w3(2); % input sum, filter 3
end

figure;
plot(0 : length(y3) - 1, y3, "LineWidth", 2);
grid on; grid minor;
axis([-2, length(y3) + 2, 1.1 * min(y3), 1.1 * max(y3)]);
title("Impulse response");
xlabel("Time index"); ylabel("Magnitude");

```



```
plot_freq_resp_from_imp_resp(y3, 1024, 1000);
```

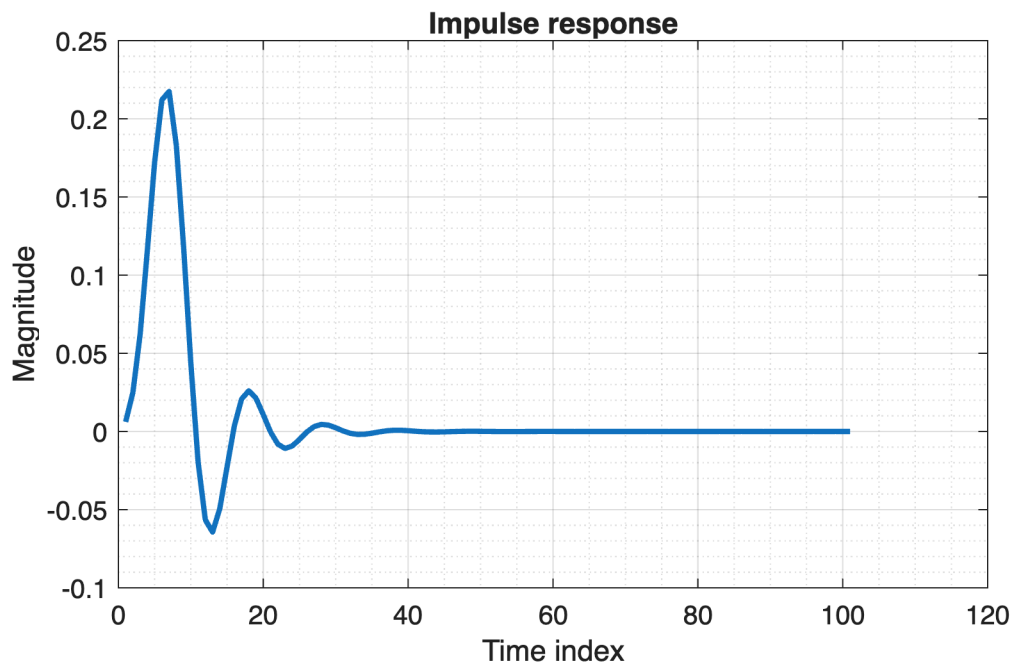


3c

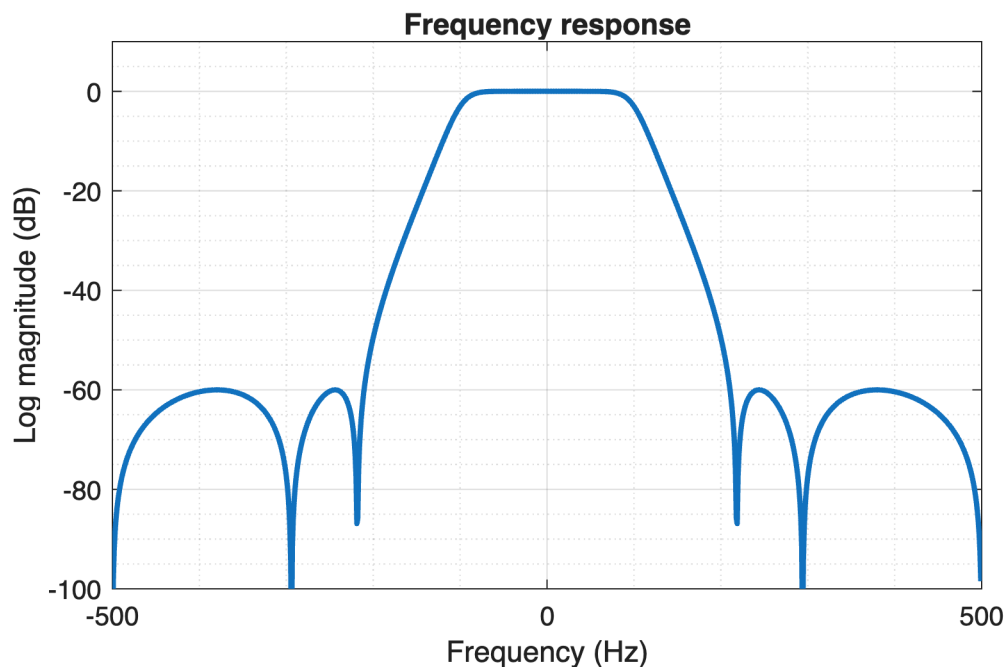
```
b10 = 0.2633;
b20 = 0.1128;
b30 = 0.2069;

h2 = filter(b10 * [1.0 1.0 0.0], [1.0 -0.4743 0.0], [1 zeros(1, 100)]);
h2 = filter(b20 * [1.0 0.5470 1.0], [1.0 -1.0625 0.3498], h2);
h2 = filter(b30 * [1.0 -0.3959 1.0], [1.0 -1.3811 0.7130], h2);

figure;
plot(1:length(h2), h2, "LineWidth", 2);
grid on; grid minor; axis on;
title("Impulse response");
xlabel("Time index"); ylabel("Magnitude");
```



```
plot_freq_resp_from_imp_resp(h2, 1024, 1000);
```



These match with the plots in 3b.

Problem 4

4a

```
% 1, 5, 9, and D for 100 msec
% 8-bit ADC and sampling frequency of 8000 Hz

fs = 8000;           % In Hz
key_press_time = 0.100; % In sec

lo_freqs = [ 697,  770,  852,  941]; % In Hz
hi_freqs = [1209, 1336, 1477, 1633]; % In Hz

% Indices of the key's row and column frequencies
key_1    = [1 1];
%key_2    = [1 2];
%key_3    = [1 3];
key_A    = [1 4];
%key_4    = [2 1];
key_5    = [2 2];
%key_6    = [2 3];
%key_B    = [2 4];
%key_7    = [3 1];
%key_8    = [3 2];
key_9    = [3 3];
key_C    = [3 4];
%key_astsk = [4 1];
%key_0    = [4 2];
%key_hash  = [4 3];
key_D    = [4 4];

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%
% dtmf_time_signal_and_fft
%
% Plot time domain signal for the sum of two sine waves with frequencies
% f_1 and f_2, sampled at fs Hz.
%
% Inputs:
%   fs - Sampling frequency (in Hz)
%   f1 - DTMF row frequency (in Hz)
%   f2 - DTMF column frequency (in Hz)
%   key_press_time - Time duration of the key press (in seconds)
%   key_string - Key character (for title)
%   plot_sig_fft - Boolean parameter to indicate whether time signal and FFT
%                  is to be plotted or not
%
% Outputs:
%   x - Time series
```

```

function [x] = ...
    dtmf_time_signal_and_fft(fs, f1, f2, key_press_time, key_string,
    plot_sig_fft)

    % Number of samples generated with every key press will be fs (in Hz) *
    % key_press_time (in sec).
    n = 0 : (fs * key_press_time) - 1;
    x = cos(2 * pi * f1 .* n / fs) + cos(2 * pi * f2 .* n / fs);

    if (plot_sig_fft)
        figure("Position", [0 0 900 950]);
        subplot(3, 1, 1);
        plot(n / fs, x);
        title(sprintf("Time domain signal for key press %s", key_string));
        grid on; grid minor; axis on;
        ylim([1.1 * min(x), 1.1 * max(x)]);
        xlabel("Time (seconds)"); ylabel("Amplitude");

        subplot(3, 1, 2);
        plot(n(1 : length(n)/4) / fs, x(1 : length(n)/4), "LineWidth", 1.5);
        ylim([1.1 * min(x), 1.1 * max(x)]);
        title(...
            sprintf("Zoomed in time domain signal for key press %s", key_string));
        grid on; grid minor; axis on;
        xlabel("Time (seconds)"); ylabel("Amplitude");

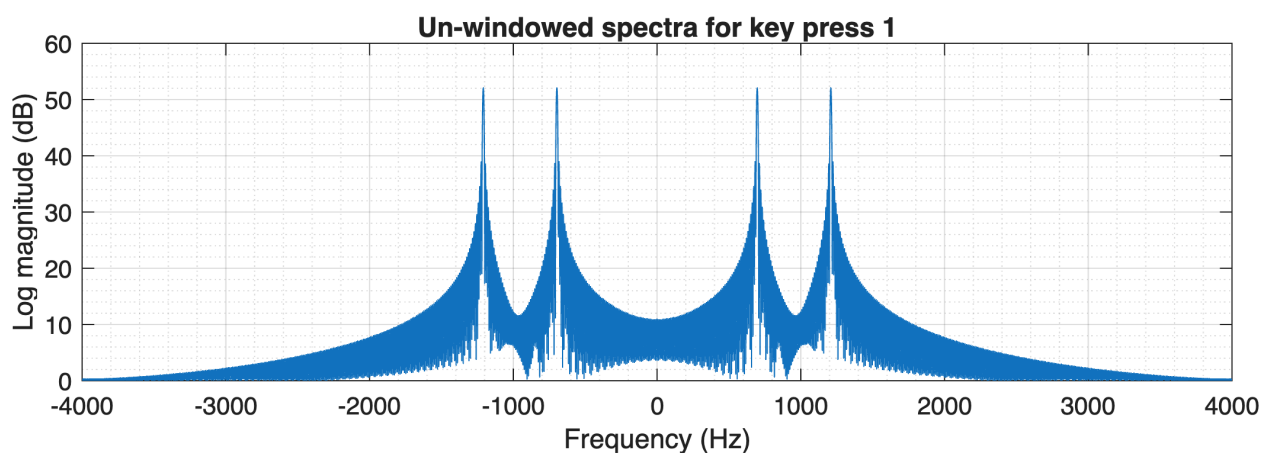
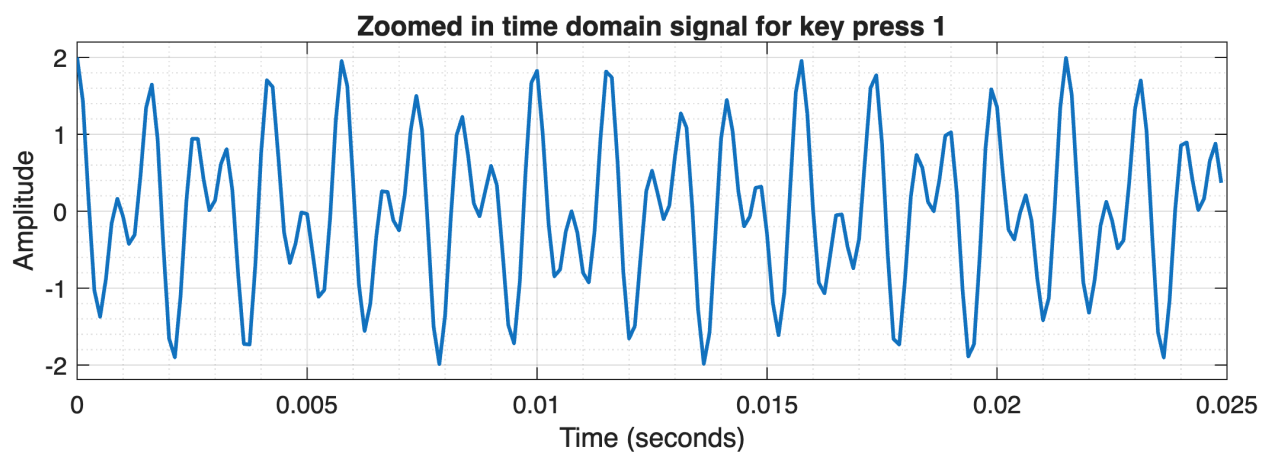
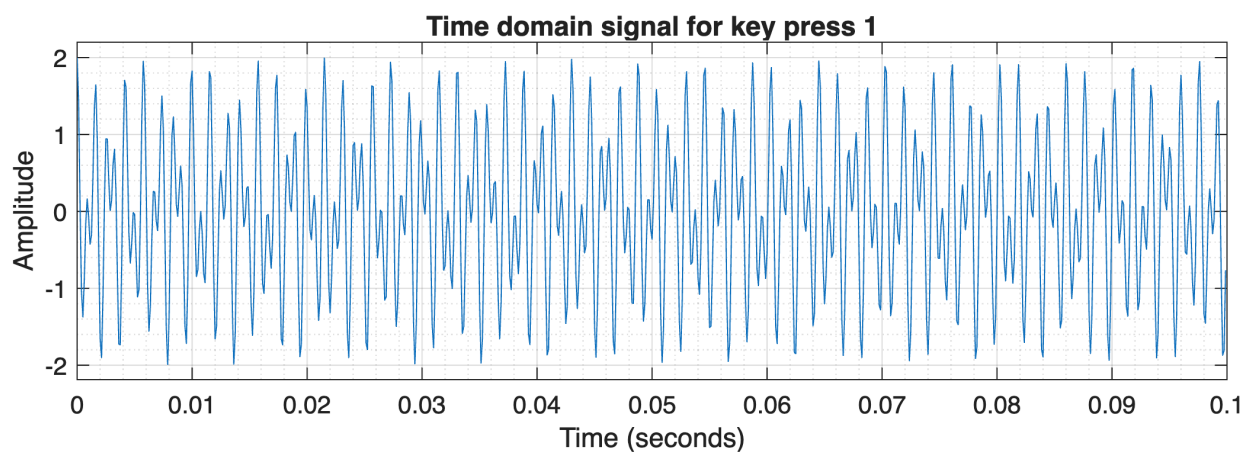
        NFFT = 8192;
        f_x = fftshift(20 * log10(abs(fft(x, NFFT))));

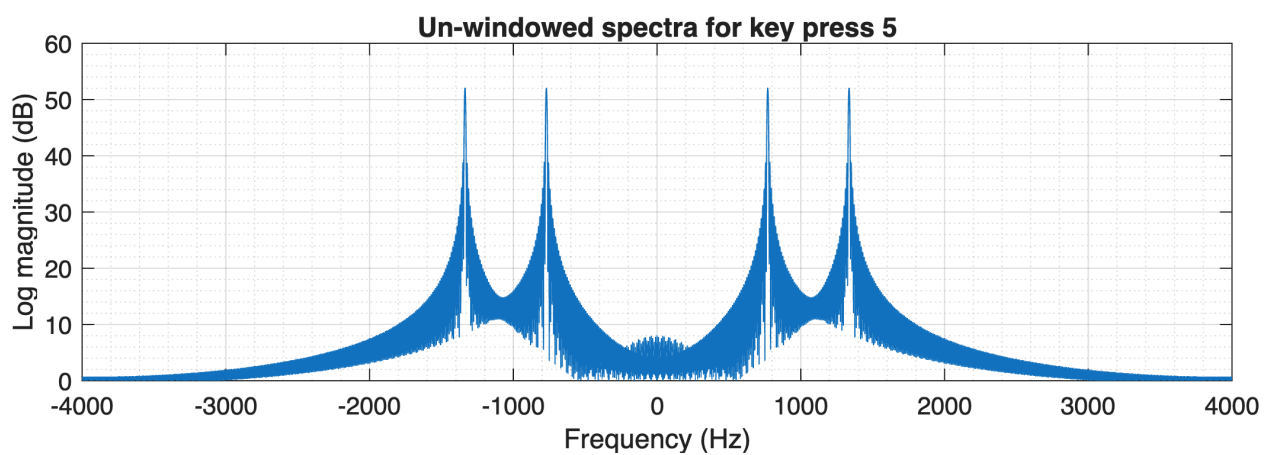
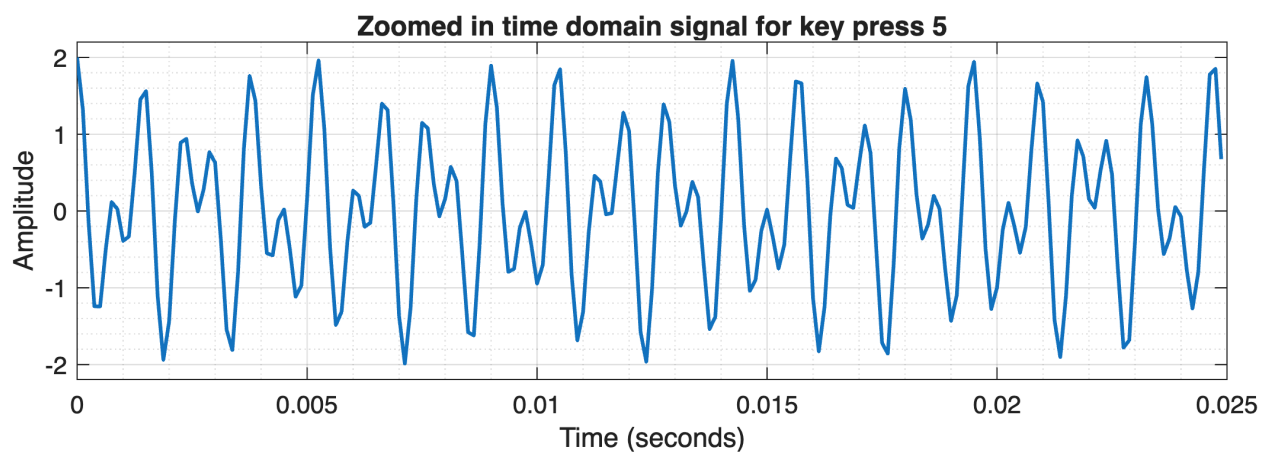
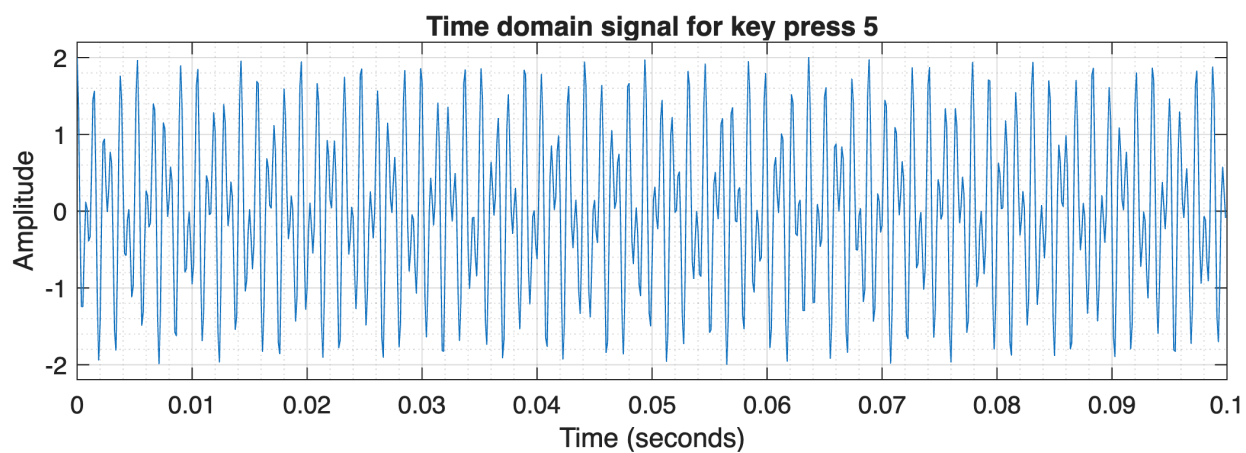
        subplot(3, 1, 3);
        plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, abs(f_x));
        grid on; grid minor;
        title(...
            sprintf("Un-windowed spectra for key press %s", key_string));
        xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");
    end
end

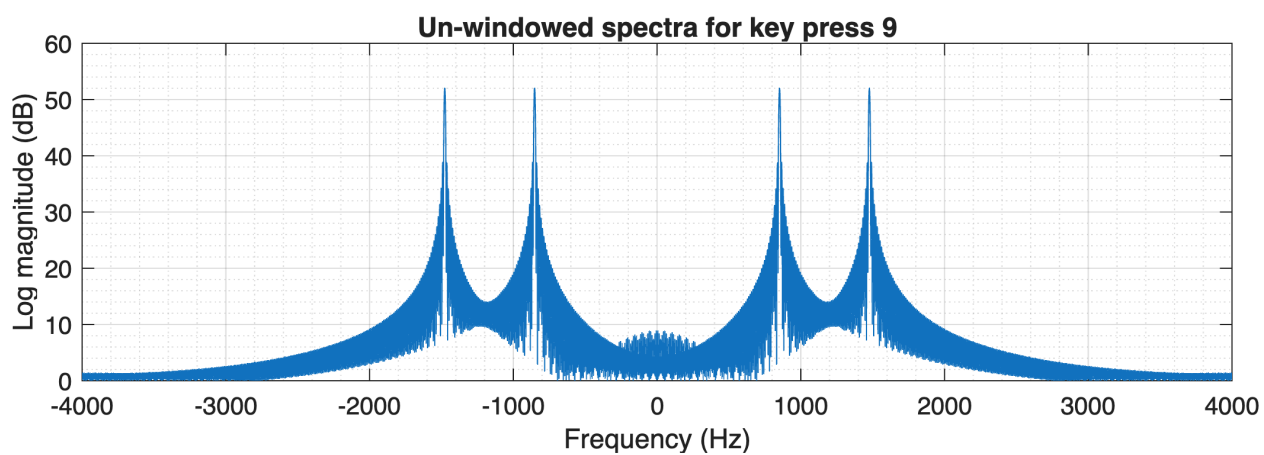
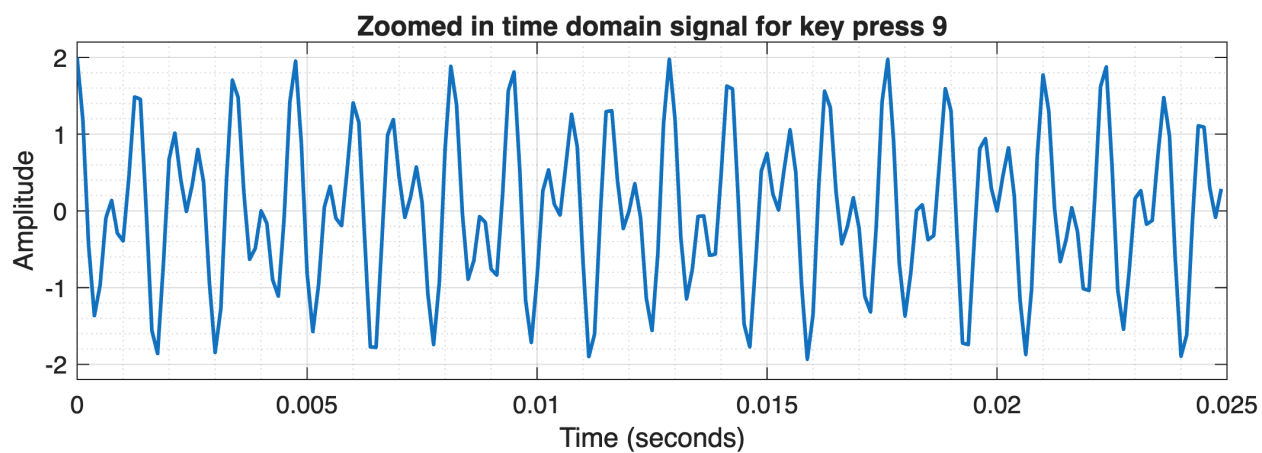
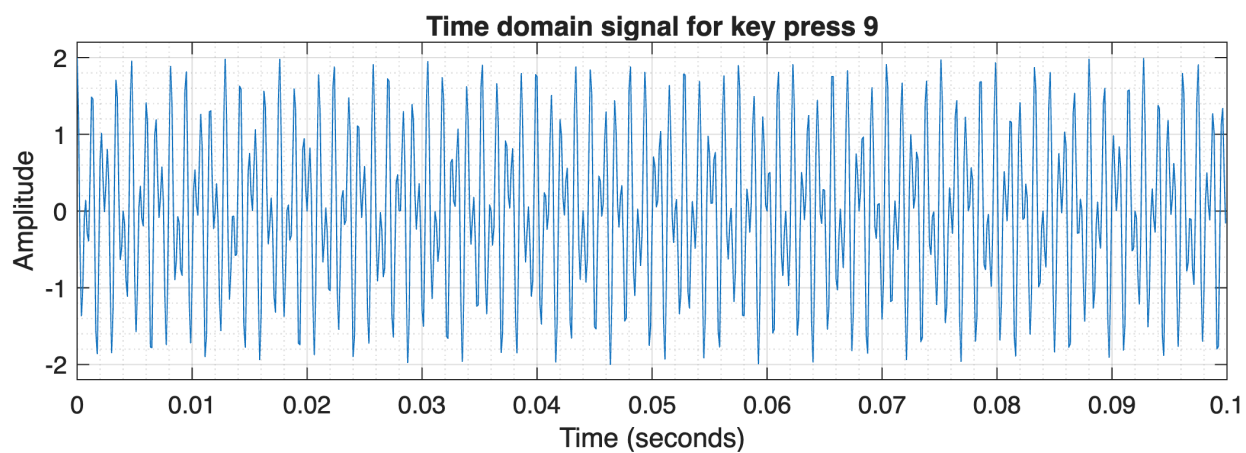
pressed_keys = [key_1; key_5; key_9; key_D];
pressed_key_str = ["1", "5", "9", "D"];

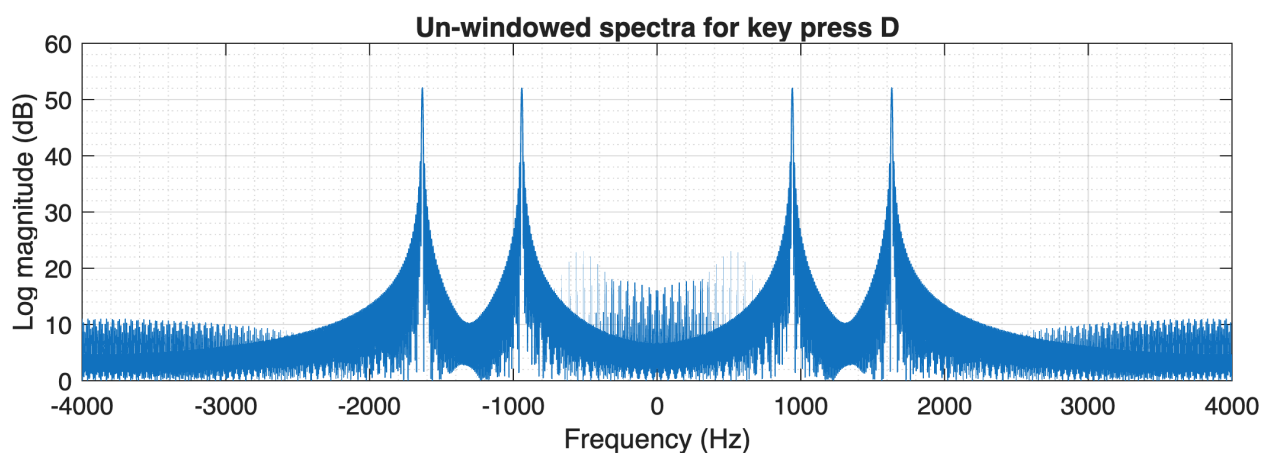
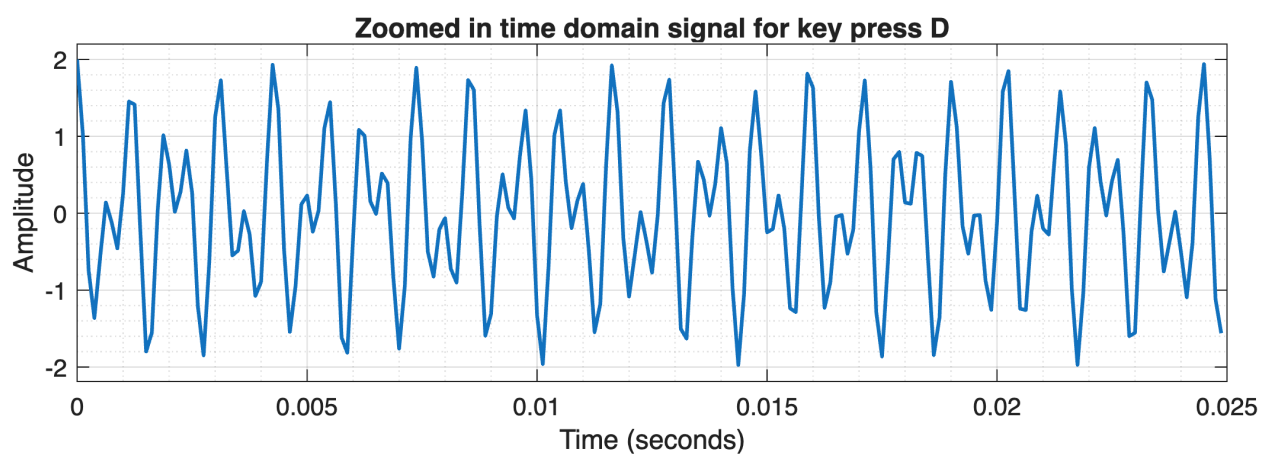
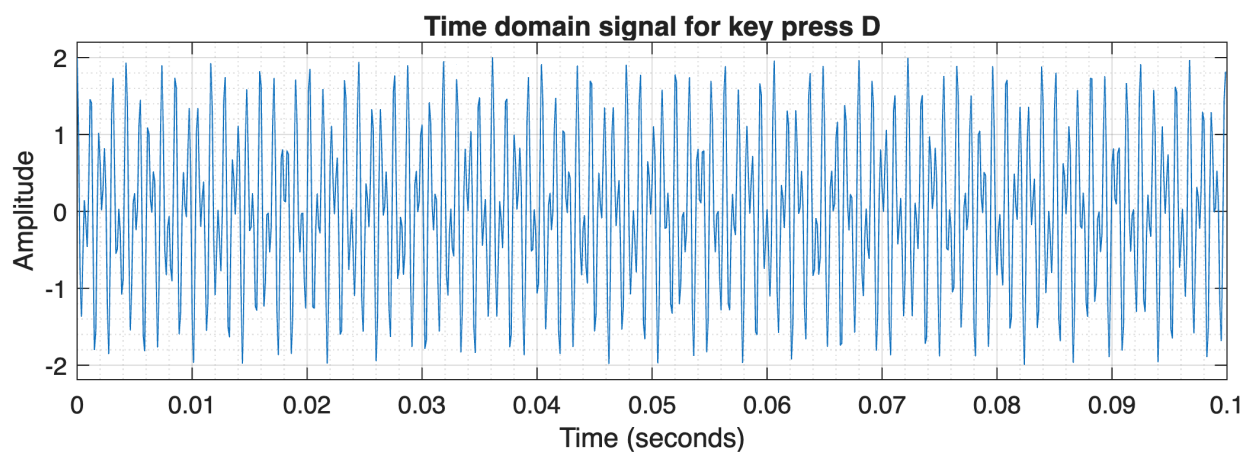
for idx = 1 : length(pressed_keys)
    pressed_key = pressed_keys(idx, 1:2);
    key_lo_freq = lo_freqs(pressed_key(1));
    key_hi_freq = hi_freqs(pressed_key(2));
    dtmf_time_signal_and_fft(...
        fs, key_lo_freq, key_hi_freq, key_press_time, ...
        pressed_key_str(idx), true);
end

```









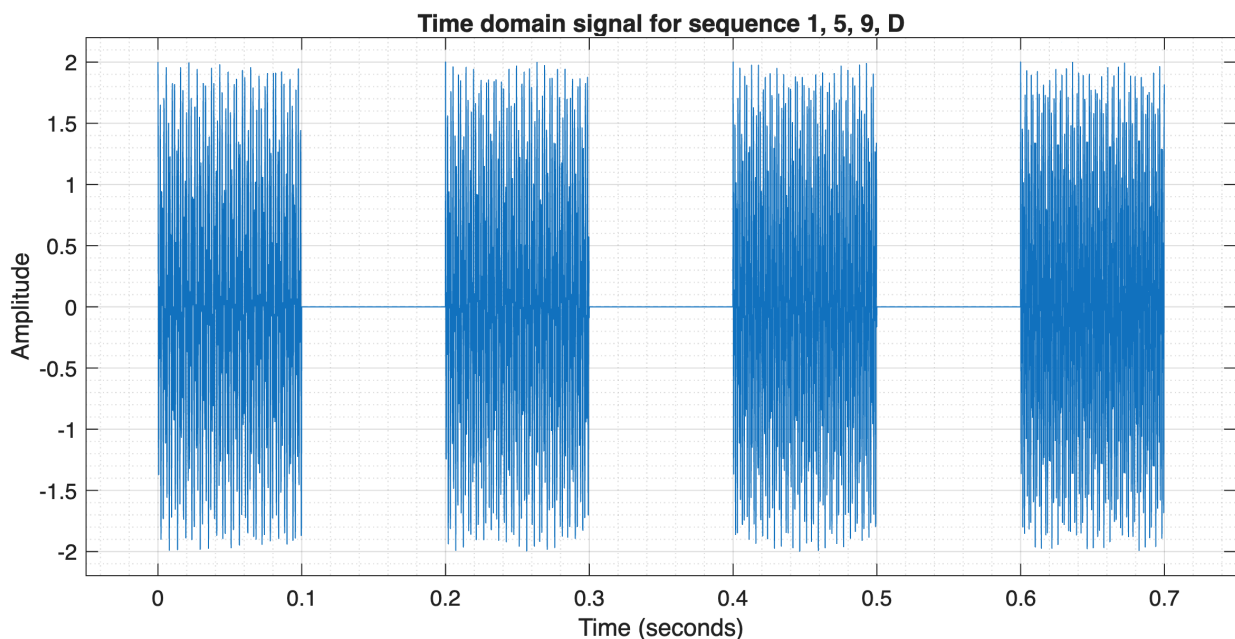
4b

```
% Key press sequence time series
y = zeros(1, (2 * length(pressed_keys) - 1) * fs * key_press_time);

for idx = 1 : length(pressed_keys)
    pressed_key = pressed_keys(idx, 1:2);
    key_lo_freq = lo_freqs(pressed_key(1));
    key_hi_freq = hi_freqs(pressed_key(2));
    x = dtmf_time_signal_and_fft(...
        fs, key_lo_freq, key_hi_freq, key_press_time, ...
        pressed_key_str(idx), false);

    % Time gap will only be in between the key presses
    if (idx ~= length(pressed_keys))
        x = [x, zeros(1, length(x))];
        y(1 + ((idx-1) * length(x)) : length(x) + ((idx-1) * length(x))) = x;
    else
        y(length(y) - length(x) + 1 : length(y)) = x;
    end
end

figure("Position", [0 0 900 400]);
n = (0 : length(y) - 1) / fs;
plot(n, y);
title("Time domain signal for sequence 1, 5, 9, D");
grid on; grid minor;
axis([-0.05, max(n) + 0.05, 1.1 * min(y), 1.1 * max(y)]);
xlabel("Time (seconds)"); ylabel("Amplitude");
```



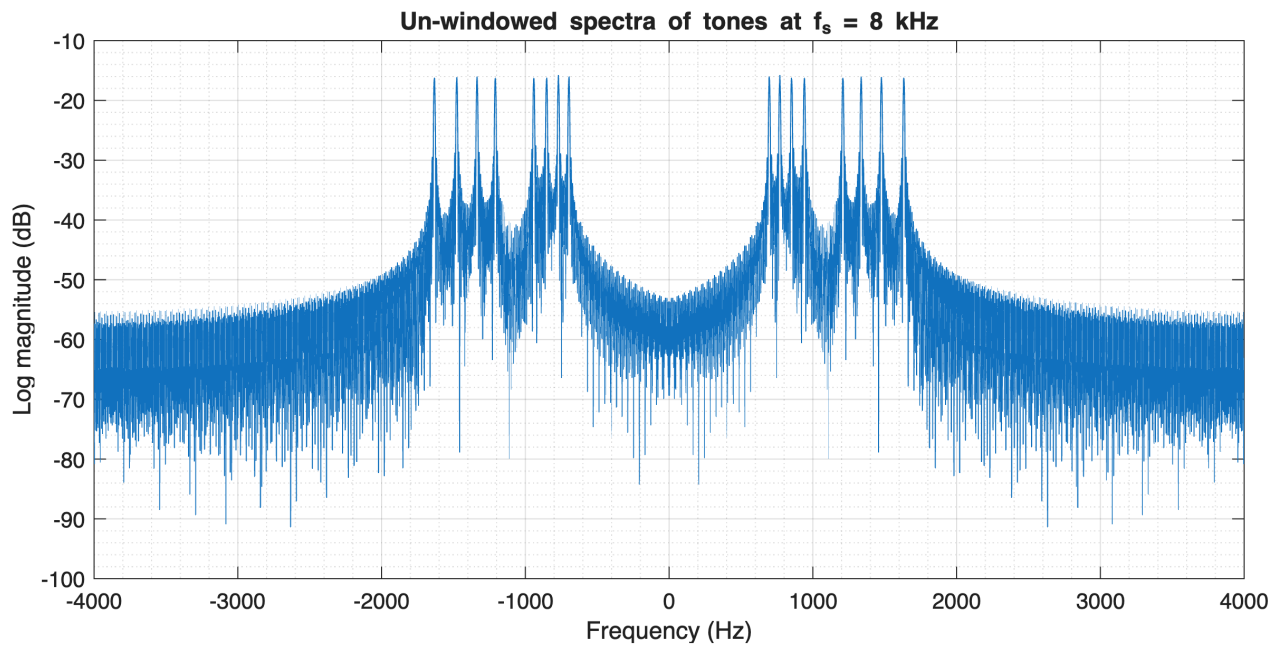
Short segments of key presses are shown in the previous part to illustrate details.

```

% Spectra of tones
NFFT = 8192;
% Scale the time series
scl = 1 / sum(abs(y));
y = y .* scl;
f_y = fftshift(20 * log10(abs(fft(y, NFFT))));

figure("Position", [0 0 900 400]);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs, f_y);
grid on; grid minor; axis on;
title(...)
    sprintf("Un-windowed spectra of tones at f_s = %.5g kHz", fs/1000));
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



4c

In-built Matlab routine `cheb2ord` was used to determine the required order of the Chebyshev II filter.

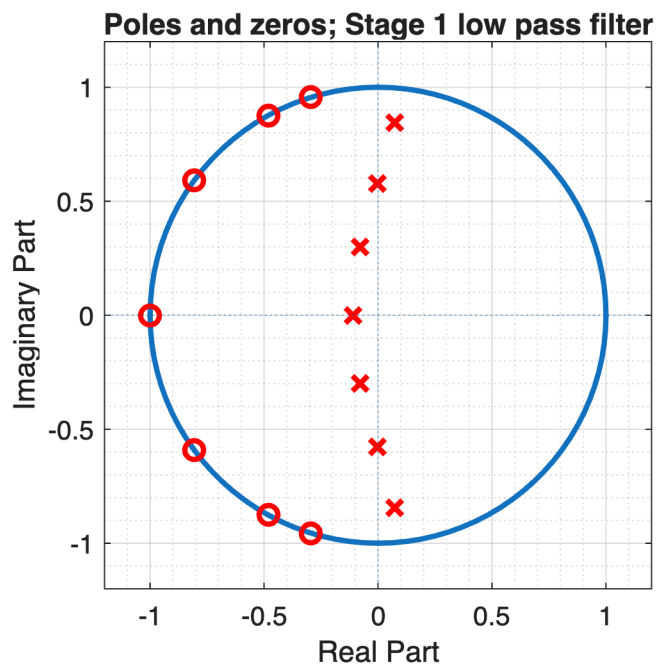
```
% Stage 1 (low pass filter)
fs1 = 8000;
Wp1 = 1650 / (fs1 / 2); % Passband edge (normalized to Nyquist frequency)
Ws1 = 2350 / (fs1 / 2); % Stopband edge (normalized to Nyquist frequency)
Rp = 0.2; % Passband ripple (in dB)
A_dB = 50; % Stopband attenuation (in dB)

% Get the order for first cheby2 filter
n1 = cheb2ord(Wp1, Ws1, Rp, A_dB);

% Below code is leveraged from Prof. harris' IIR_cheby2_x.m
[B1, A1] = cheby2(n1, A_dB, Ws1);

scl = B1(1);
b = B1 / scl;
a = A1;
h = scl * filter(b, a, [1 zeros(1, 200)]);

% Show the pole-zero diagrams of the filter
plot_pole_zero(b, a, "Stage 1 low pass filter");
```

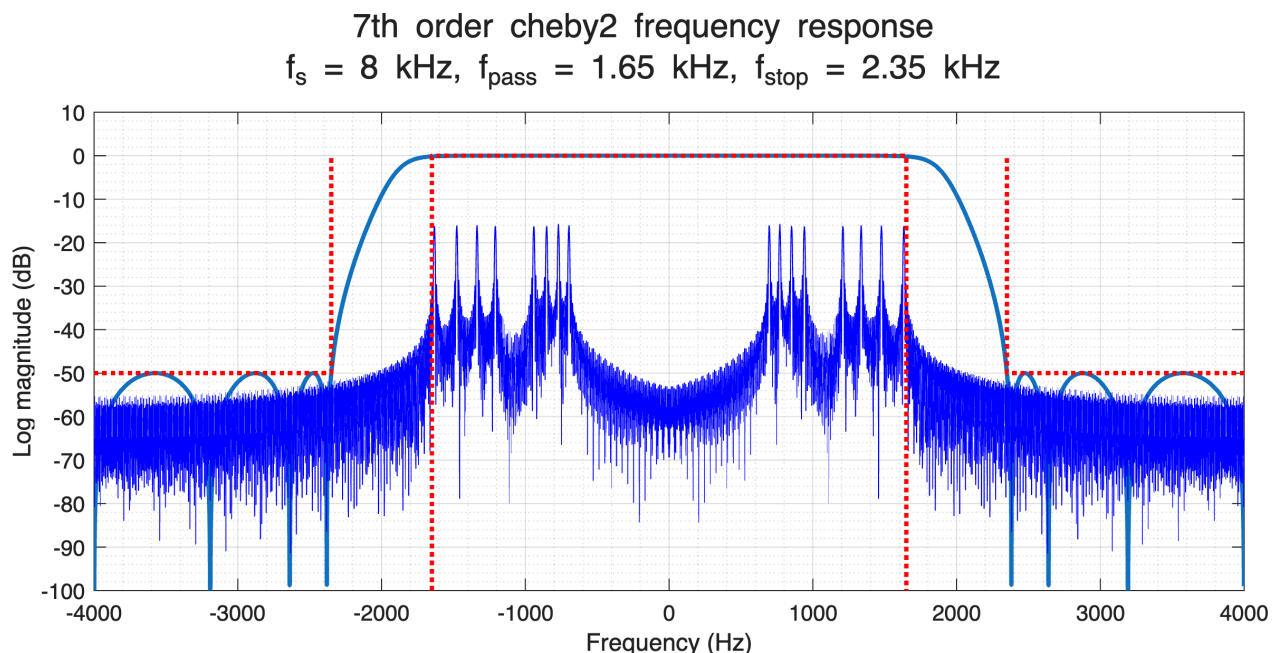


```

% Show the frequency response of the filter along with the DTMF spectral
% lines at the input rate.
f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs1;
f_h = fftshift(20 * log10(abs(fft(h, NFFT))));

f1 = Wp1 * fs1/2;
f2 = Ws1 * fs1/2;
figure("Position", [0 0 900 400]);
sgtitle(...
    {...
        sprintf(...
            "%dth order cheby2 frequency response", n1), ...
        sprintf(...
            "f_s = %.5g kHz, f_{pass} = %.5g kHz, f_{stop} = %.5g kHz", ...
            fs1 / 1000, f1 / 1000, f2 / 1000)});
subplot(1, 1, 1);
plot(f, f_h, "LineWidth", 2);
hold on;
plot(f, f_y, "Color", "b"); % Tones
% Spectral mask for low pass filter
plot([-fs1/2 -f2 -f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([fs1/2 f2 f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([-f1 -f1 f1 f1], [-100 0 0 -100], ":r", "LineWidth", 2);
hold off;
grid on; grid minor;
axis([-fs1/2 fs1/2 -100 10]);
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



```

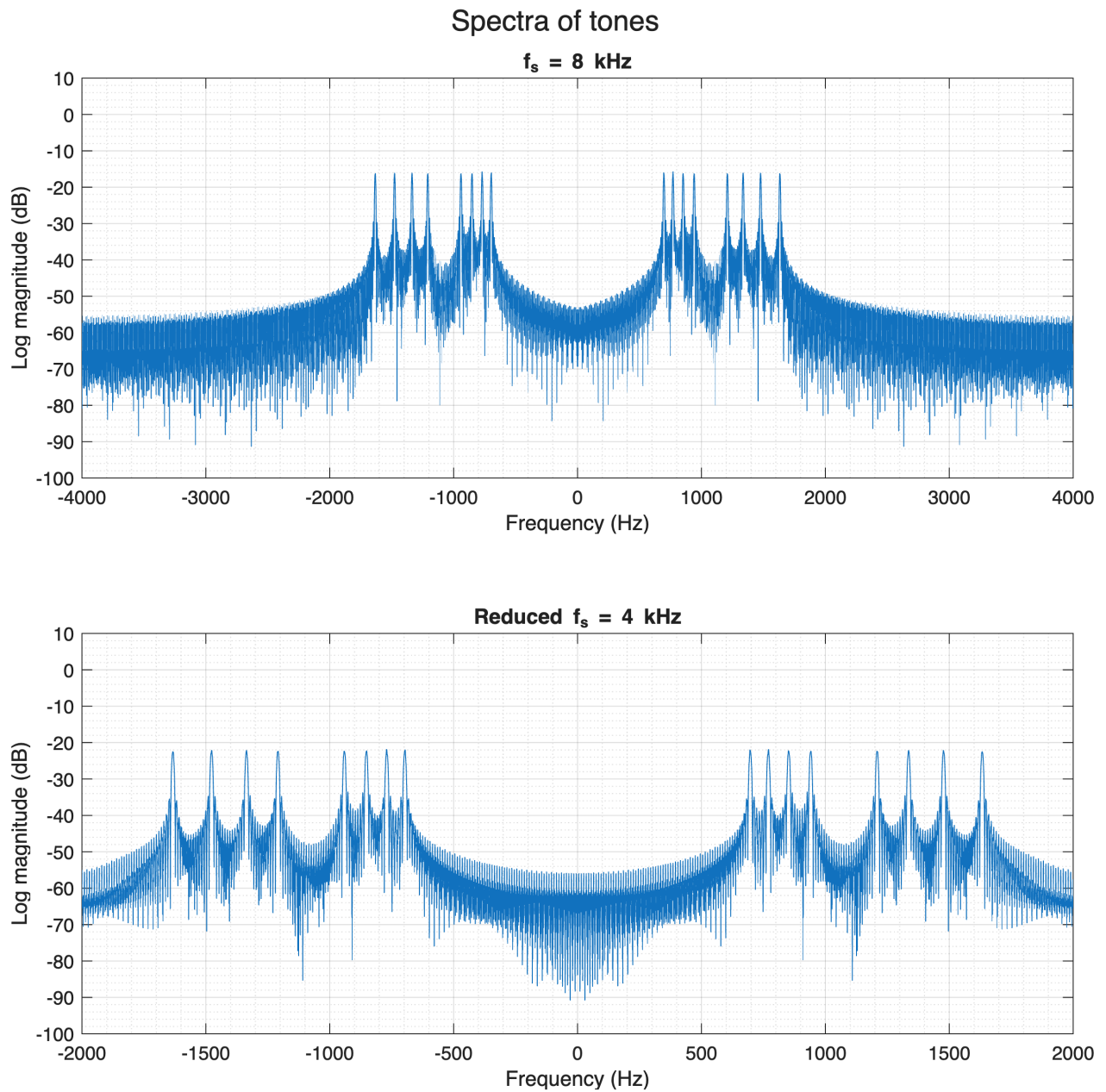
% Pass the set of tones through the low pass filter and then reduce sample
rate
% 2-to-1. Show the spectra of the tones at the input sample rate and at the
% output sample rate.

y = scl * filter(b, a, y);
y_ds = y(1 : 2 : length(y));

figure("Position", [0 0 900 850]);
sgtitle("Spectra of tones");
subplot(2, 1, 1);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs1, f_y);
grid on; grid minor; axis on;
ylim([-100 10]);
title(sprintf("f_s = %.5g kHz", fs1/1000));
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

f_y_ds = fftshift(20 * log10(abs(fft(y_ds, NFFT))));
subplot(2, 1, 2);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * (fs1/2), f_y_ds);
grid on; grid minor; axis on;
ylim([-100 10]);
title(sprintf("Reduced f_s = %.5g kHz", fs1/2000));
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



From inspection of above spectral lines, the center frequencies of the DTMF tones at the reduced sample rate are:

696.2 Hz, 770 Hz, 852 Hz, 941.4 Hz, 1209.4 Hz, 1335.4 Hz, 1476.5 Hz, 1632.3 Hz

Actual DTMF frequencies for key presses "1", "5", "9", "D" are:

697 Hz, 770 Hz, 852 Hz, 941 Hz, 1209 Hz, 1336 Hz, 1477 Hz, 1633 Hz.

Thus, the center frequencies in the spectra at the output rate are very close to the defined tone frequencies.

```

% Stage 2 (low and high pass filters)

% Below code is leveraged from Prof. harris' Project_2025_part_4D.m

% Project_2025_part_4D
% 4d) Design the set of IIR low pass and high pass cheby2 filters to
separate
%     the tones in the low band and high band.
%
% Filter requirements:      Low Pass          High Pass
% Passband ripple          0.2 dB             0.2 dB
% Stop band attenuation    50 dB              50 dB
% Passband                  0-950 Hz          1200-2000 Hz
% Stopband                  1200-2000 Hz      0-950 Hz
%
% Pass the 2-to-1 down sampled time series from the first low pass filter
% through the low pass and high pass filters. Show their spectra at the
% output prior to the 2-to-1 down sample and then after the 2-to-1 down
sample.

% low freq 0.697, 0.770, 0.852, 0.941 kHz
% hi freq  1.209, 1.336, 1.477. 1.633 kHz

n = 0 : (fs1 * key_press_time) - 1;

% Form the tones; add 6 dB (2x) amplitude gain to the last tone to
% differentiate it from the rest of the tones in case aliasing occurs!
s_lo = ...
    cos(2 * pi * n * 0.697 / 4.0) + ...
    cos(2 * pi * n * 0.770 / 4.0) + ...
    cos(2 * pi * n * 0.852 / 4.0) + ...
    2 * cos(2 * pi * n * 0.941 / 4.0);

s_hi = ...
    cos(2 * pi * n * 1.209 / 4.0) + ...
    cos(2 * pi * n * 1.336 / 4.0) + ...
    cos(2 * pi * n * 1.477 / 4.0) + ...
    2 * cos(2 * pi * n * 1.633 / 4.0);

ww = kaiser(length(n), 8)';
ww = ww / sum(ww);

f_s_lo = fftshift(20 * log10(abs(fft(s_lo .* ww, NFFT))));
f_s_hi = fftshift(20 * log10(abs(fft(s_hi .* ww, NFFT))));

% Update the sample rate to 2-to-1 downsample rate after the first filter
fs2 = fs1 / 2; % Sampling frequency is 4000 Hz now

```

```

% Update the filter specifications for low and high pass filters
Wp2_lo = 950 / (fs2 / 2); % Passband edge (normalized to Nyquist frequency)
Ws2_lo = 1200 / (fs2 / 2); % Stopband edge (normalized to Nyquist frequency)

Wp2_hi = 1200 / (fs2 / 2); % Passband edge (normalized to Nyquist frequency)
Ws2_hi = 950 / (fs2 / 2); % Stopband edge (normalized to Nyquist frequency)

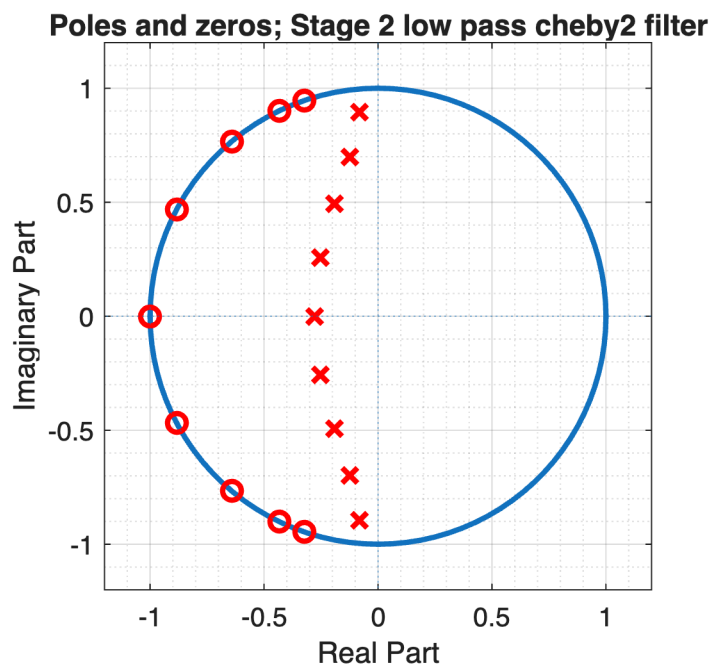
% NOTE: Prof uses order 7, but that doesn't meet the ripple spec; the
frequency
% response goes down to -2 dB with 7th order; cheb2ord gives 9
% Asked the prof in class: He chose to ignore the ripple at the very edges
of
% the passband.
[B2_lo, A2_lo] = cheby2(7, A_dB, Ws2_lo);
[B2_hi, A2_hi] = cheby2(7, A_dB, Ws2_hi, "high");

n2_lo = cheb2ord(Wp2_lo, Ws2_lo, Rp, A_dB);
[B2_lo, A2_lo] = cheby2(n2_lo, A_dB, Ws2_lo);

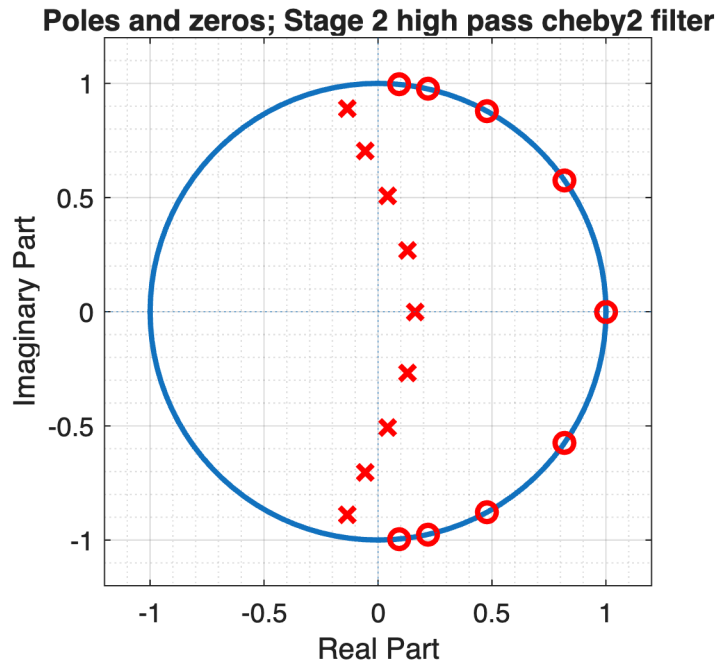
n2_hi = cheb2ord(Wp2_hi, Ws2_hi, Rp, A_dB);
[B2_hi, A2_hi] = cheby2(n2_hi, A_dB, Ws2_hi, "high");

% Show the pole-zero diagrams of both the filters
plot_pole_zero(B2_lo, A2_lo, "Stage 2 low pass cheby2 filter");

```



```
plot_pole_zero(B2_hi, A2_hi, "Stage 2 high pass cheby2 filter");
```



```
h_lo = filter(B2_lo, A2_lo, [1, zeros(1, 200)]);
h_hi = filter(B2_hi, A2_hi, [1, zeros(1, 200)]);

f_h_lo = fftshift(20 * log10(abs(fft(h_lo, NFFT))));
f_h_hi = fftshift(20 * log10(abs(fft(h_hi, NFFT))));

f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs2;
f1 = Wp2_lo * fs2 / 2;
f2 = Ws2_lo * fs2 / 2;

figure("Position", [0 0 900 950]);
subplot(2, 1, 1);
plot(f, f_h_lo, "LineWidth", 2);
hold on;
% Spectral mask for low pass filter
plot([-fs2/2 -f2 -f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([fs2/2 f2 f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([-f1 -f1 f1 f1], [-100 0 0 -100], ":r", "LineWidth", 2);
% Tones
plot(f, f_s_lo, "b", "LineWidth", 1.5);
plot(f, f_s_hi, ":b", "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([-fs2/2 fs2/2 -80 10]);
title(...
    {sprintf(...
        "%dth order low pass cheby2 filter spectrum; 100ms input tone
spectra", ...
        n2_lo), ...
```

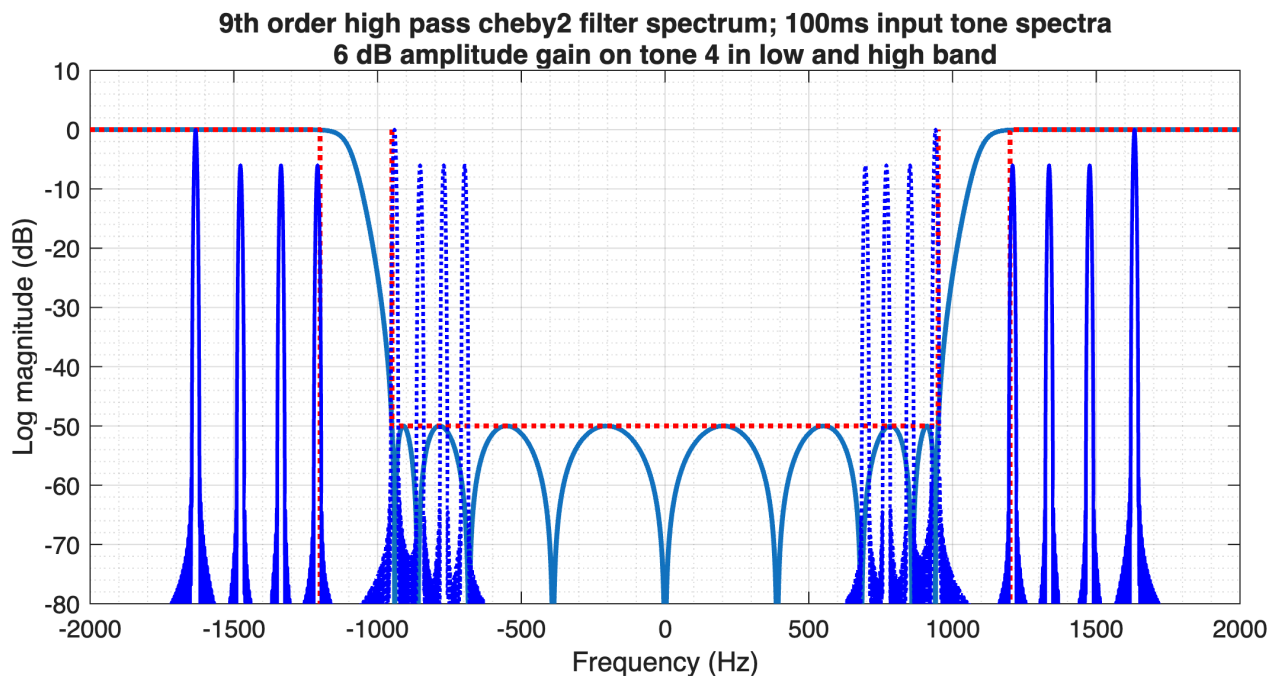
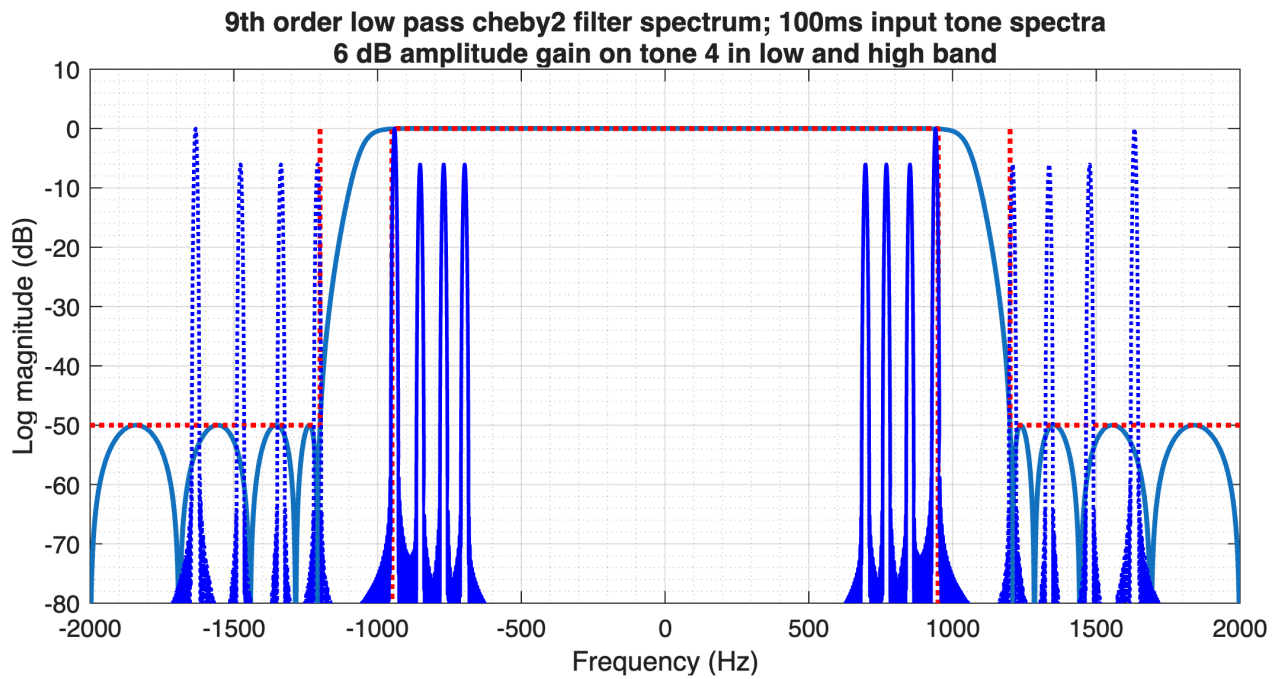
```

    "6 dB amplitude gain on tone 4 in low and high band"});
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

f2 = Wp2_hi * fs2 / 2;
f1 = Ws2_hi * fs2 / 2;
subplot(2, 1, 2);
plot(f, f_h_hi, "LineWidth", 2);
hold on;
% Spectral mask for high pass filter
plot([-f1 -f1 f1 f1], [0 -A_dB -A_dB 0], "r", "LineWidth", 2);
plot([-fs2/2 -f2 -f2], [0 0 -100], "r", "LineWidth", 2);
plot([f2 f2 fs2/2], [-100 0 0], "r", "LineWidth", 2);

% Tones
plot(f, f_s_lo, "b", "LineWidth", 1.5);
plot(f, f_s_hi, "b", "LineWidth", 1.5);
hold off;
grid on; grid minor;
axis([-fs2/2 fs2/2 -80 10]);
title(...
    {sprintf(...
        "%dth order high pass cheby2 filter spectrum; 100ms input tone
spectra", ...
        n2_hi), ...
    "6 dB amplitude gain on tone 4 in low and high band"});
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



```
% Pass tones through low pass and high pass filters and downsample 2-to-1
% TODO: Why the additional 100 zeros? The 100ms gap?
ss_lo = filter(B2_lo, A2_lo, [s_lo + s_hi zeros(1, 100)]);
ss_hi = filter(B2_hi, A2_hi, [s_lo + s_hi zeros(1, 100)]);
ww = kaiser(length(n), 8)';
ww = ww / sum(ww);
```

```

fss_lo = fftshift(20 * log10(abs(fft(ss_lo(1 : length(n)) .* ww, NFFT))));
fss_hi = fftshift(20 * log10(abs(fft(ss_hi(1 : length(n)) .* ww, NFFT))));

% Downsample 2-to-1
ss_lo_ds = ss_lo(1 : 2 : length(ss_lo));
ss_hi_ds = ss_hi(1 : 2 : length(ss_hi));
ww = kaiser(length(n)/2, 8)';
ww = ww / sum(ww);

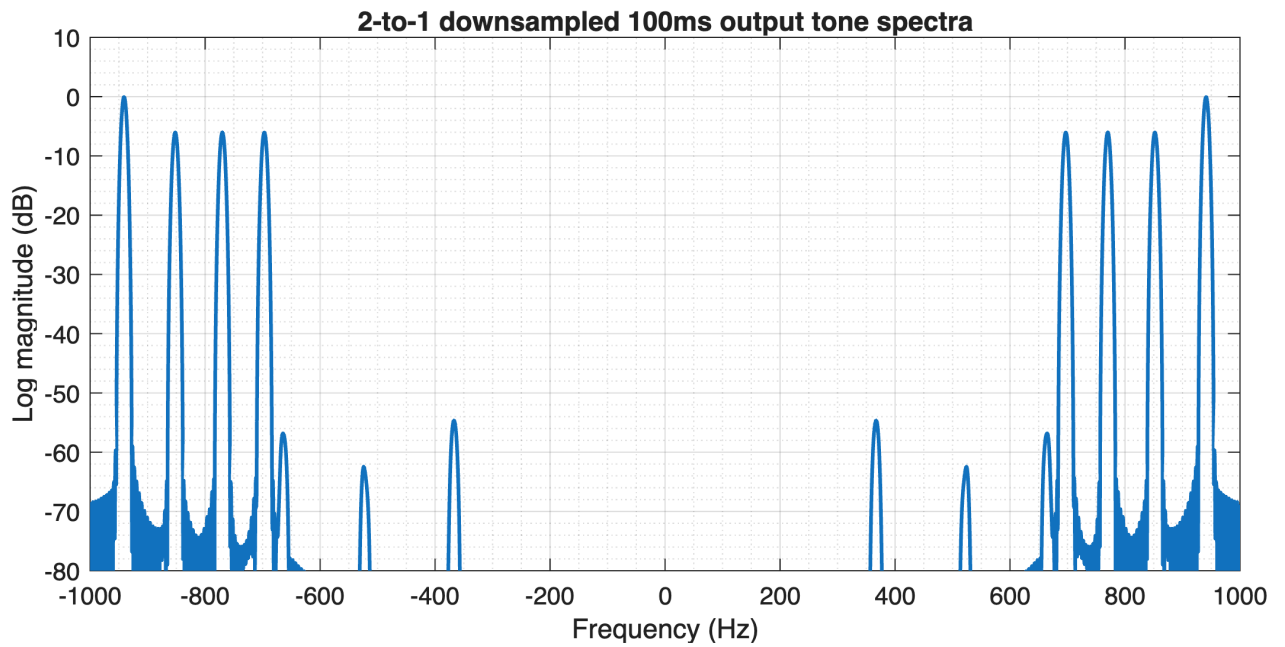
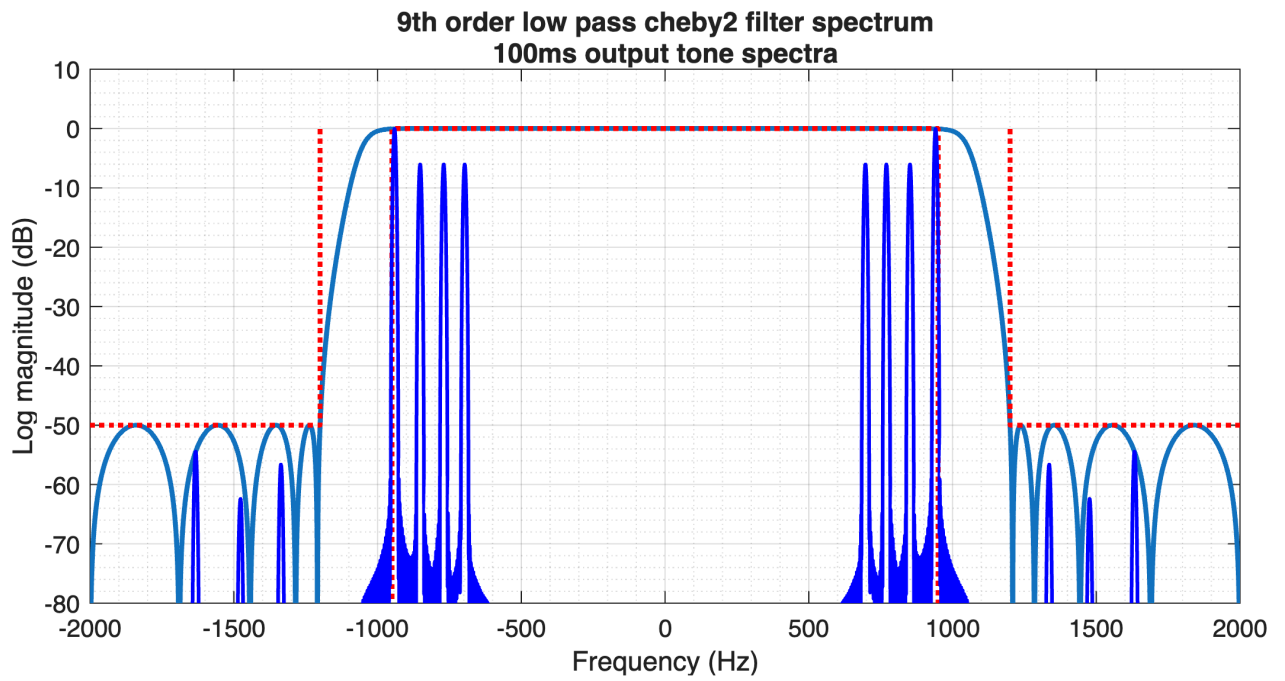
figure("Position", [0 0 900 950]);
subplot(2, 1, 1);
plot(f, f_h_lo, "LineWidth", 2);
hold on;
% Spectral mask for low pass filter
plot([-fs2/2 -f2 -f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([fs2/2 f2 f2], [-A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([-f1 -f1 f1 f1], [-100 0 0 -100], ":r", "LineWidth", 2);
% Tones
plot(f, fss_lo, "b", "LineWidth", 1.5);
% TODO: What are these?
%plot([-fs2/4 -fs2/4], [-80 5], "--k", "LineWidth", 2);
%plot([+fs2/4 +fs2/4], [-80 5], "--k", "LineWidth", 2);
hold off;
grid on; grid minor;
axis([-fs2/2 fs2/2 -80 10]);
title(...
    {sprintf("%dth order low pass cheby2 filter spectrum", n2_lo), ...
    "100ms output tone spectra"});
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

NFFT = NFFT/2;
fss_ds_lo = ...
    fftshift(20 * log10(abs(fft(ss_lo_ds(1 : length(n)/2) .* ww, NFFT))));
fss_ds_hi = ...
    fftshift(20 * log10(abs(fft(ss_hi_ds(1 : length(n)/2) .* ww, NFFT))));

f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs2/2;

subplot(2, 1, 2);
% Tones
plot(f, fss_ds_lo, "LineWidth", 1.5);
grid on; grid minor;
axis([-fs2/4 fs2/4 -80 10]);
title("2-to-1 downsampled 100ms output tone spectra");
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



```

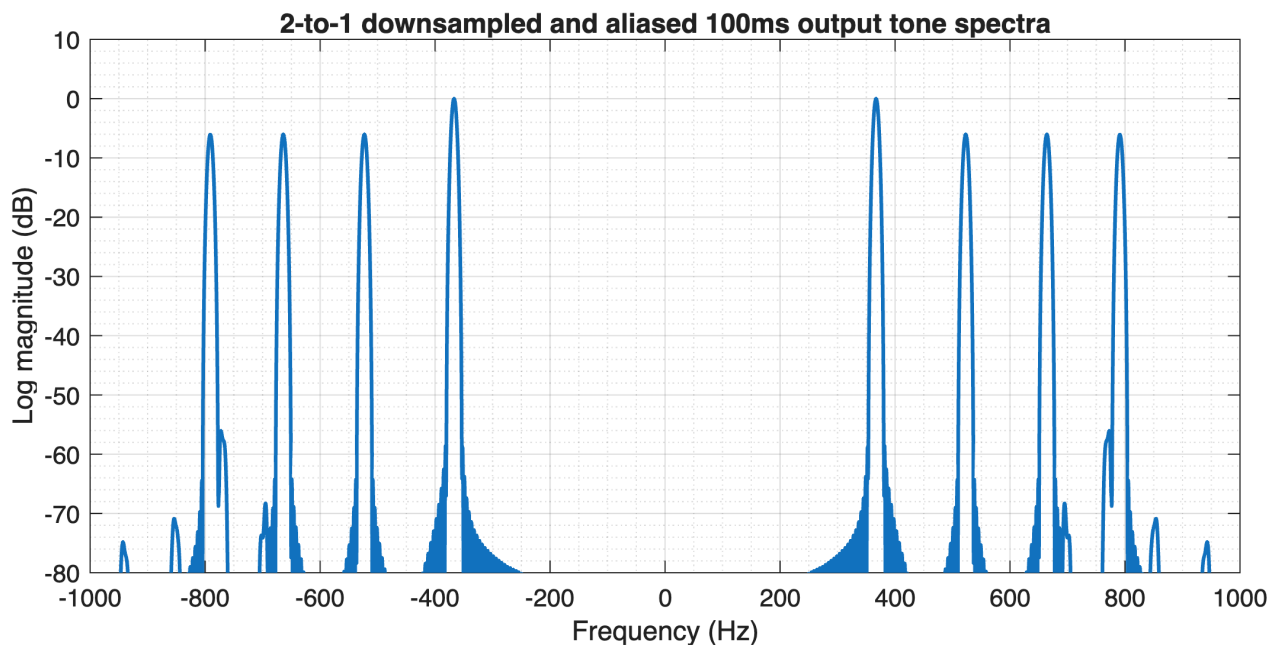
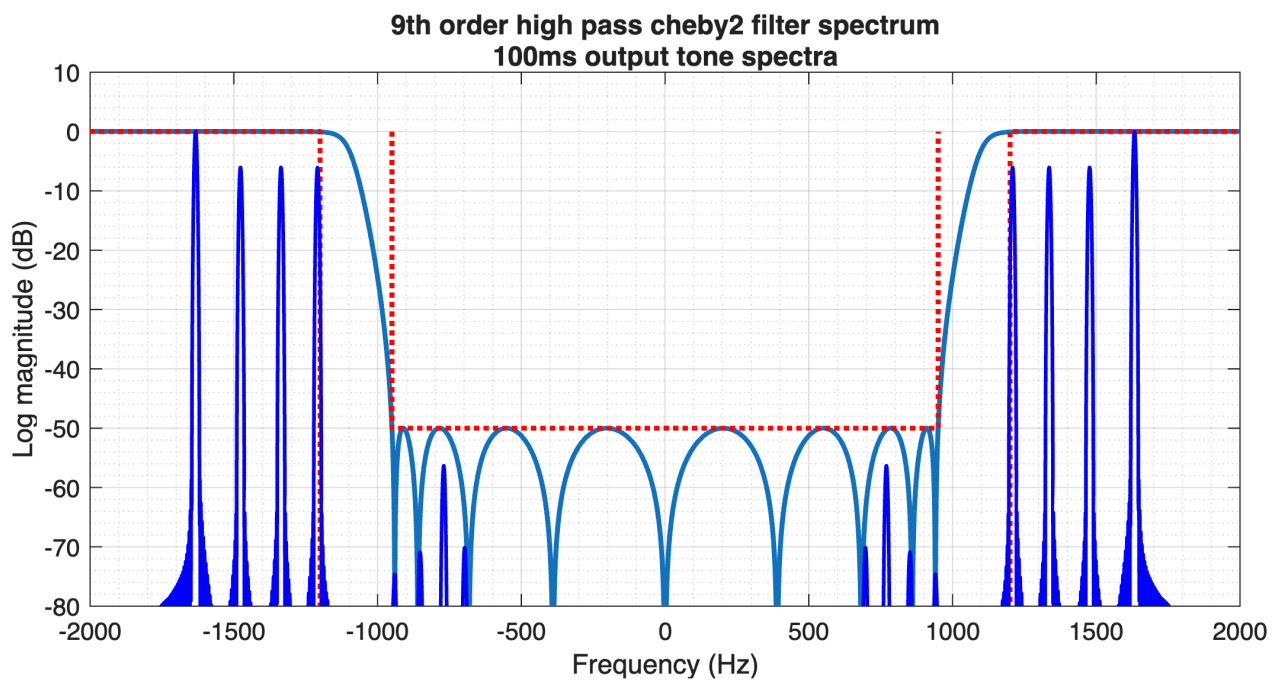
figure("Position", [0 0 900 950]);
subplot(2, 1, 1);
NFFT = length(f_h_lo);
f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs2;
plot(f, f_h_hi, "LineWidth", 2);
hold on;
% Spectral mask for high pass filter
plot([-f1 -f1 f1 f1], [0 -A_dB -A_dB 0], ":r", "LineWidth", 2);
plot([-fs2/2 -f2 -f2], [0 0 -100], ":r", "LineWidth", 2);
plot([f2 f2 fs2/2], [-100 0 0], ":r", "LineWidth", 2);

% Tones
plot(f, fss_hi, "b", "LineWidth", 1.5);
% TODO: What are these?
%plot([-fs2/4 -fs2/4], [-80 5], "--k", "LineWidth", 2);
%plot([+fs2/4 +fs2/4], [-80 5], "--k", "LineWidth", 2);
hold off;
grid on; grid minor;
axis([-fs2/2 fs2/2 -80 10]);
title(...
    {sprintf("%dth order high pass cheby2 filter spectrum", n2_hi), ...
    "100ms output tone spectra"});
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

NFFT = NFFT/2;
f = (-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs2/2;

subplot(2, 1, 2);
% Tones
plot(f, fss_ds_hi, "LineWidth", 1.5);
grid on; grid minor;
axis([-fs2/4 fs2/4 -80 10]);
title("2-to-1 downsampled and aliased 100ms output tone spectra");
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");

```



```

% Stage 3 filters (lo-f1/2/3 and hi-f1/2/3/4 bandpass filters)
% NOTE: lo-f4 (941 Hz center frequency will be high pass due to the
%       proximity to Nyquist frequency of 1 kHz.

% Update the sample rate to 2-to-1 downsample rate after the low and high
pass
% filters
fs3 = fs2 / 2; % Sampling frequency is 2000 Hz now

% Get the aliased frequencies from the middle stage high pass filter
hi_aliased_freqs = sort(fs3 - hi_freqs); % In Hz

% Half bandwidth around the center (tone) frequency
% For example, the lo-f1 filter for 697 Hz tone would be centered at 697 Hz
and
% passband would be from 687 Hz to 707 Hz with FPASS_HALFBW of 10 Hz
FPASS_HALFBW = 10; % In Hz

% Transition band width (measured from the corresponding passband edge)
% Continuing the example in the comment above.
% With FSTOP_WIDTH of 20 Hz, the lower transition band would be from 667 to
% 687 Hz and higher transition band would be from 707 to 727 Hz.
FSTOP_WIDTH = 20; % In Hz

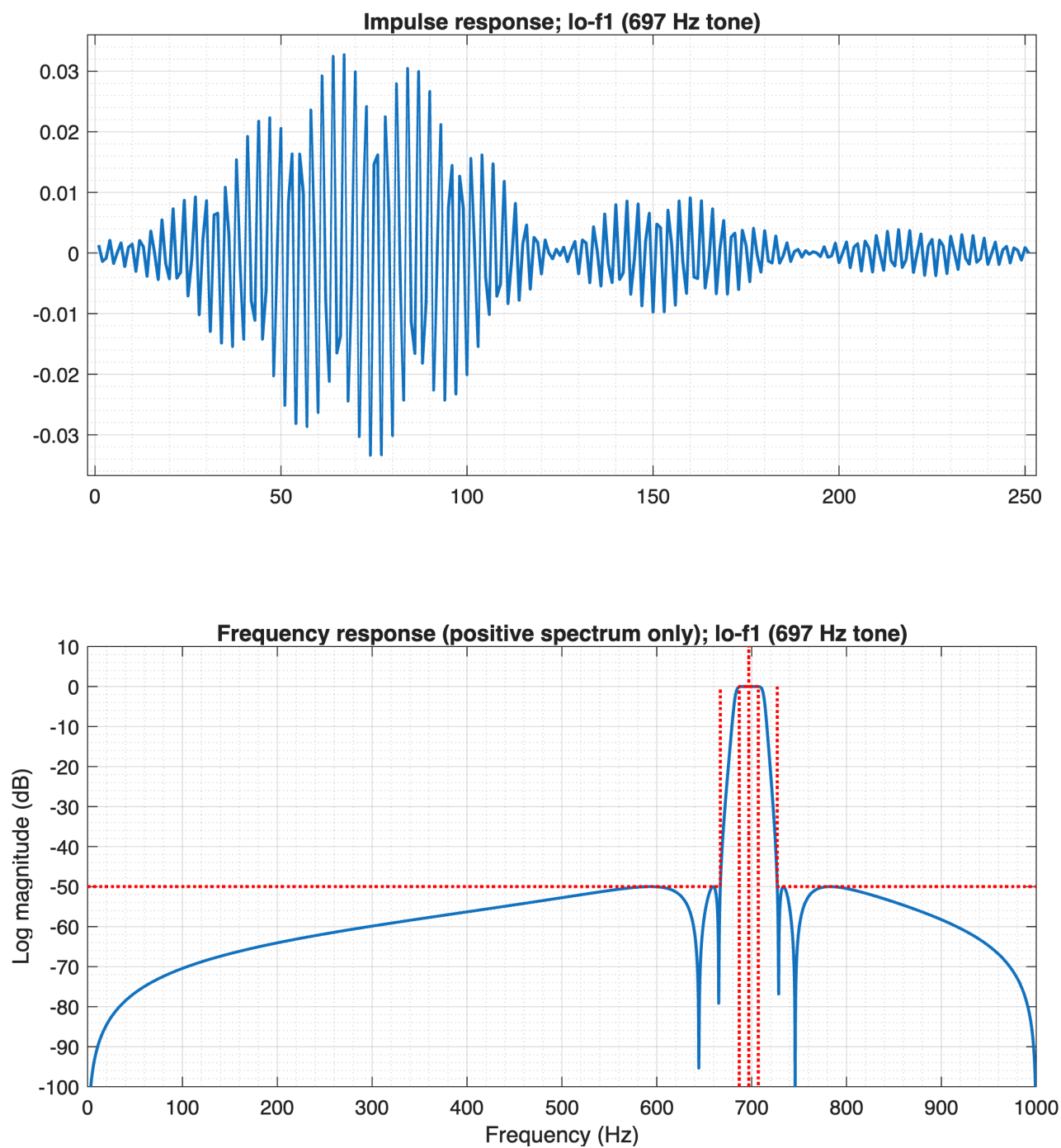
for i = 1 : (length(lo_freqs) - 1)
    % Passband edge frequencies (normalized to Nyquist frequency)
    Wp = [lo_freqs(i) - FPASS_HALFBW, lo_freqs(i) + FPASS_HALFBW] / (fs3/2);

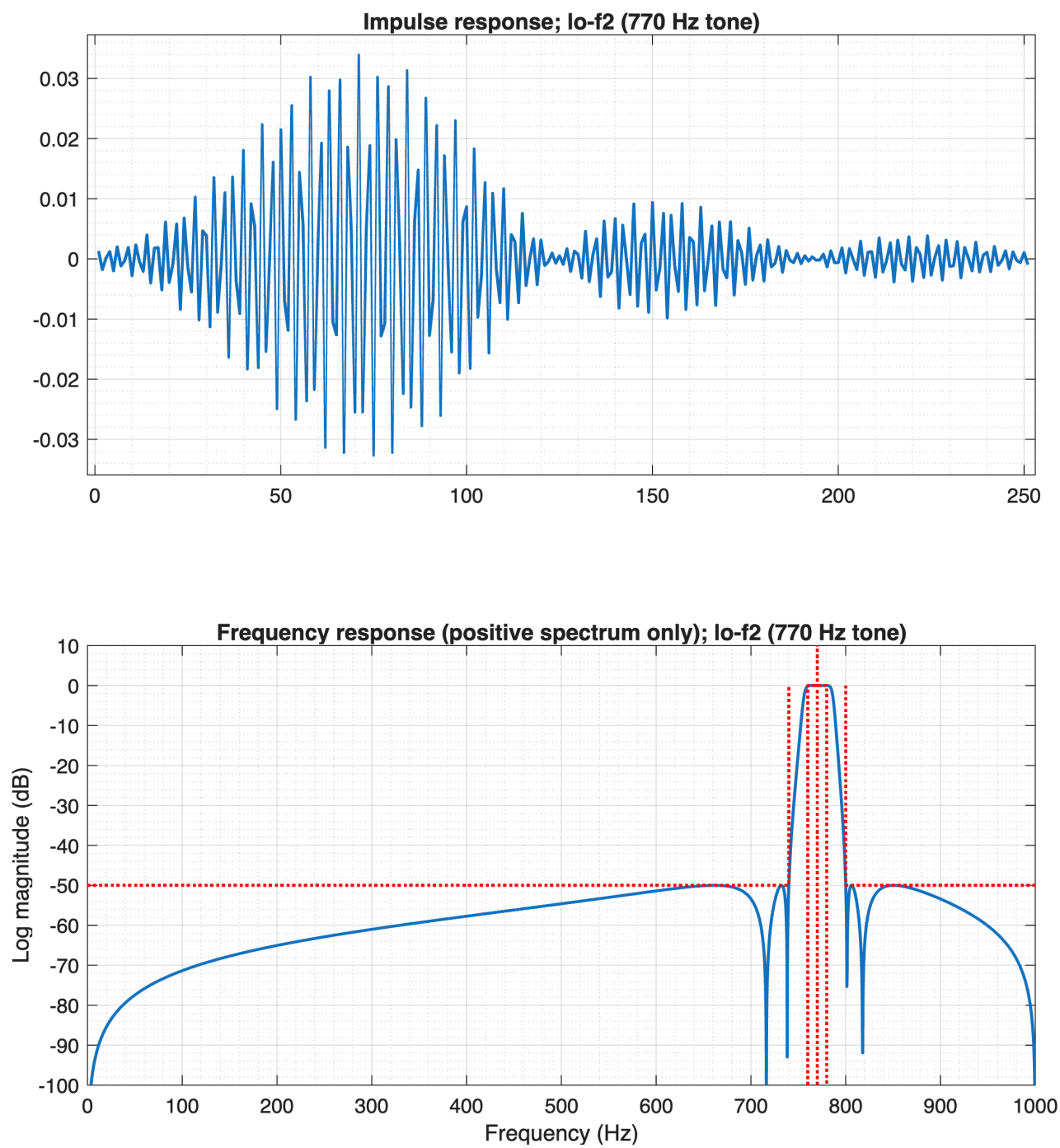
    % Stopband edge frequencies (normalized to Nyquist frequency)
    Ws = [lo_freqs(i) - (FPASS_HALFBW + FSTOP_WIDTH), ...
          lo_freqs(i) + (FPASS_HALFBW + FSTOP_WIDTH)] / (fs3/2);

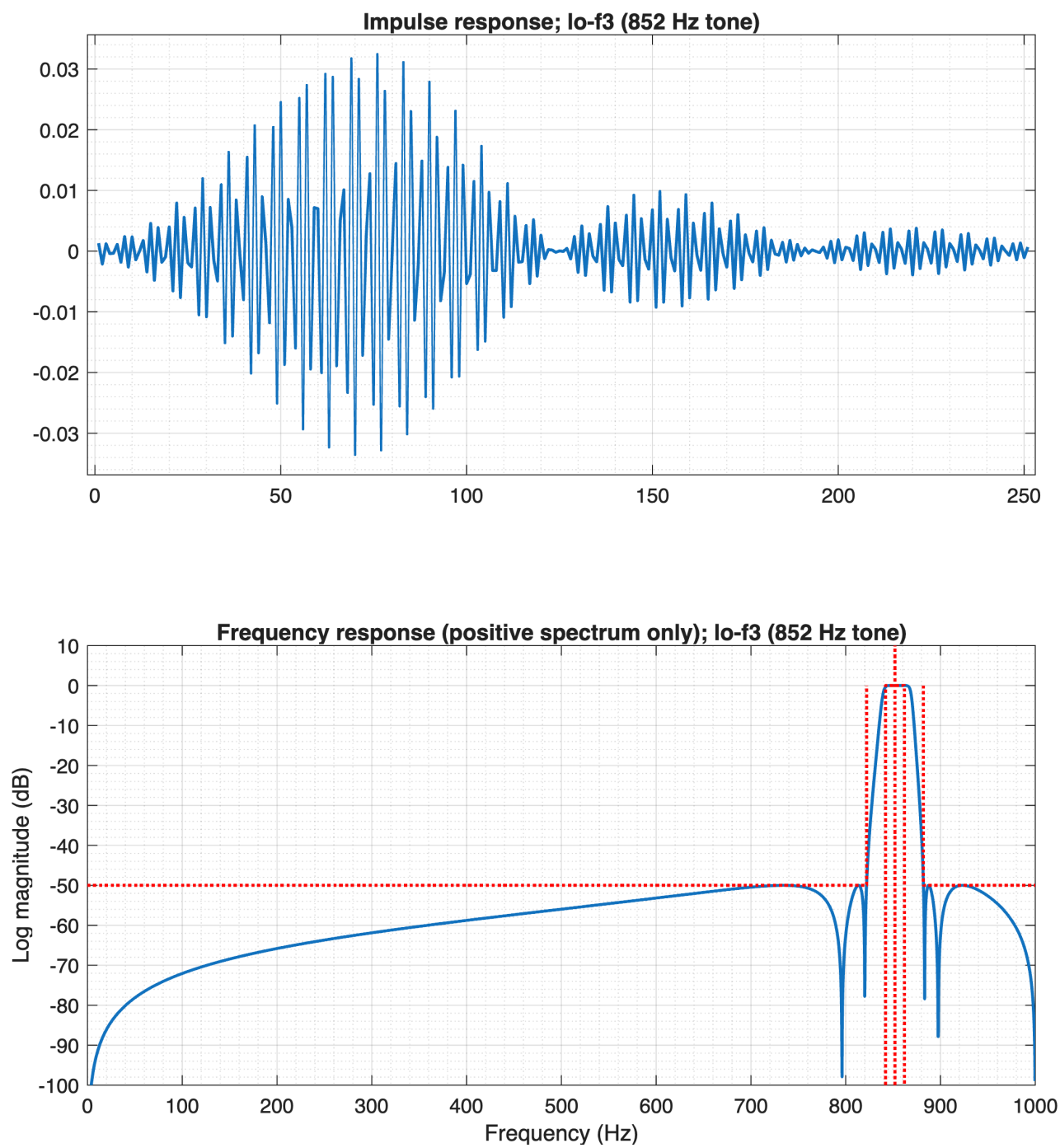
    NFILTER = cheb2ord(Wp, Ws, Rp, A_dB);
    [B, A] = cheby2(NFILTER, A_dB, Ws);

    % Convert to Second-Order Systems to account for numerical instabilities
    sos = tf2sos(B, A);
    plot_filt_resp_from_transfer_fn(...
        sos, 1, fs3, ...
        sprintf("lo-f%d (%d Hz tone)", i, lo_freqs(i)), false, ...
        false, true, true, 1251, true, Wp, Ws, A_dB);
end

```







```
% Center frequency of 941 Hz is too close to the Nyquist frequency, and a  
% bandpass filter does not work. Designing a high pass filter for just that  
% one
```

```
Wp = (lo_freqs(4) - FPASS_HALFBW) / (fs3/2);
```

```
Ws = (lo_freqs(4) - (FPASS_HALFBW + FSTOP_WIDTH)) / (fs3/2);
```

```
n3_lo_f4 = cheb2ord(Wp, Ws, Rp, A_dB);
```

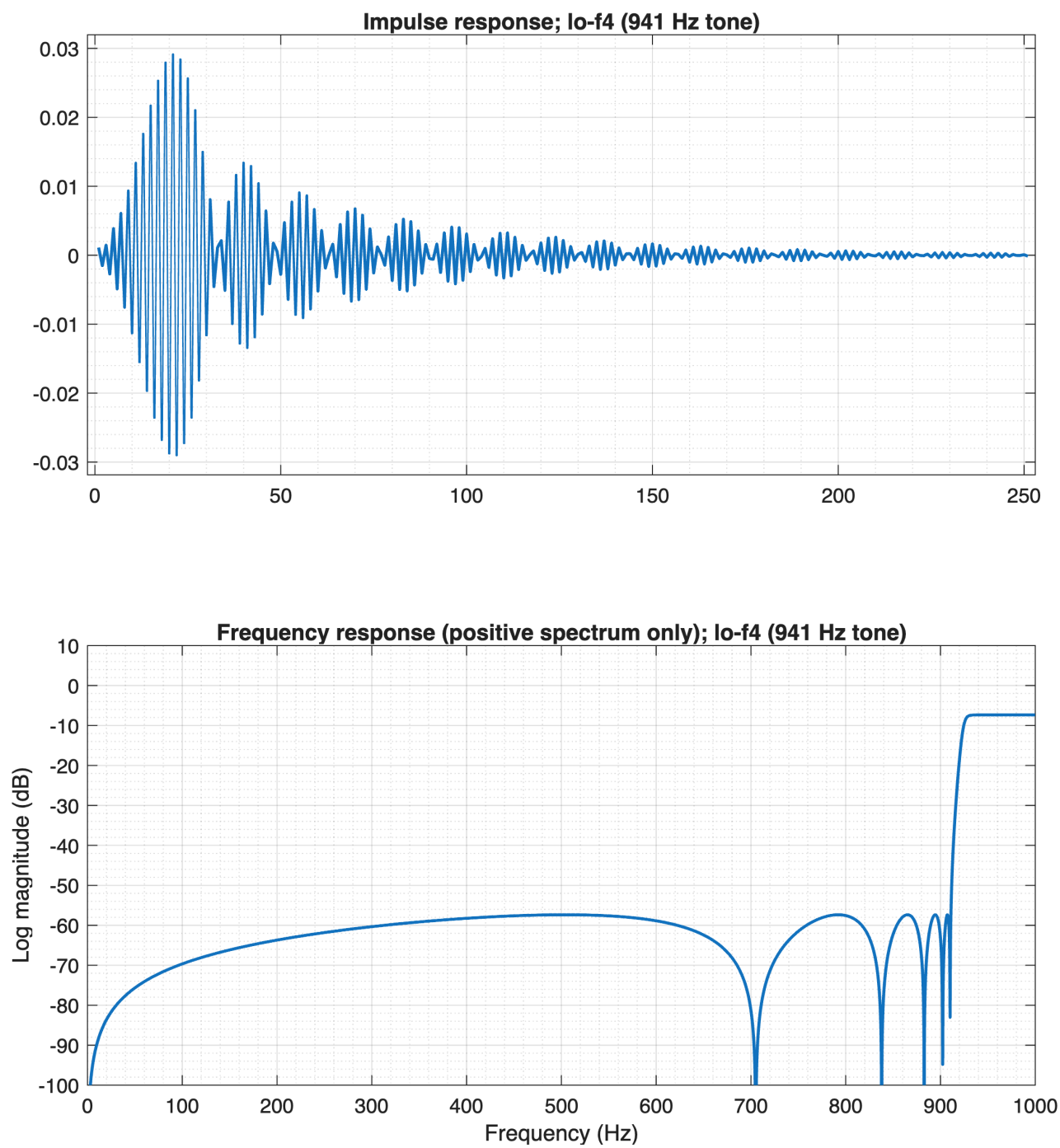
```
[B3_lo_f4, A3_lo_f4] = cheby2(n3_lo_f4, A_dB, Ws, "high");
```

```
plot_filt_resp_from_transfer_fn(...
```

```
    B3_lo_f4, A3_lo_f4, fs3, ...
```

```
    sprintf("lo-f%d (%d Hz tone)", 4, lo_freqs(4)), false, ...
```

```
    false, true, true, 1251, true, Wp, Ws, A_dB);
```



```

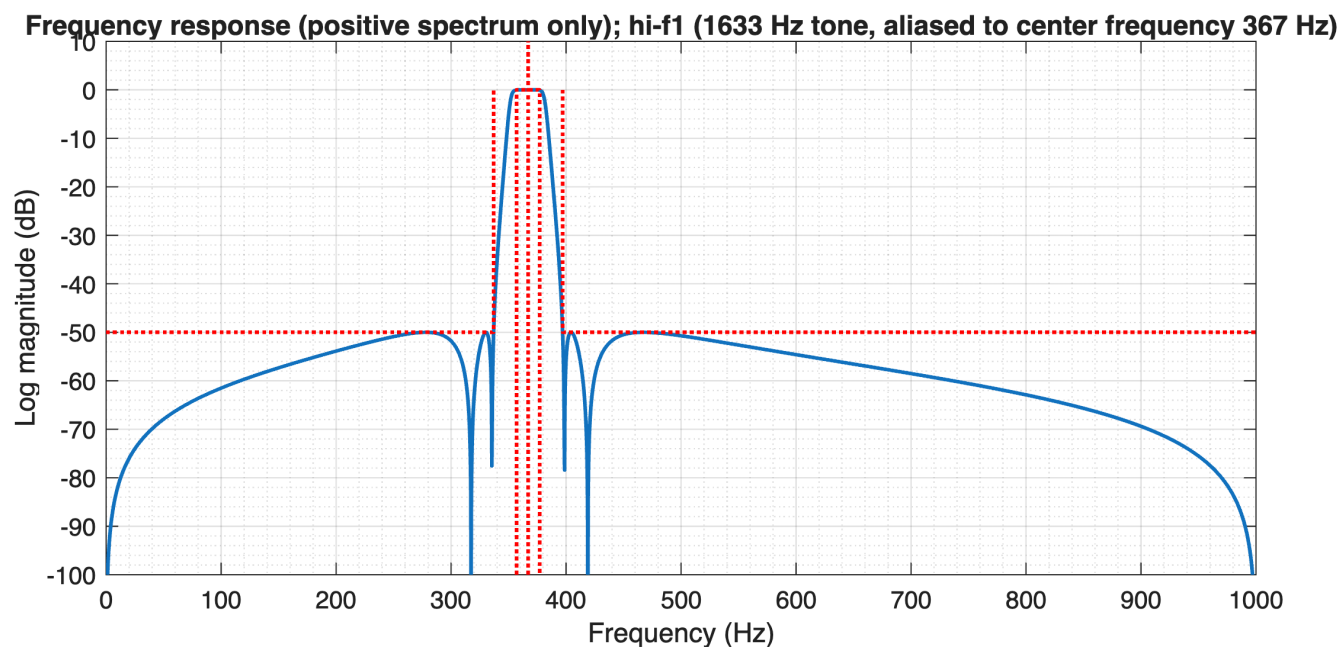
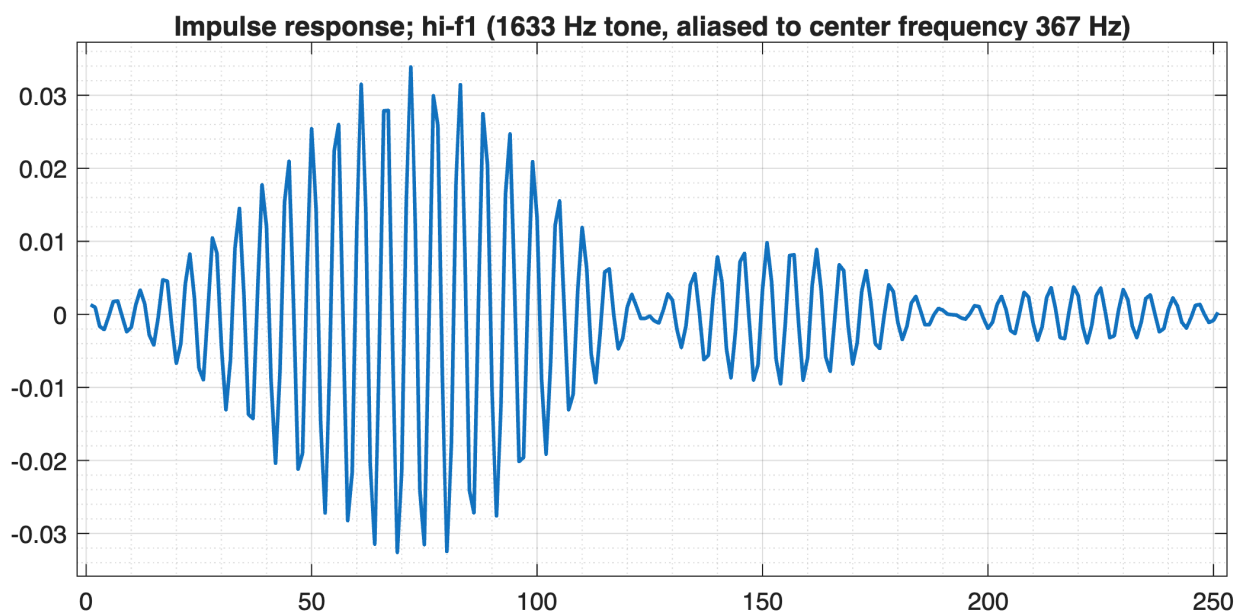
for i = 1:length(hi_aliased_freqs)
    % Passband edge frequencies (normalized to Nyquist frequency)
    Wp = [hi_aliased_freqs(i) - FPASS_HALFBW, ...
          hi_aliased_freqs(i) + FPASS_HALFBW] / (fs3/2);

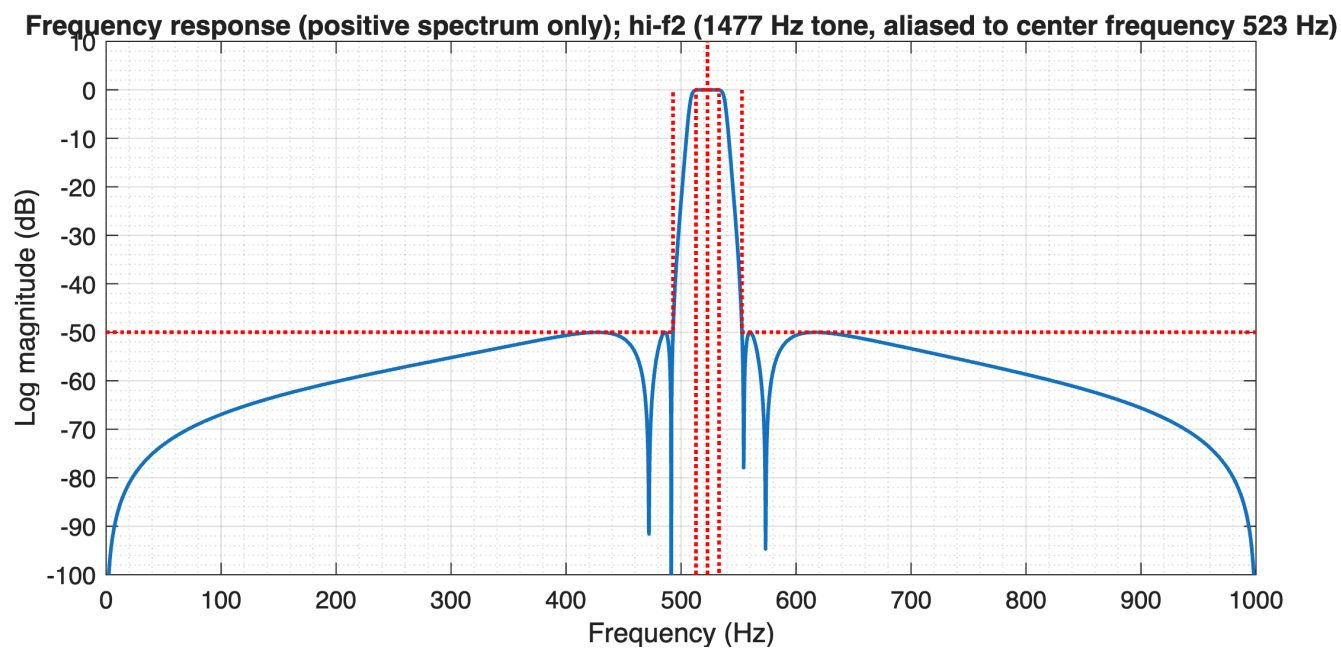
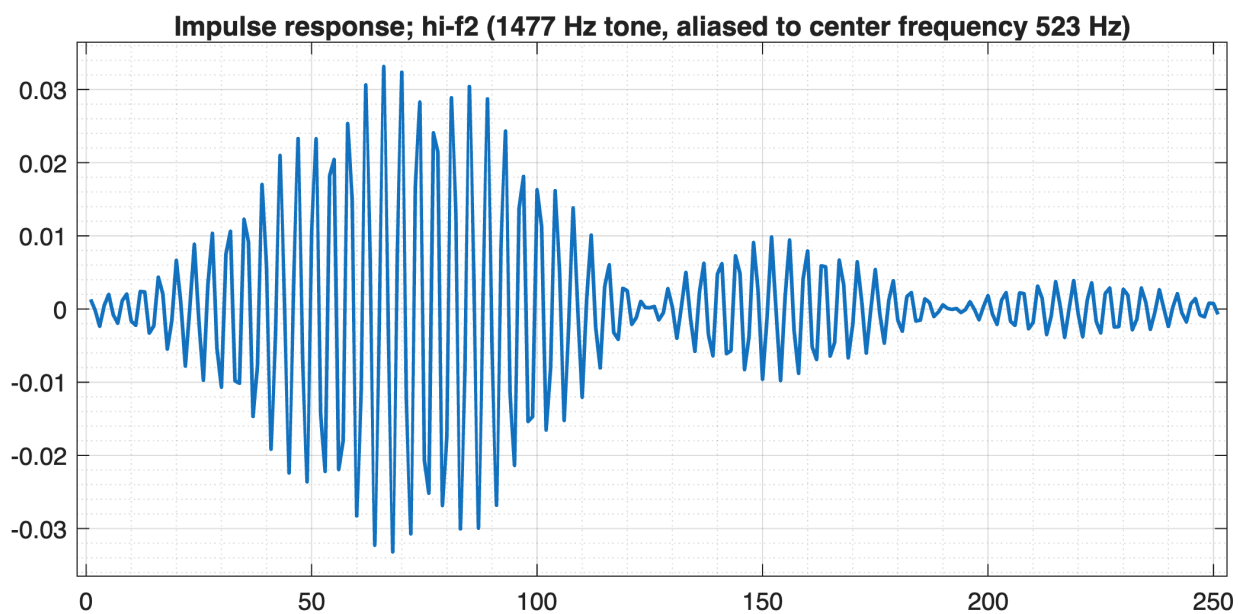
    % Stopband edge frequencies (normalized to Nyquist frequency)
    Ws = [hi_aliased_freqs(i) - (FPASS_HALFBW + FSTOP_WIDTH), ...
          hi_aliased_freqs(i) + (FPASS_HALFBW + FSTOP_WIDTH)] / (fs3/2);

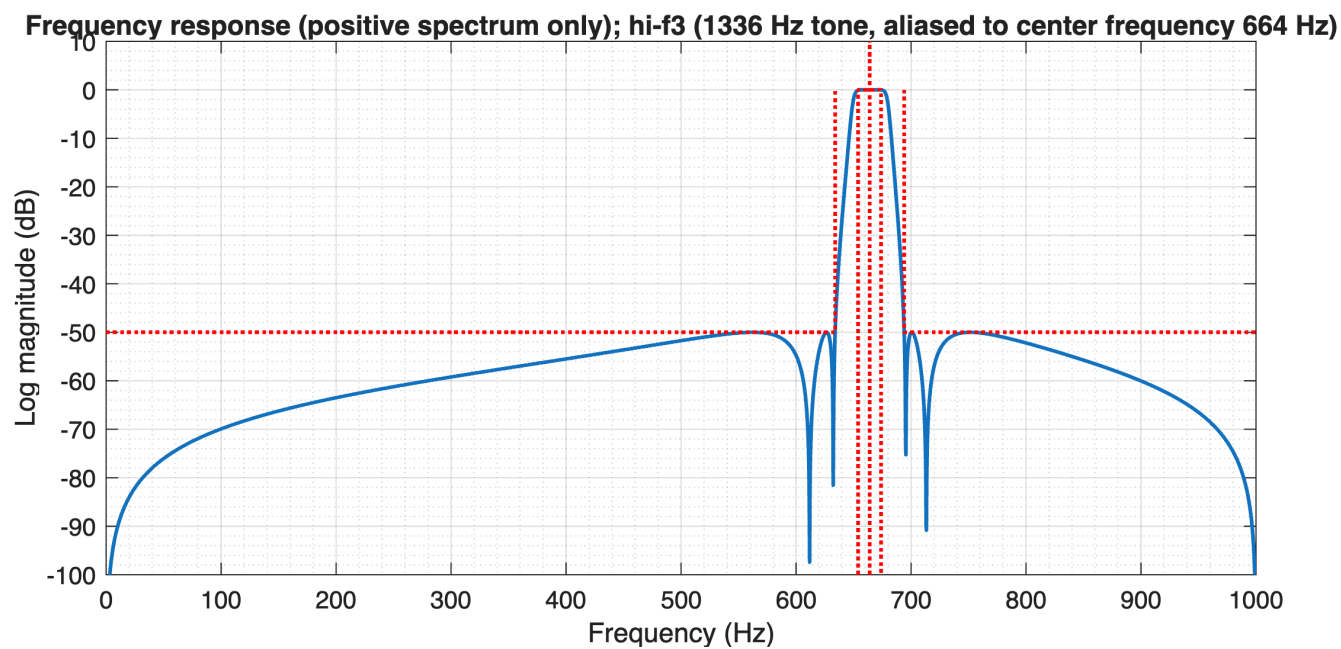
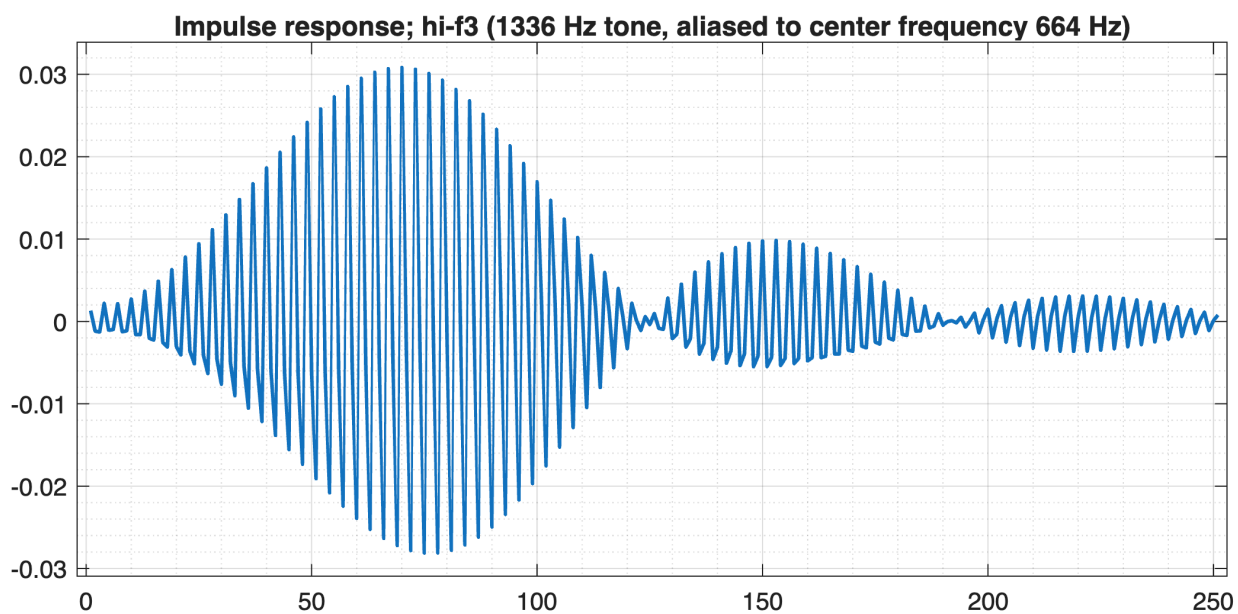
    NFILTER = cheb2ord(Wp, Ws, Rp, A_dB);
    [B, A] = cheby2(NFILTER, A_dB, Ws);

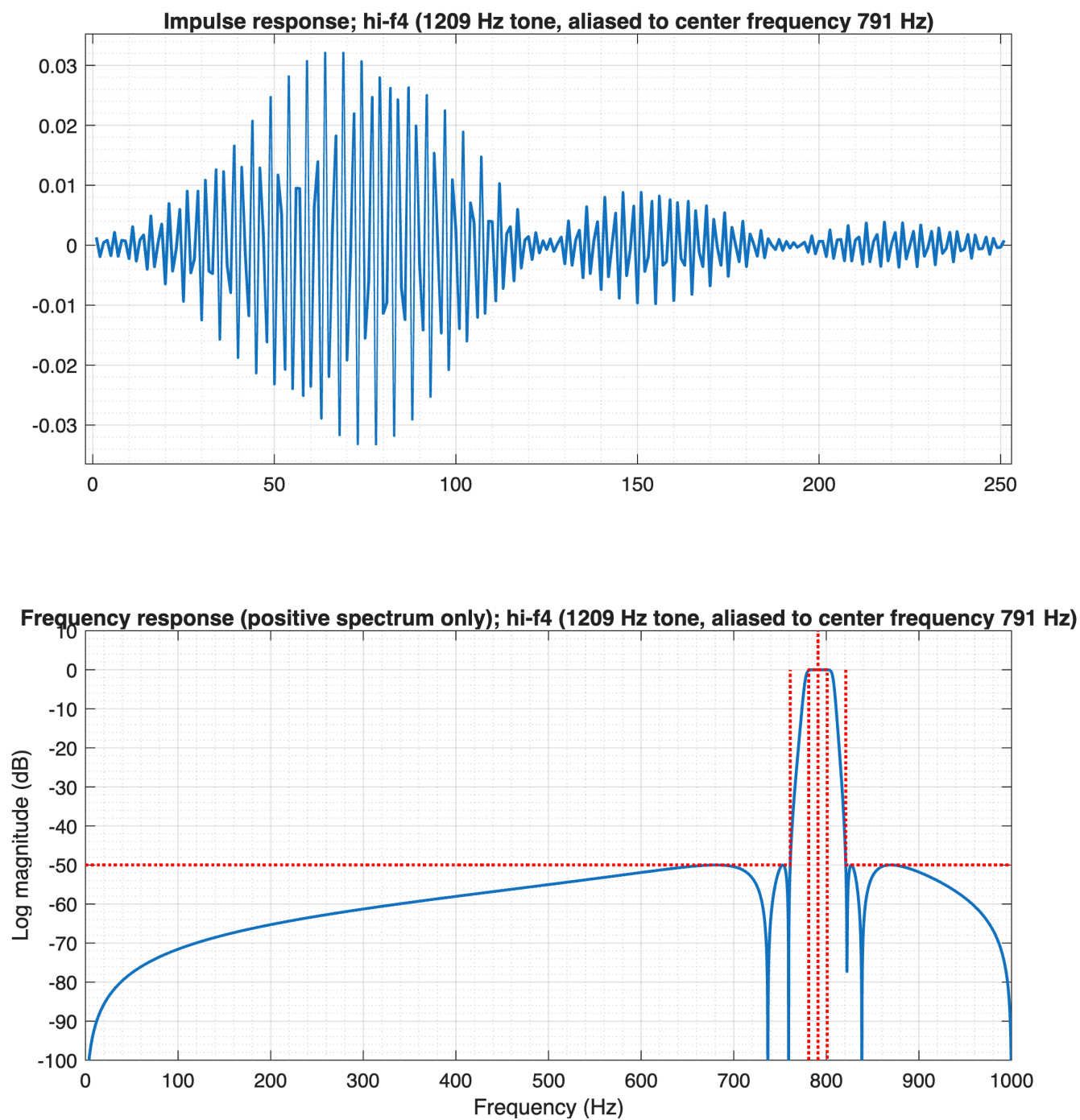
    % Convert to Second-Order Systems to account for numerical instabilities
    sos = tf2sos(B, A);
    plot_filt_resp_from_transfer_fn(...
        sos, 1, fs3, ...
        sprintf("hi-f%d (%d Hz tone, aliased to center frequency %d Hz)", ...
            i, fs3 - hi_aliased_freqs(i), hi_aliased_freqs(i)), ...
        false, false, true, true, 1251, true, Wp, Ws, A_dB);
end

```









As a nod to DSP course, I picked "A", "D" and "C" ("ADC") as the key presses.

```

pressed_keys = [key_A; key_D; key_C];
pressed_key_str = ["A", "D", "C"];

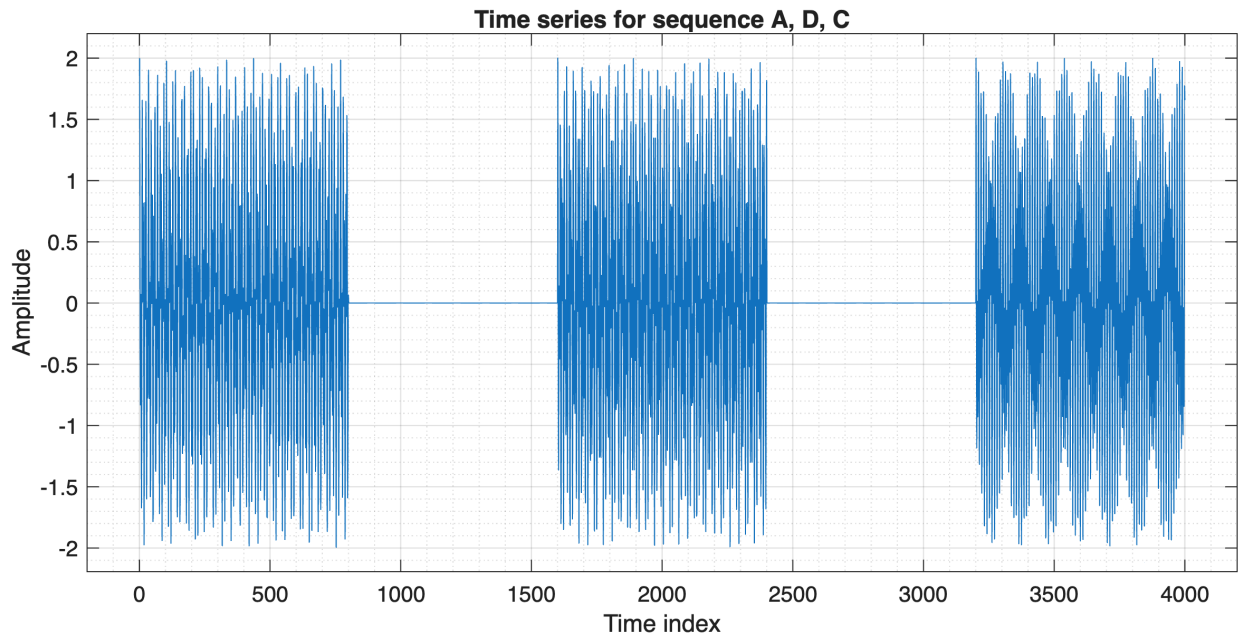
% Key press sequence time series
y = zeros(1, (2 * length(pressed_keys) - 1) * fs1 * key_press_time);

for idx = 1 : length(pressed_keys)
    pressed_key = pressed_keys(idx, 1:2);
    key_lo_freq = lo_freqs(pressed_key(1));
    key_hi_freq = hi_freqs(pressed_key(2));
    x = dtmf_time_signal_and_fft(...
        fs1, key_lo_freq, key_hi_freq, key_press_time, ...
        pressed_key_str(idx), false);

    % Time gap will only be in between the key presses
    if (idx ~= length(pressed_keys))
        x = [x, zeros(1, length(x))];
        y(1 + ((idx-1) * length(x)) : length(x) + ((idx-1) * length(x))) = x;
    else
        y(length(y) - length(x) + 1 : length(y)) = x;
    end
end

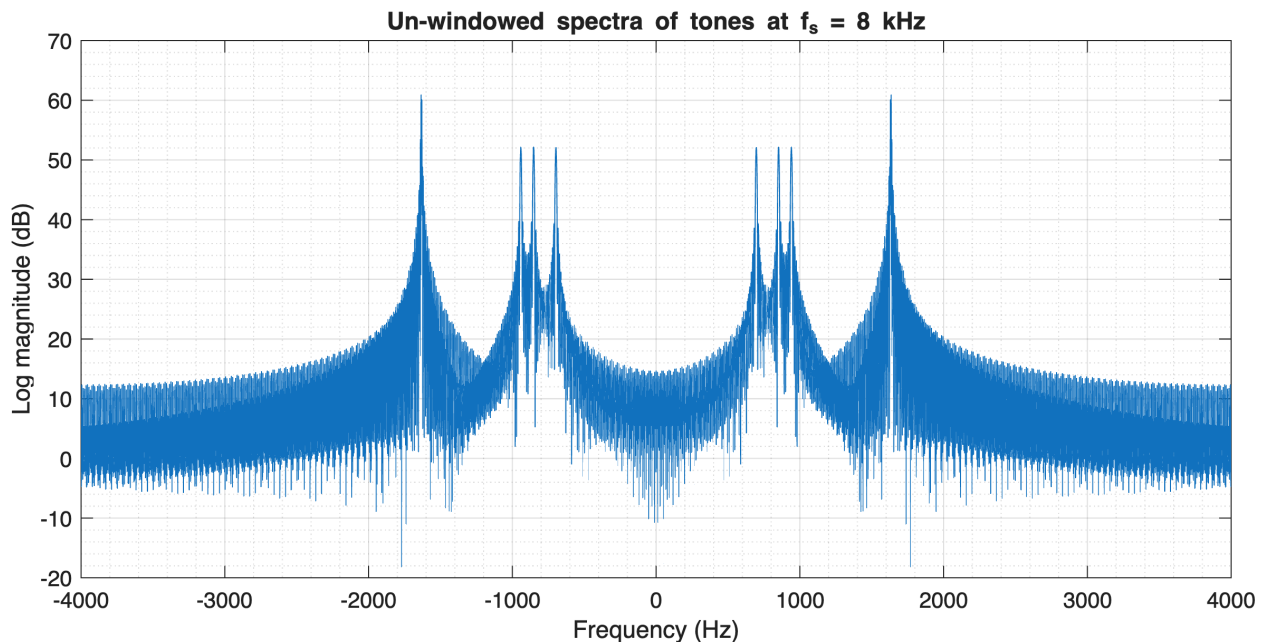
figure("Position", [0 0 900 400]);
plot(y);
title("Time series for sequence A, D, C");
grid on; grid minor; axis on;
axis([-0.05 * length(y), 1.05 * length(y), 1.1 * min(y), 1.1 * max(y)]);
xlabel("Time index"); ylabel("Amplitude");

```



```
% Spectra of tones
NFFT = 8192;
f_y = fftshift(20 * log10(abs(fft(y, NFFT))));

figure("Position", [0 0 900 400]);
plot((-0.5 : 1/NFFT : 0.5 - 1/NFFT) * fs1, f_y);
grid on; grid minor; axis on;
title(...
    sprintf("Un-windowed spectra of tones at f_s = %.5g kHz", fs1/1000));
xlabel("Frequency (Hz)"); ylabel("Log magnitude (dB)");
```



```

% Pass the DTMF composite signal through your filter bank and show the time
% response of the filter bank.

% Stage 1 (low pass filter)
y = filter(B1, A1, y);
y_ds = y(1 : 2 : length(y));

% Stage 2 (low and high pass filters)
% Pass tones through low pass and high pass filters and downsample 2-to-1
s_lo = filter(B2_lo, A2_lo, y_ds);
s_hi = filter(B2_hi, A2_hi, y_ds);
s_lo_ds = s_lo(1 : 2 : length(s_lo));
s_hi_ds = s_hi(1 : 2 : length(s_hi));

% Stage 3 filters (lo/hi-f1/2/3/4 filters)
figure("Position", [0 0 900 950]);
for i = 1 : (length(lo_freqs) - 1)
    % Passband edge frequencies (normalized to Nyquist frequency)
    Wp = [lo_freqs(i) - FPASS_HALFBW, lo_freqs(i) + FPASS_HALFBW] / (fs3/2);

    % Stopband edge frequencies (normalized to Nyquist frequency)
    Ws = [lo_freqs(i) - (FPASS_HALFBW + FSTOP_WIDTH), ...
          lo_freqs(i) + (FPASS_HALFBW + FSTOP_WIDTH)] / (fs3/2);

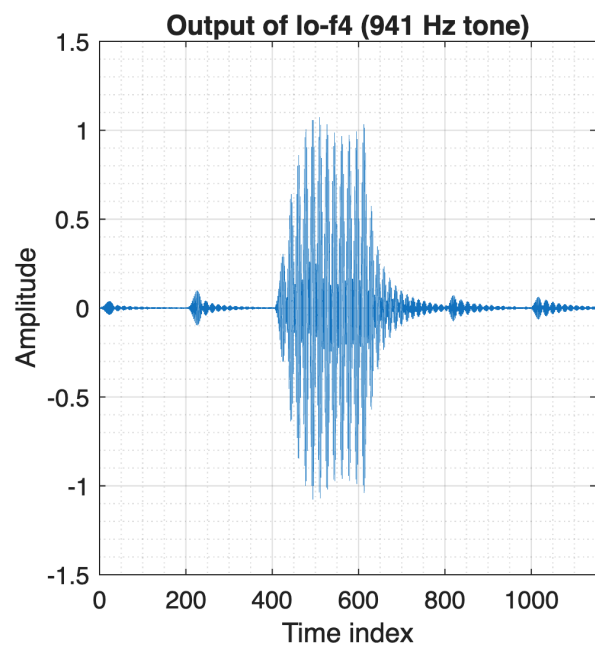
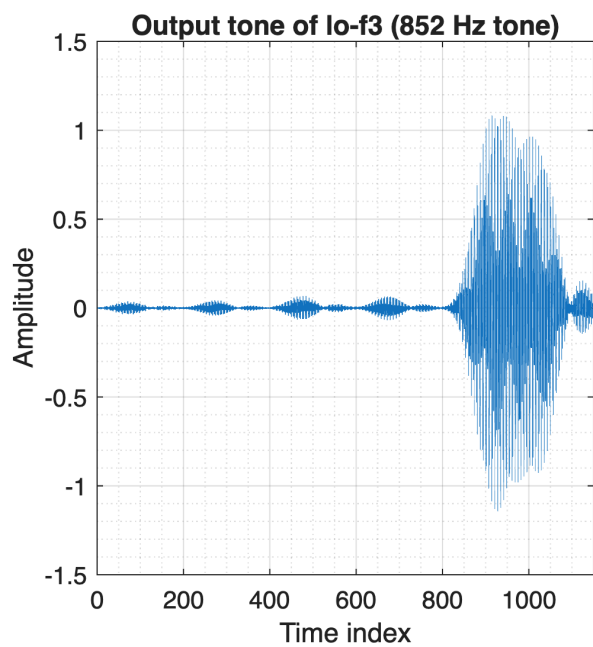
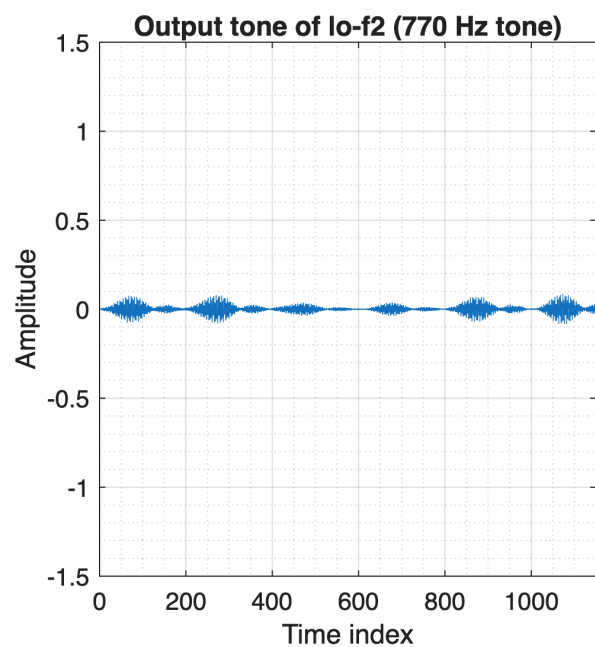
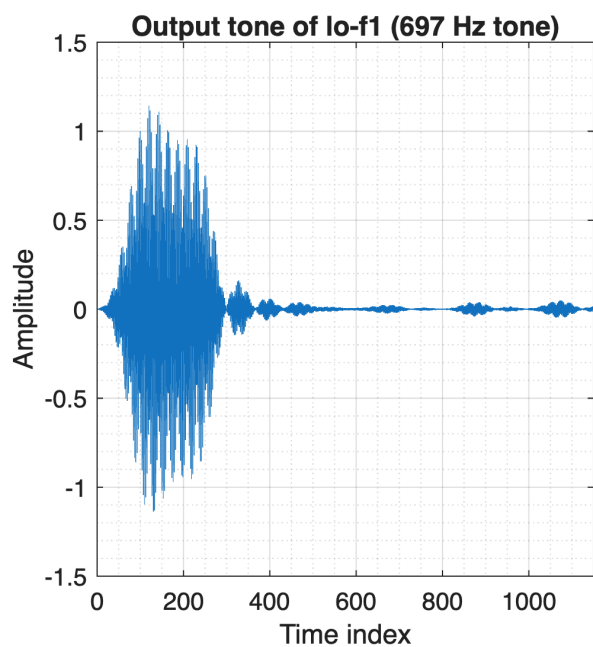
    NFILTER = cheb2ord(Wp, Ws, Rp, A_dB);
    [B, A] = cheby2(NFILTER, A_dB, Ws);

    % Convert to Second-Order Systems to account for numerical instabilities
    sos = tf2sos(B, A);

    subplot(2, 2, i);
    plot(sosfilt(sos, [s_lo_ds zeros(1, 150)]));
    title(sprintf("Output tone of lo-f%d (%d Hz tone)", i, lo_freqs(i)));
    grid on; grid minor; axis on;
    ylim([-1.5 1.5]);
    xlabel("Time index"); ylabel("Amplitude");
end

subplot(2, 2, 4);
plot(filter(B3_lo_f4, A3_lo_f4, [s_lo_ds zeros(1, 150)]));
title(sprintf("Output of lo-f4 (%d Hz tone)", lo_freqs(4)));
grid on; grid minor; axis on;
ylim([-1.5 1.5]);
xlabel("Time index"); ylabel("Amplitude");

```



```

figure("Position", [0 0 900 950]);

for i = 1:length(hi_aliased_freqs)
    % Passband edge frequencies (normalized to Nyquist frequency)
    Wp = [hi_aliased_freqs(i) - FPASS_HALFBW, ...
          hi_aliased_freqs(i) + FPASS_HALFBW] / (fs3/2);

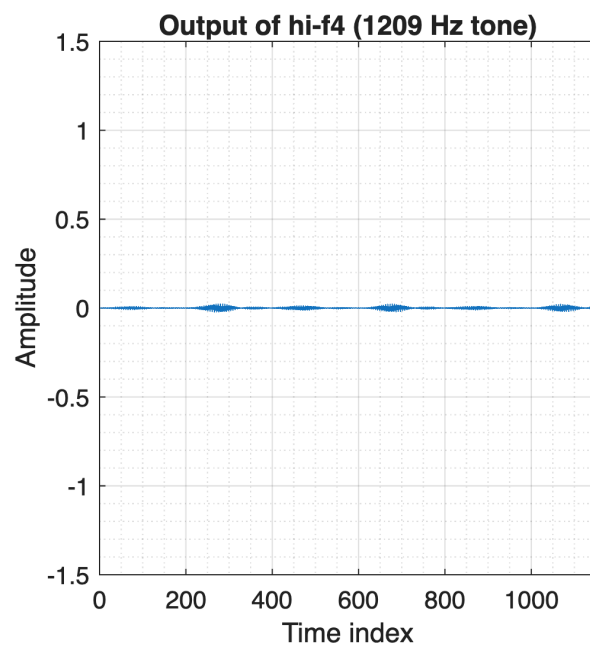
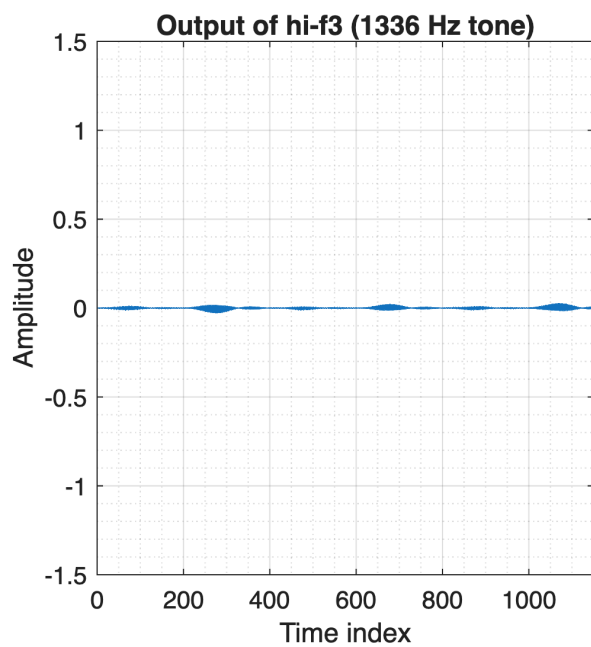
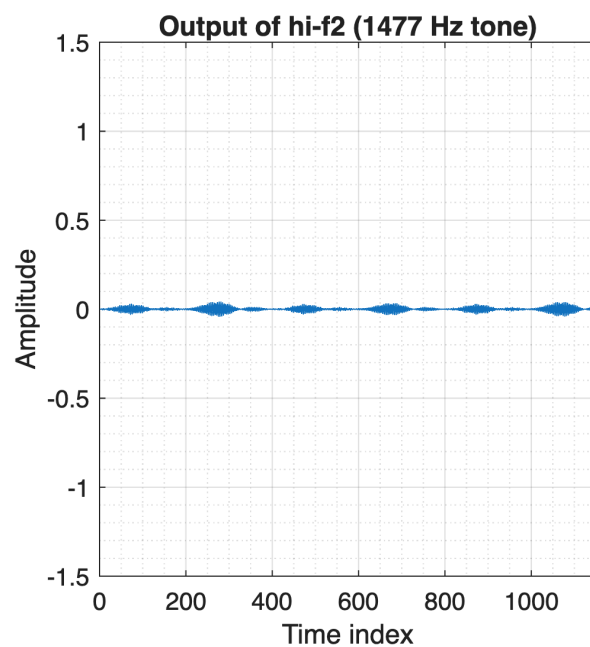
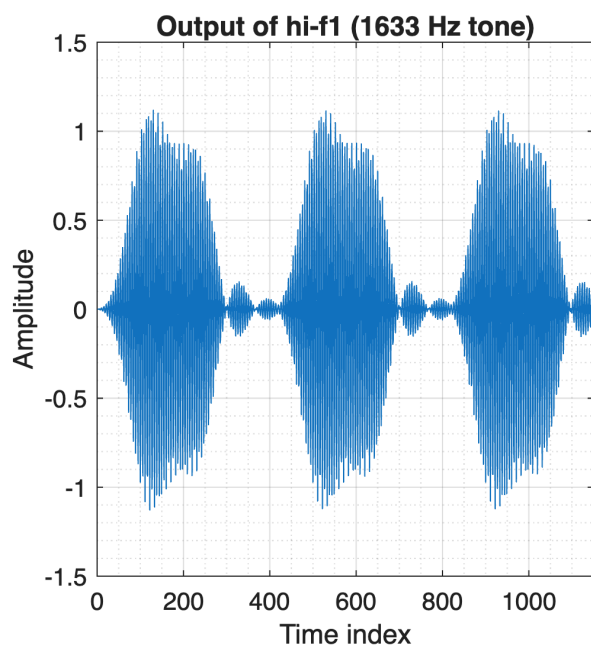
    % Stopband edge frequencies (normalized to Nyquist frequency)
    Ws = [hi_aliased_freqs(i) - (FPASS_HALFBW + FSTOP_WIDTH), ...
          hi_aliased_freqs(i) + (FPASS_HALFBW + FSTOP_WIDTH)] / (fs3/2);

    NFILTER = cheb2ord(Wp, Ws, Rp, A_dB);
    [B, A] = cheby2(NFILTER, A_dB, Ws);

    % Convert to Second-Order Systems to account for numerical instabilities
    sos = tf2sos(B, A);

    subplot(2, 2, i);
    plot(sosfilt(sos, [s_hi_ds zeros(1, 150)]));
    title(...
        sprintf("Output of hi-f%d (%d Hz tone)", i, (fs3 -
hi_aliased_freqs(i))));
    grid on; grid minor; axis on;
    ylim([-1.5 1.5]);
    xlabel("Time index"); ylabel("Amplitude");
end

```



4h

Number of co-efficients (filter order) for FIR filters is much larger than that of IIR cheby2 filters.

The same is noted in each individual figure.

```
% Stage 1 (low pass filter)
% FIR filter design
f1 = Wp1 * fs1 / 2; % Passband edge
f2 = Ws1 * fs1 / 2; % Stopband edge

delta_1 = (10 ^ (Rp / 20)) - 1;
delta_2 = 10 ^ (-A_dB / 20);

w1 = 1;
w2 = delta_1 / delta_2;

% Use the harris approximation
M = floor((fs1 / (f2 - f1)) * A_dB / 15);

% This seems to over-estimate the filter order; we get a much higher
% stopband attenuation than the specification
M = M - 12;

% If M is even, make it odd
if (M - 2 * ceil(M / 2) == 0)
    M = M + 1;
end

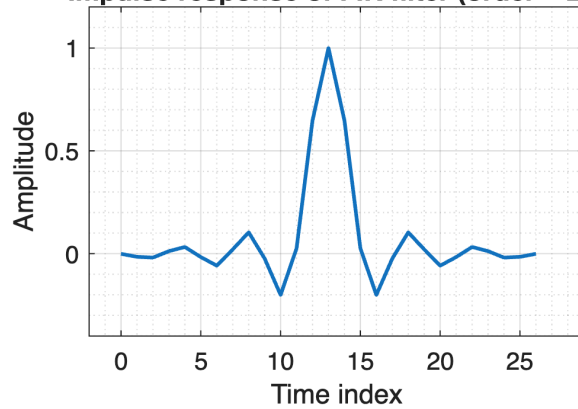
% Get the FIR filter co-efficients
h1_fir = firpm(M - 1, [0 Wp1 Ws1 1], {"myfrf", [1 1 0 0]}, [w1 w2]);

% Get the IIR filter response from previously calculated co-efficients
h1_iir = filter(B1, A1, [1 zeros(1, 80)]);

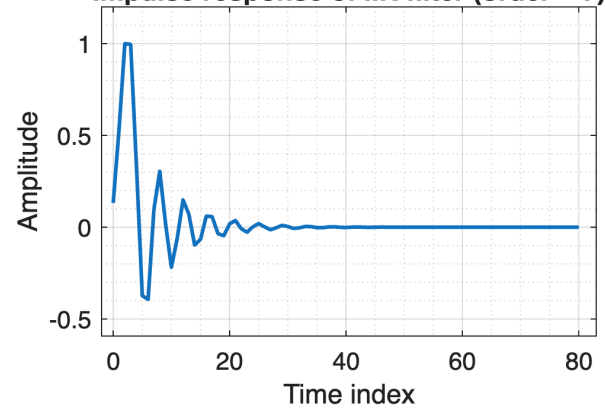
plot_fir_iir_filt_resp(h1_fir, h1_iir, M, n1, f1, f2, fs1, A_dB);
```

Low Pass Filter, $f_s = 8 \text{ kHz}$, $f_{\text{pass}} = 1.65 \text{ kHz}$, $f_{\text{stop}} = 2.35 \text{ kHz}$

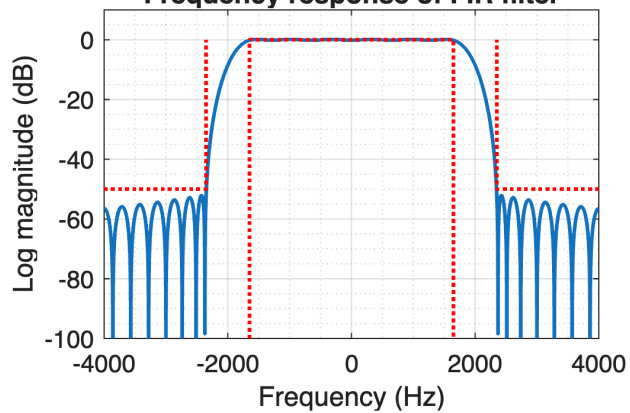
Impulse response of FIR filter (order = 27)



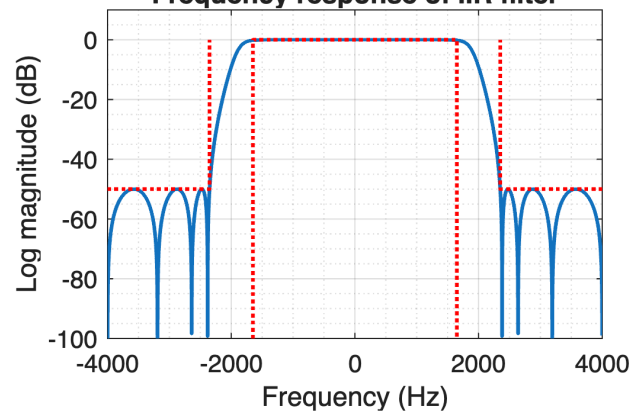
Impulse response of IIR filter (order = 7)



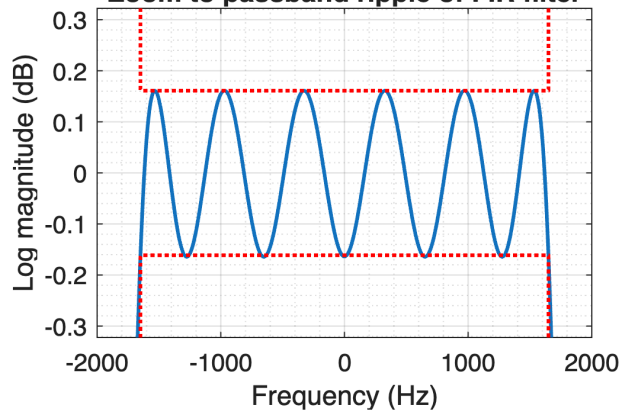
Frequency response of FIR filter



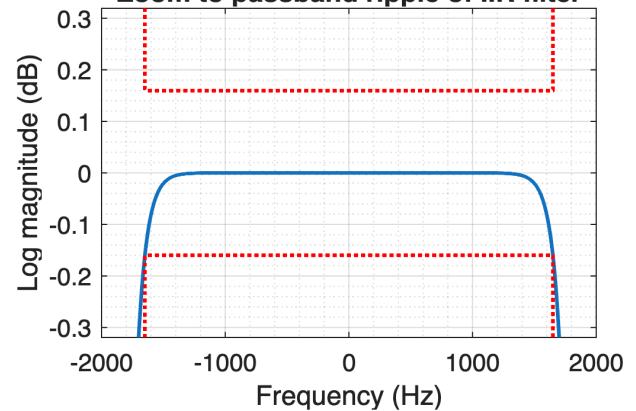
Frequency response of IIR filter



Zoom to passband ripple of FIR filter



Zoom to passband ripple of IIR filter



```

% Stage 2 (low and high pass filters)
% Low pass FIR filter design
f1 = Wp2_lo * fs2 / 2; % Passband edge
f2 = Ws2_lo * fs2 / 2; % Stopband edge

delta_1 = (10 ^ (Rp / 20)) - 1;
delta_2 = 10 ^ (-A_dB / 20);

w1 = 1;
w2 = delta_1 / delta_2;

% Use the harris approximation
M = floor((fs2 / (f2 - f1)) * A_dB / 15);

% This seems to over-estimate the filter order; we get a much higher
% stopband attenuation than the specification
M = M - 19;

% If M is even, make it odd
if (M - 2 * ceil(M / 2) == 0)
    M = M + 1;
end

% Get the FIR filter co-efficients
h2_lo_fir = firpm(M - 1, [0 Wp2_lo Ws2_lo 1], {"myfrf", [1 1 0 0]}, [w1
w2]);

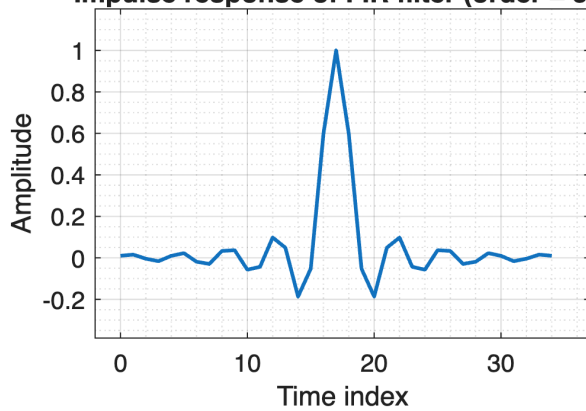
% Get the IIR filter response from previously calculated co-efficients
h2_lo_iir = filter(B2_lo, A2_lo, [1 zeros(1, 80)]);

plot_fir_iir_filt_resp(...
    h2_lo_fir, h2_lo_iir, M, n2_lo, f1, f2, fs2, A_dB);

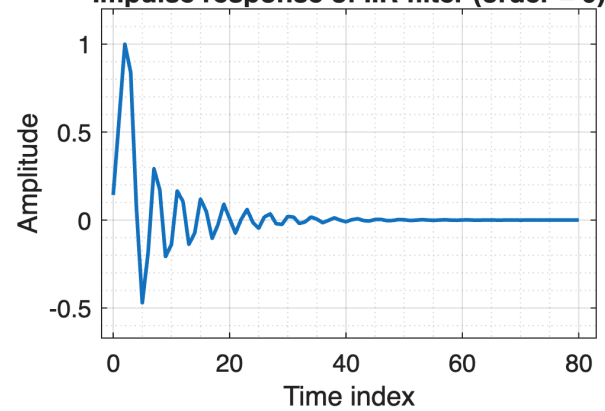
```

Low Pass Filter, $f_s = 4 \text{ kHz}$, $f_{\text{pass}} = 0.95 \text{ kHz}$, $f_{\text{stop}} = 1.2 \text{ kHz}$

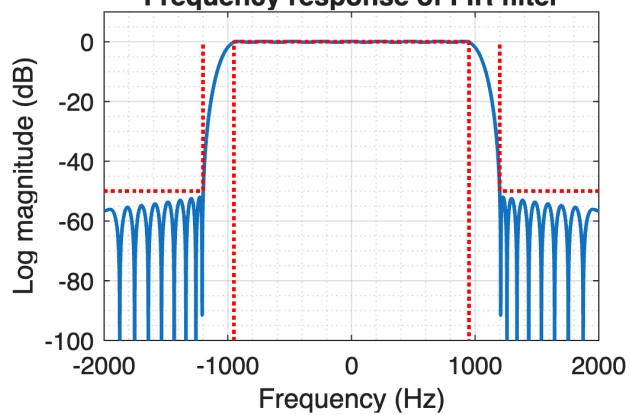
Impulse response of FIR filter (order = 35)



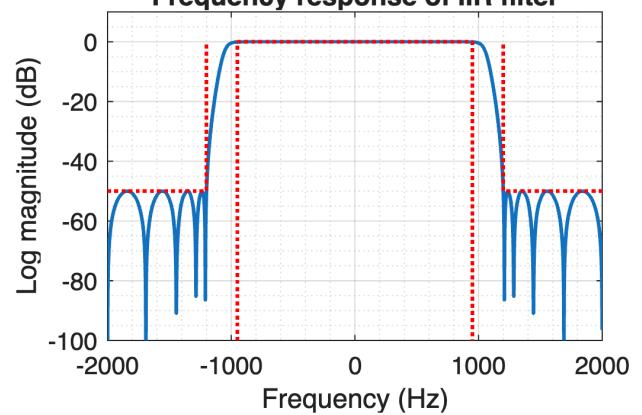
Impulse response of IIR filter (order = 9)



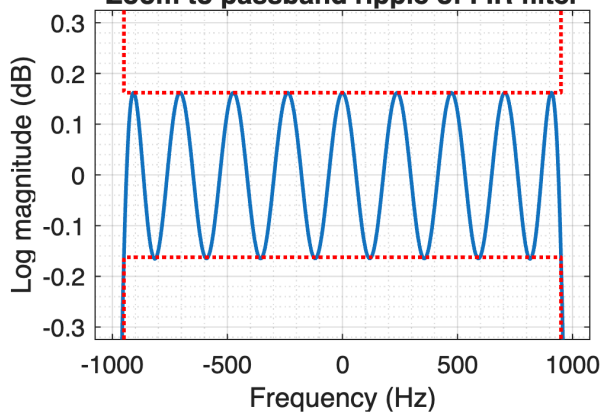
Frequency response of FIR filter



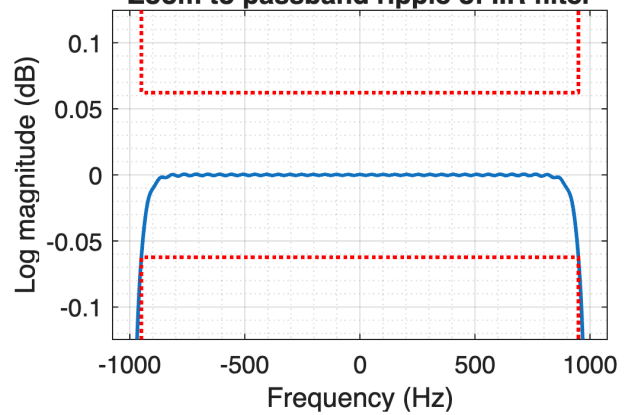
Frequency response of IIR filter



Zoom to passband ripple of FIR filter



Zoom to passband ripple of IIR filter



```

% High pass FIR filter design
% NOTE: "myfrf" does not work with high pass filter?
%       Heterodyne of above LPF doesn't quite give the f_pass & f_stop we
%       need
% Asked the prof in class: Yes, "myfrf" was only written for LPF.
%MM = (M - 1) / 2;
%h2_hi_fir = h2_lo_fir .* cos(pi * (-MM : MM));
%h2_hi_fir = firpm(50, [0 f1 f2 fs/2] / (fs/2), {"myfrf", [0 0 1 1]}, [w1
w2]);

f1 = Wp2_hi * fs2 / 2; % Passband edge
f2 = Ws2_hi * fs2 / 2; % Stopband edge

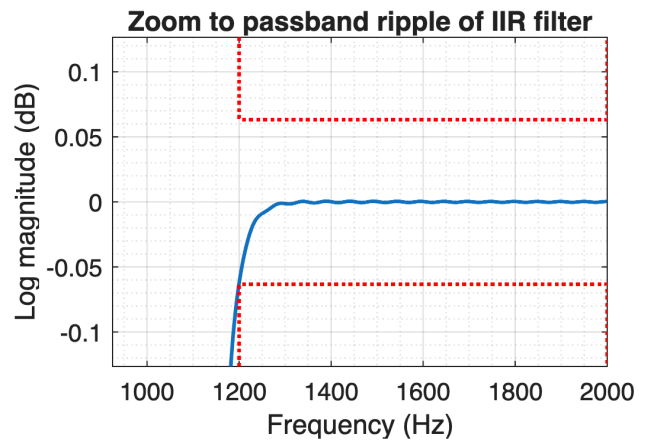
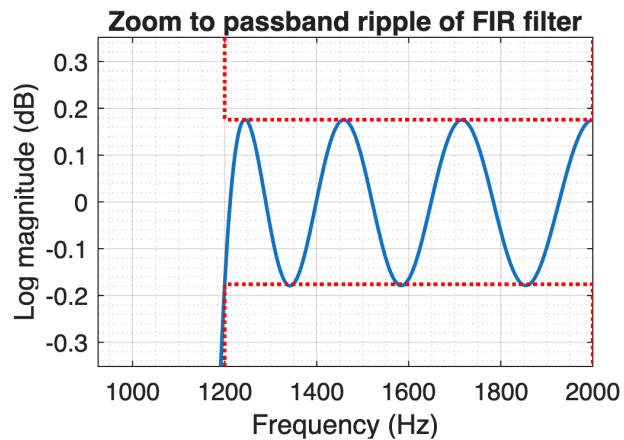
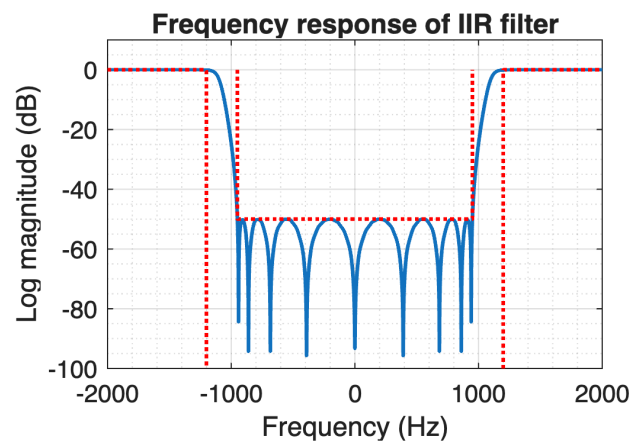
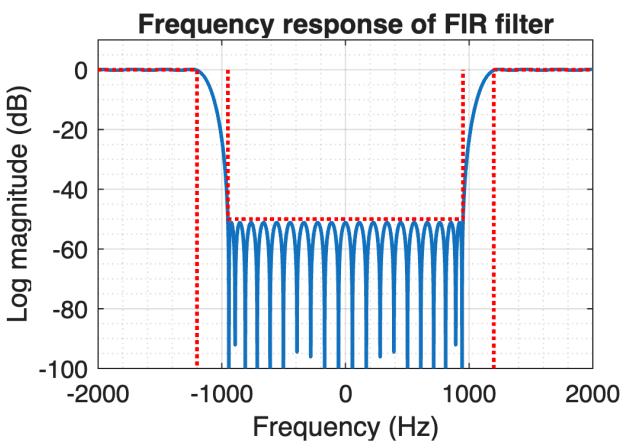
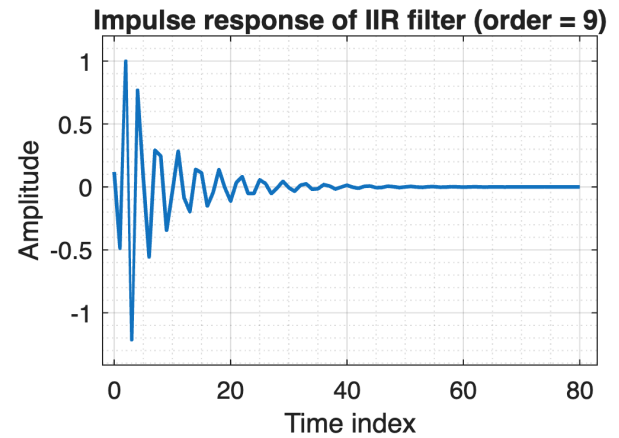
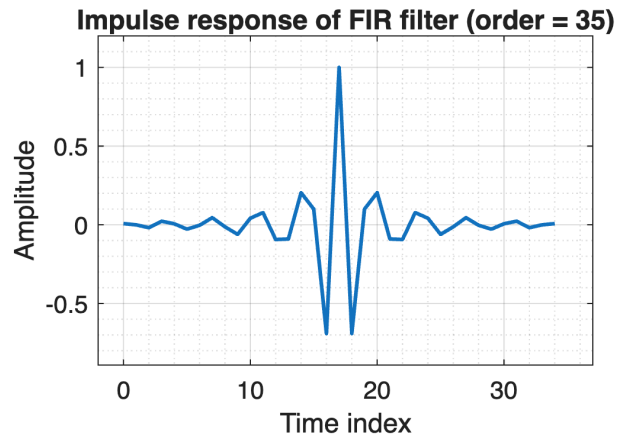
% Get the FIR filter co-efficients
h2_hi_fir = firpm(M - 1, [0 Ws2_hi Wp2_hi 1], [0 0 1 1], [w2 w1]);

% Get the IIR filter response from previously calculated co-efficients
h2_hi_iir = filter(B2_hi, A2_hi, [1 zeros(1, 80)]);

plot_fir_iir_filt_resp(...
    h2_hi_fir, h2_hi_iir, M, n2_hi, f1, f2, fs2, A_dB, "High");

```

High Pass Filter, $f_s = 4 \text{ kHz}$, $f_{\text{pass}} = 1.2 \text{ kHz}$, $f_{\text{stop}} = 0.95 \text{ kHz}$



```

% Stage 3 filters (lo-f1/2/3 and hi-f1/2/3/4 bandpass filters)
% NOTE: lo-f4 (941 Hz center frequency will be high pass due to the
%       proximity to Nyquist frequency of 1 kHz.

% Update the sample rate to 2-to-1 downsample rate after the low and high
pass
% filters
fs3 = fs2 / 2; % Sampling frequency is 2000 Hz now

lo_freqs = [ 697,  770,  852,  941]; % In Hz
hi_freqs = [1209, 1336, 1477, 1633]; % In Hz
hi_aliased_freqs = sort(fs3 - hi_freqs); % In Hz

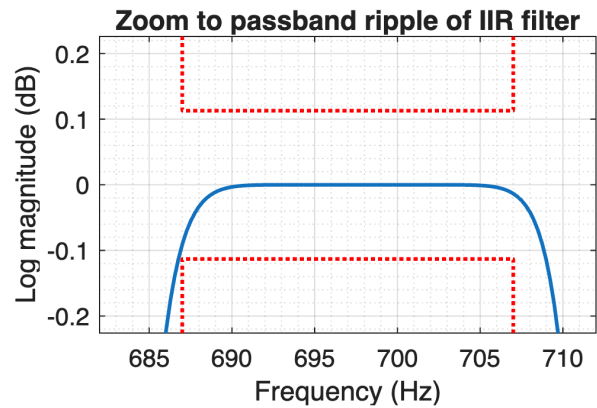
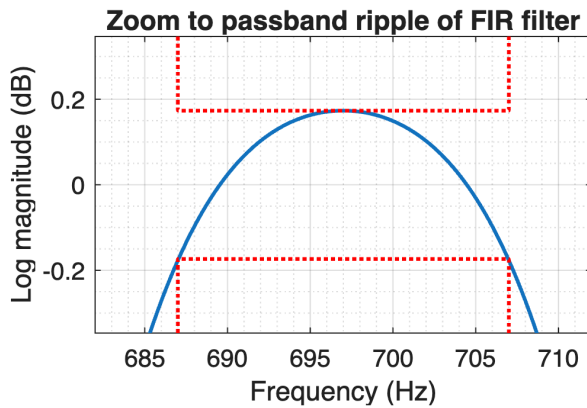
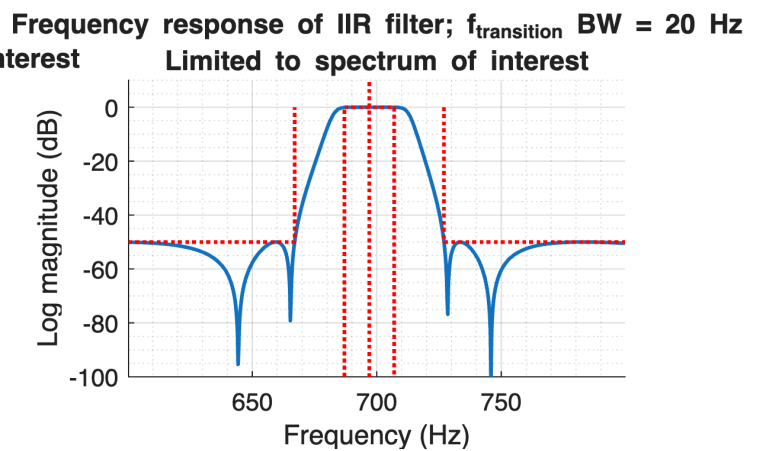
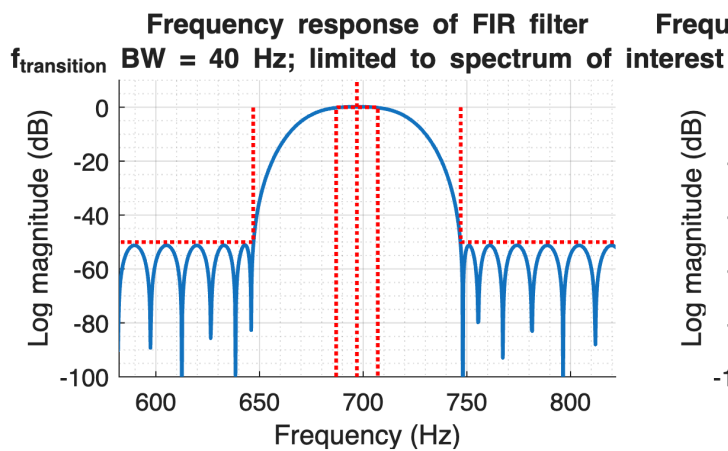
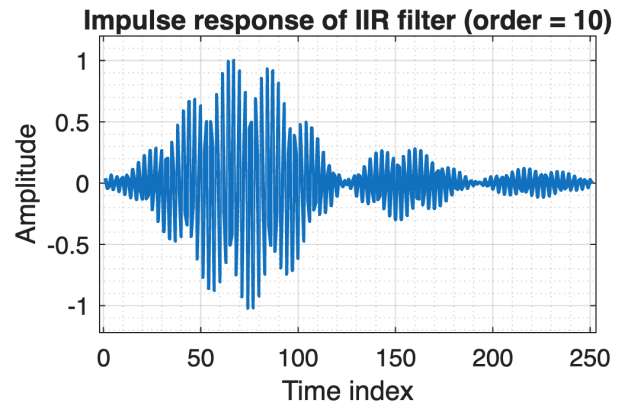
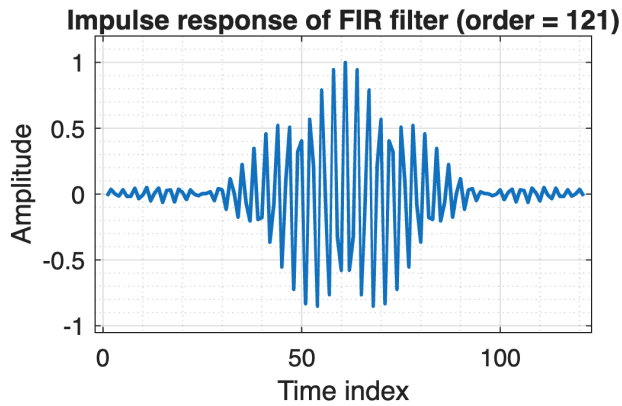
% Half bandwidth around the center (tone) frequency
% For example, the lo-f1 filter for 697 Hz tone would be centered at 697 Hz
and
% passband would be from 687 Hz to 707 Hz with FPASS_HALFBW of 10 Hz
FPASS_HALFBW = 10; % In Hz

% Transition band width (measured from the corresponding passband edge)
% Continuing the example in the comment above.
% With FSTOP_WIDTH of 20 Hz, the lower transition band would be from 667 to
% 687 Hz and higher transition band would be from 707 to 727 Hz.
FSTOP_WIDTH_FIR = 40; % In Hz
FSTOP_WIDTH_IIR = 20; % In Hz

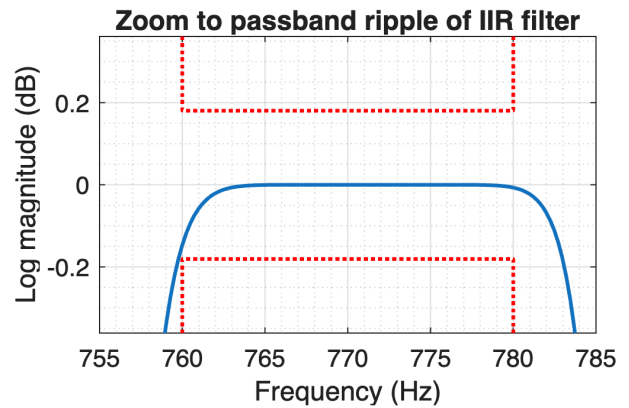
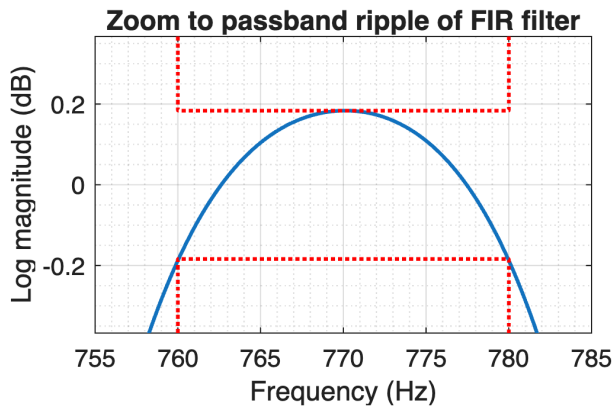
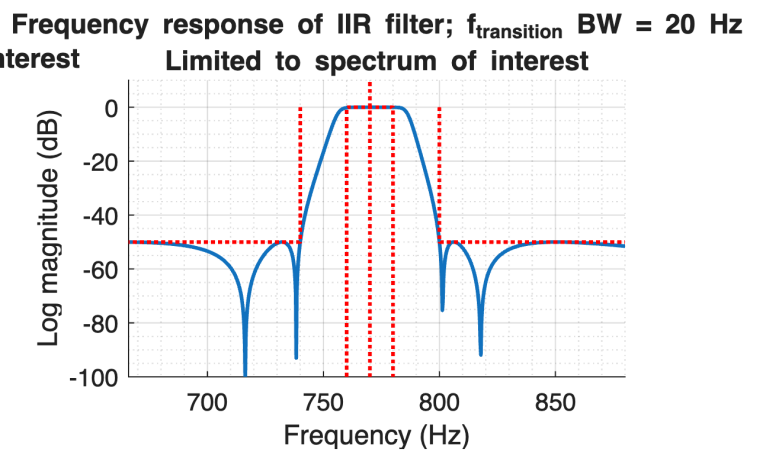
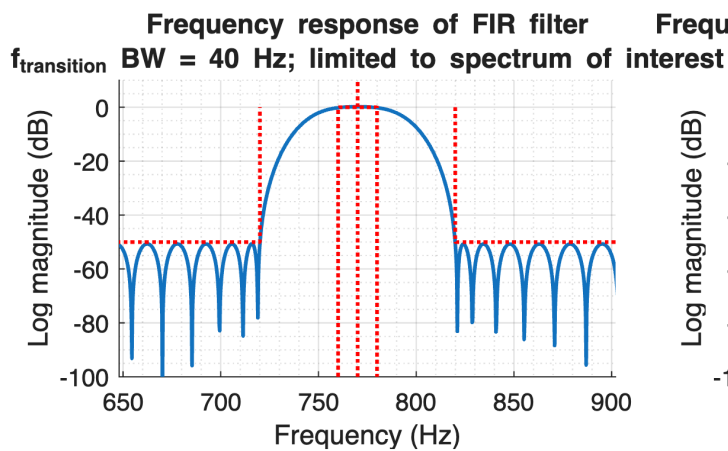
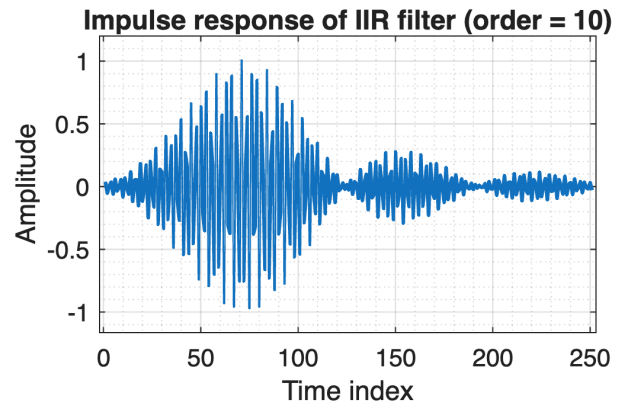
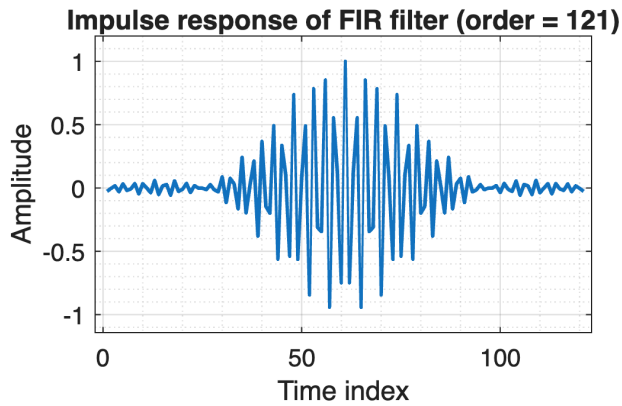
for i = 1 : (length(lo_freqs) - 1)
    design_plot_fir_iir_bandpass_filt_resp(...
        lo_freqs(i), fs3, Rp, A_dB, sprintf("Lo-f%d", i), ...
        FPASS_HALFBW, FSTOP_WIDTH_FIR, FSTOP_WIDTH_IIR);
end

```

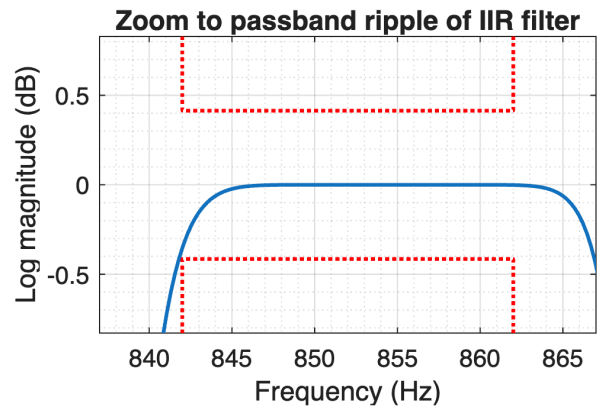
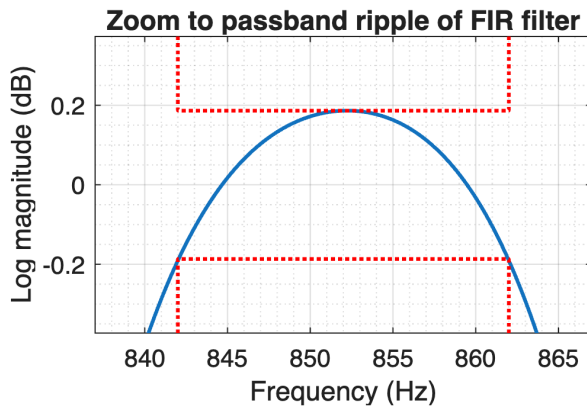
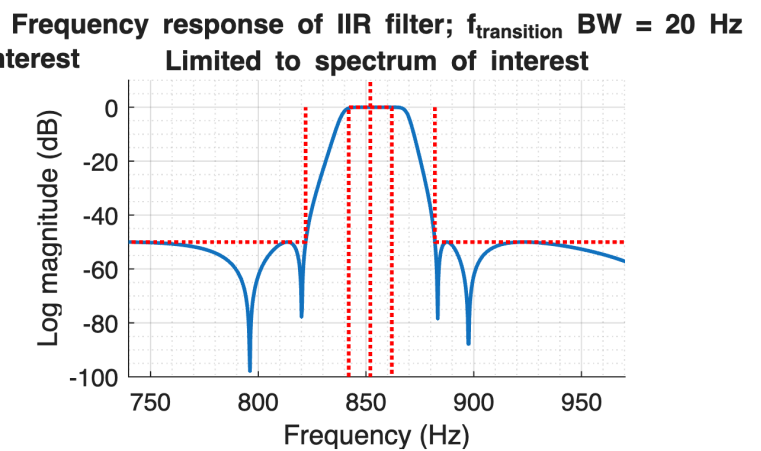
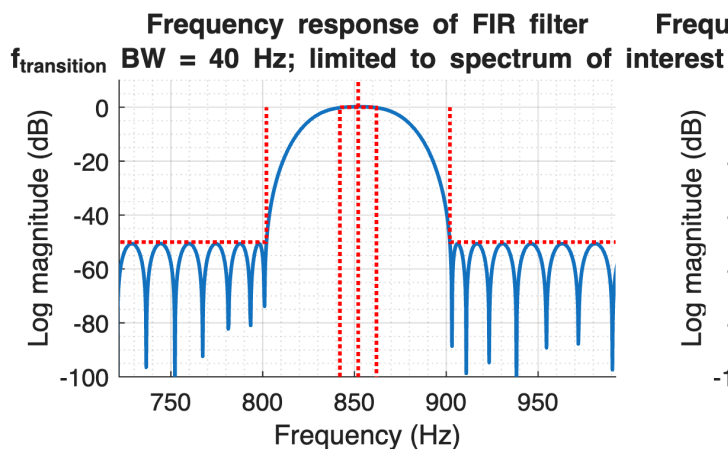
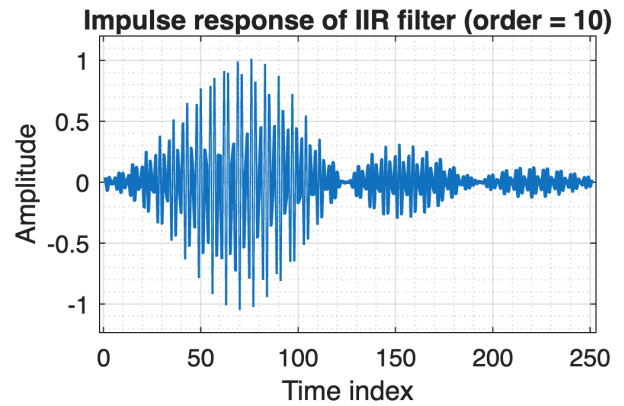
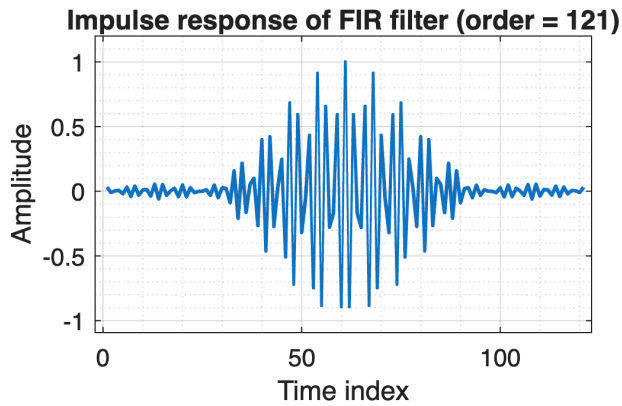
Lo-f1 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 697 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$



Lo-f2 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 770 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$



Lo-f3 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 852 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$



```

% Center frequency of 941 Hz is too close to the Nyquist frequency, and a
% bandpass filter does not work. Designing a high pass filter for just that
% one
f1 = lo_freqs(4) - FPASS_HALFBW; % Passband edge
f2 = lo_freqs(4) - (FPASS_HALFBW + FSTOP_WIDTH_FIR); % Stopband edge

delta_1 = (10 ^ (Rp / 20)) - 1;
delta_2 = 10 ^ (-A_dB / 20);

w1 = 1;
w2 = delta_1 / delta_2;

% Use the harris approximation
M = floor((fs3 / abs(f2 - f1)) * A_dB / 15);

% This seems to over-estimate the filter order; we get a much higher
% stopband attenuation than the specification
M = M - 55;

% If M is even, make it odd
if (M - 2 * ceil(M / 2) == 0)
    M = M + 1;
end

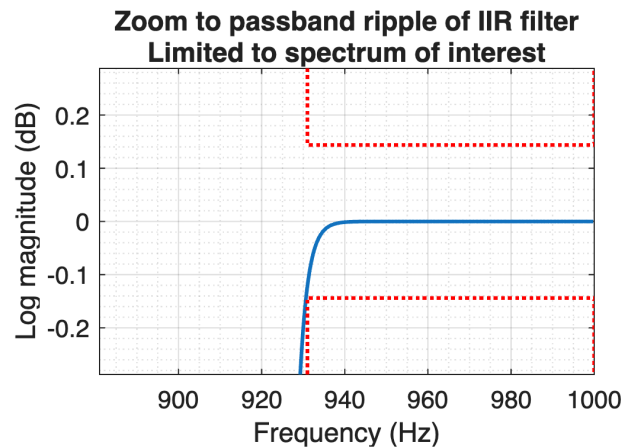
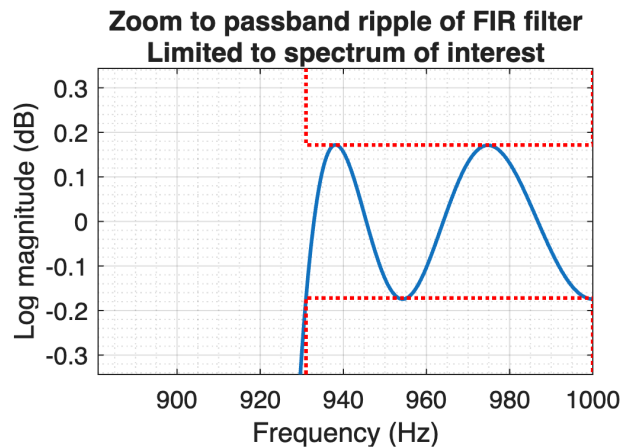
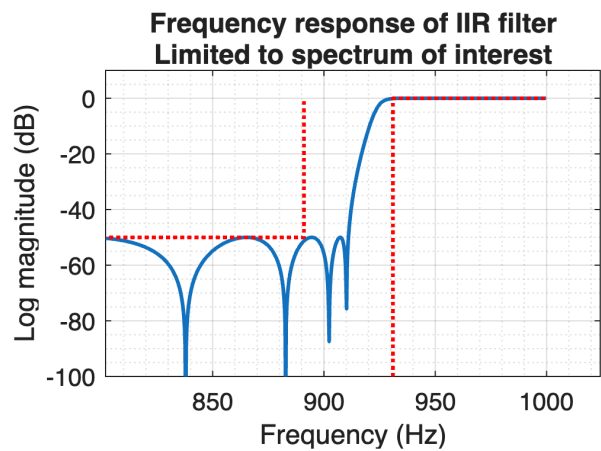
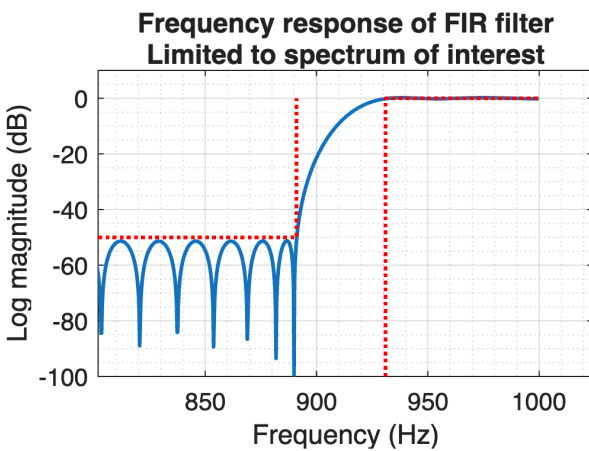
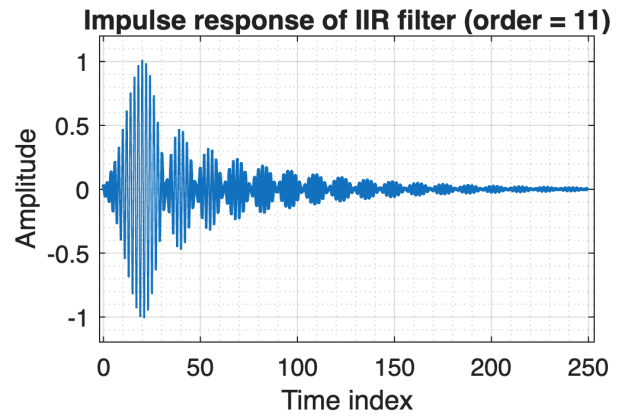
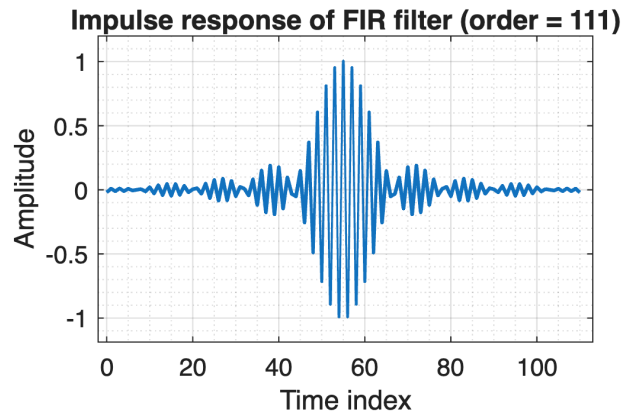
% Get the FIR filter co-efficients
h3_lo_f4_fir = ...
    firpm(M - 1, [0 f2 f1 fs3/2] / (fs3/2), [0 0 1 1], [w2 w1]);

% Get the IIR filter response from previously calculated co-efficients
h3_lo_f4_iir = filter(B3_lo_f4, A3_lo_f4, [1 zeros(1, 1250)]);

plot_fir_iir_filt_resp(...
    h3_lo_f4_fir, h3_lo_f4_iir, M, n3_lo_f4, f1, f2, fs3, A_dB, "High", ...
    sprintf(...
        "Lo-f4 High Pass Filter, f_s = %d kHz, f_{pass} = %d Hz, " + ...
        "f_{stop} = %d Hz", fs3/1000, f1, f2), ...
    true, 251);

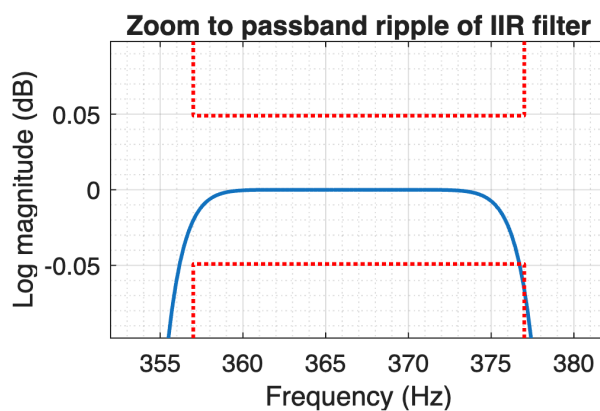
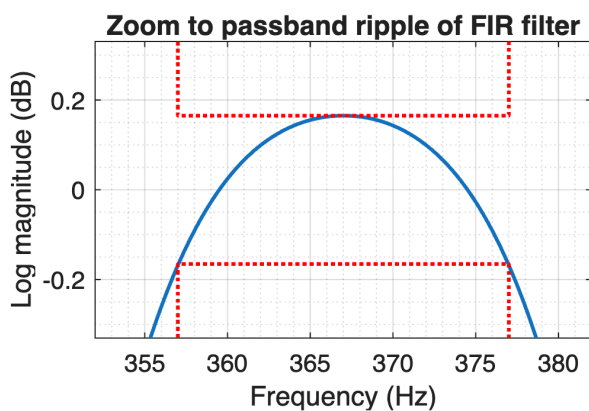
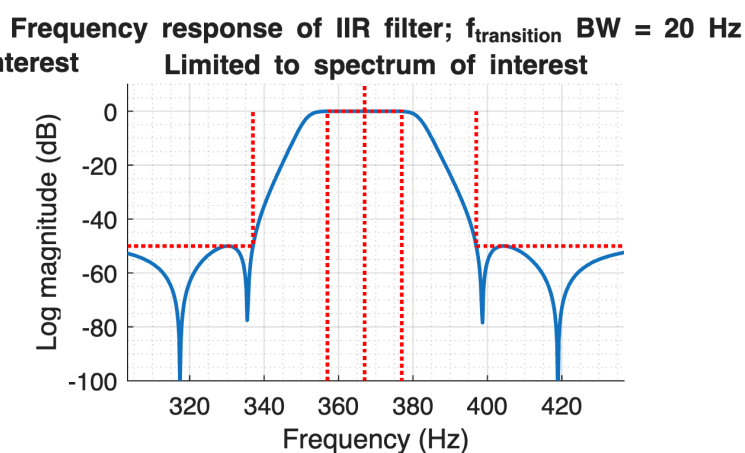
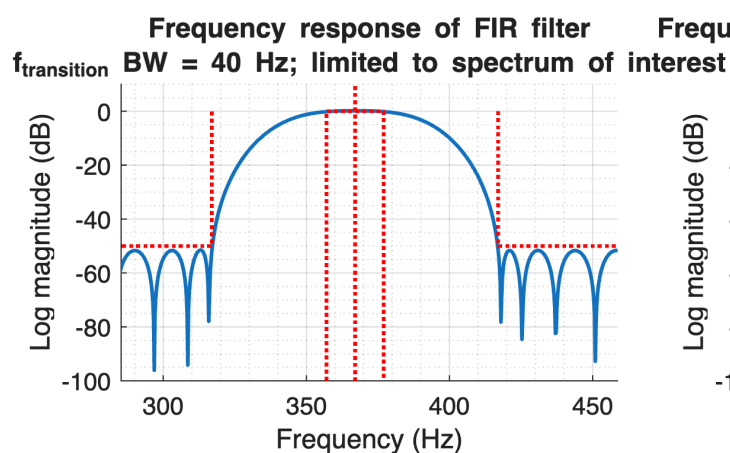
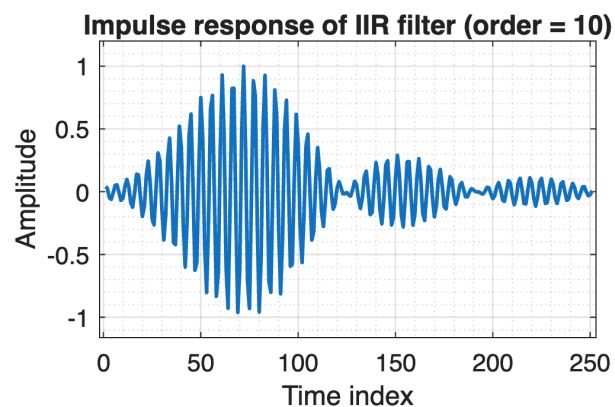
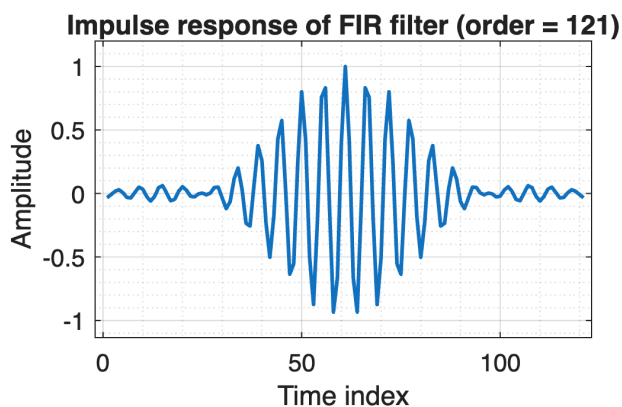
```

Lo-f4 High Pass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{pass}} = 931 \text{ Hz}$, $f_{\text{stop}} = 891 \text{ Hz}$

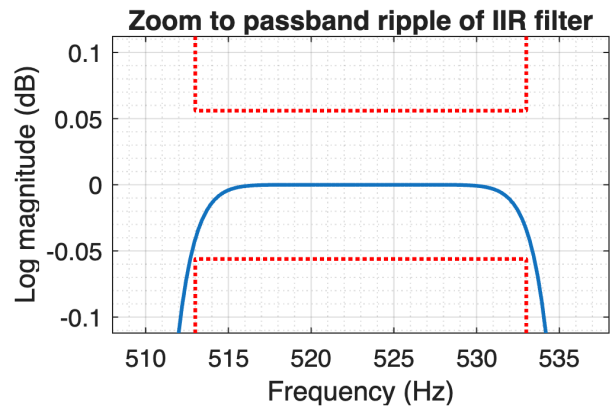
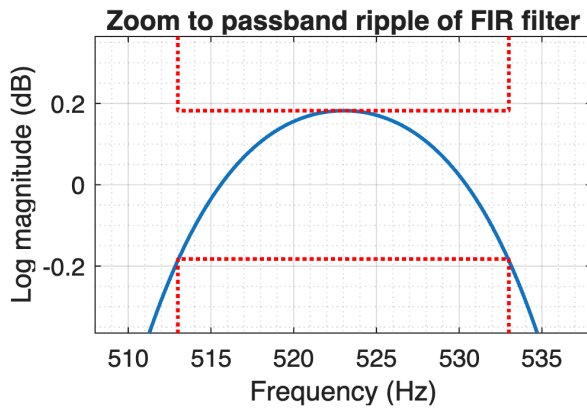
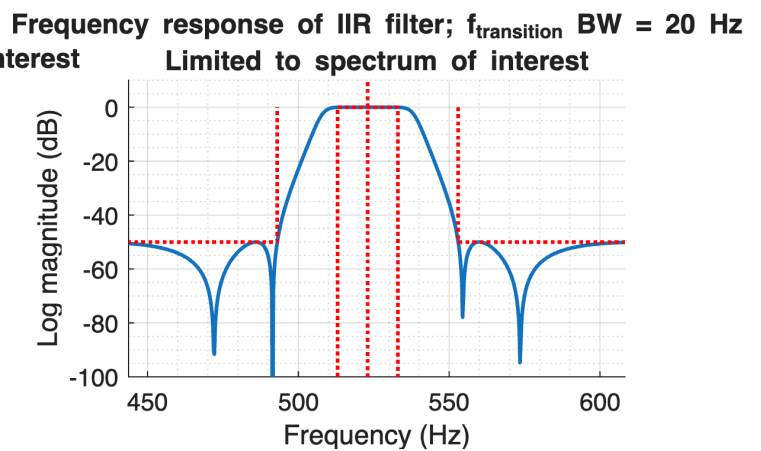
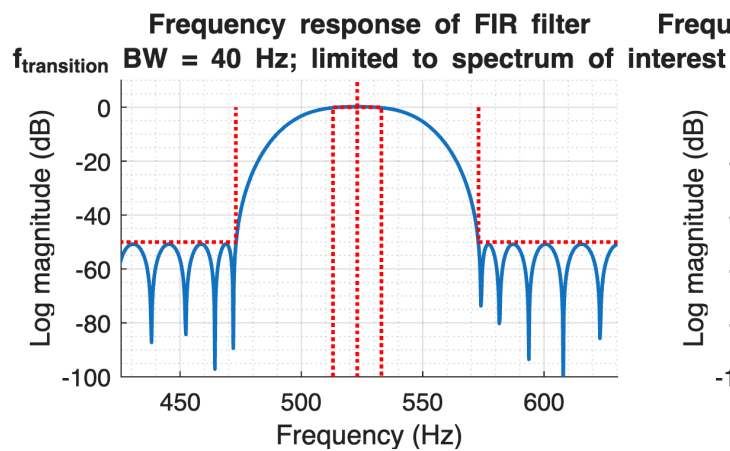
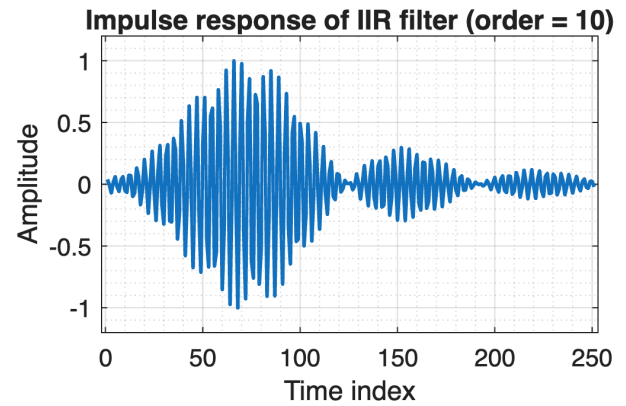
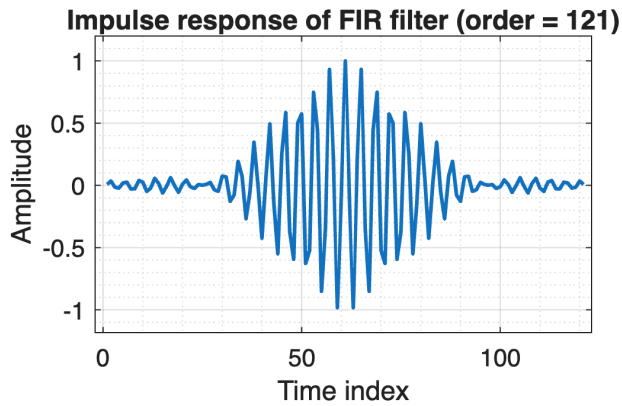


```
for i = 1:length(hi_alias_freqs)
    design_plot_fir_iir_bandpass_filt_resp(...
        hi_alias_freqs(i), fs3, Rp, A_dB, sprintf("Hi-f%d", i), ...
        FPASS_HALFBW, FSTOP_WIDTH_FIR, FSTOP_WIDTH_IIR);
end
```

Hi-f1 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 367 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$

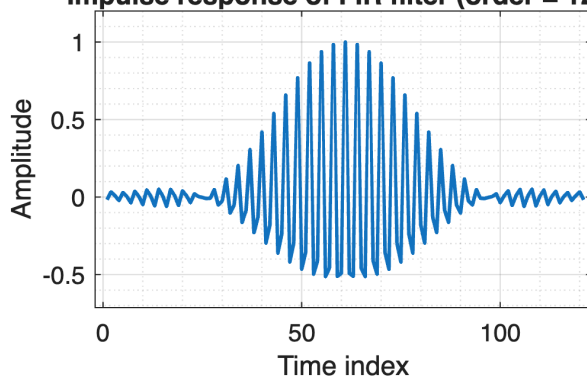


Hi-f2 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 523 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$

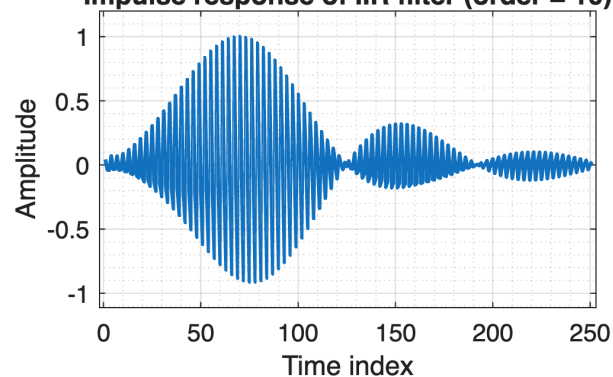


Hi-f3 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 664 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$

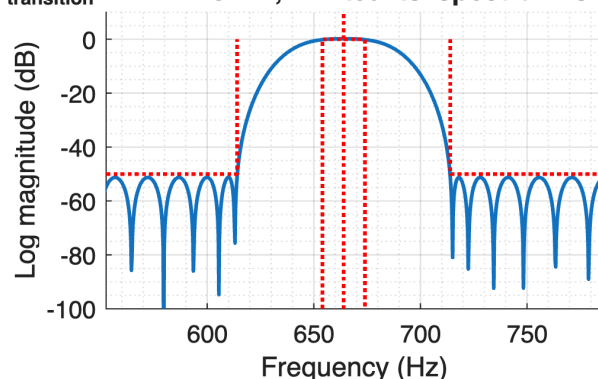
Impulse response of FIR filter (order = 121)



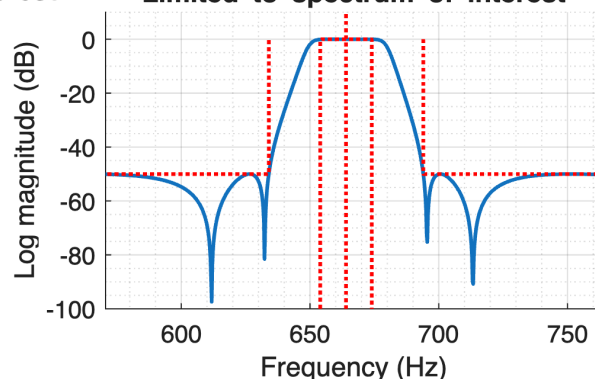
Impulse response of IIR filter (order = 10)



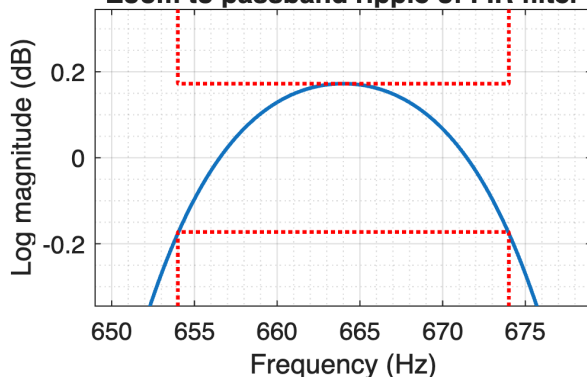
Frequency response of FIR filter
 $f_{\text{transition}}$ BW = 40 Hz; limited to spectrum of interest



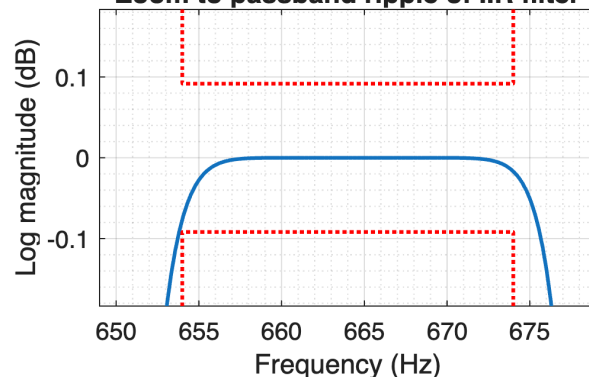
Frequency response of IIR filter; $f_{\text{transition}}$ BW = 20 Hz
 Limited to spectrum of interest



Zoom to passband ripple of FIR filter

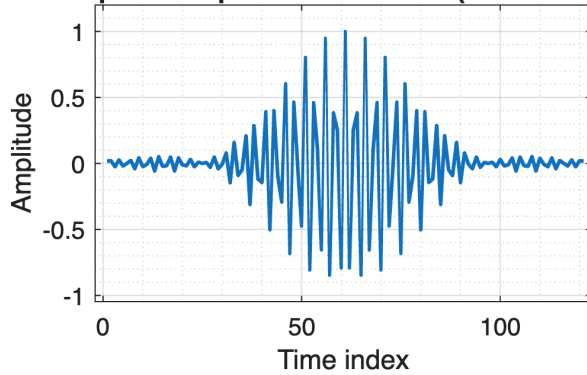


Zoom to passband ripple of IIR filter

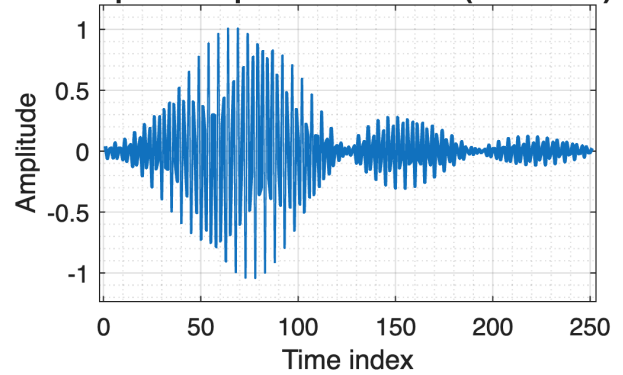


Hi-f4 Bandpass Filter, $f_s = 2 \text{ kHz}$, $f_{\text{center}} = 791 \text{ Hz}$
 $f_{\text{pass}} \text{ BW} = 20 \text{ Hz}$

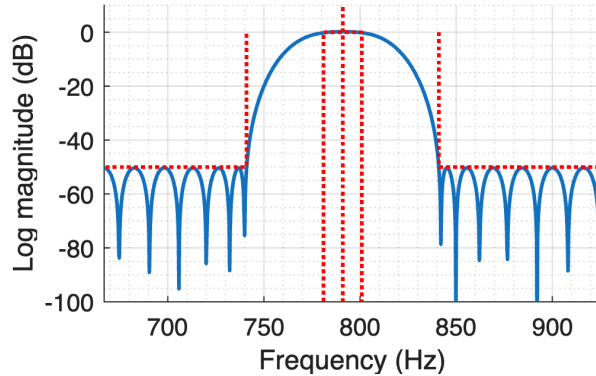
Impulse response of FIR filter (order = 121)



Impulse response of IIR filter (order = 10)



Frequency response of FIR filter
 $f_{\text{transition}}$ BW = 40 Hz; limited to spectrum of interest



Frequency response of IIR filter; $f_{\text{transition}}$ BW = 20 Hz
 Limited to spectrum of interest

